



Криптографические алгоритмы

Кунин Никита Тимофеевич

Старший преподаватель кафедры КБ-2, ИКБ

О построении шифров

Пусть X, K, Y — конечные множества открытых текстов, ключей и шифрованных текстов соответственно. Обозначим через

$$\Sigma_B = (X, K, Y, E, D, P(X), P(K))$$

вероятностную модель шифра где E и D — множества правил зашифрования и расшифрования соответственно. При этом предполагается, что априорные распределения вероятностей $P(X)$ и $P(K)$ на соответствующих множествах X и K независимы и не содержат нулевых вероятностей. Распределения $P(X)$ и $P(K)$ естественным образом индуцируют распределение вероятностей $P(Y)$ следующим образом:

$$P_Y(y) = \sum_{\substack{(x,k) \in X \times K \\ E_k(x)=y}} P_X(x) \cdot P_K(k).$$

О построении шифров

Обозначим через $K(x, y)$ множество всех таких ключей $k \in K$, для которых $E_k(x) = y$. Условная вероятность $P_{Y|X}(y|x)$ определяется естественным образом:

$$P_{Y|X}(y|x) = \begin{cases} \sum_{k \in K(x,y)} P_K(k), & K(x, y) \neq \emptyset, \\ 0, & K(x, y) = \emptyset. \end{cases}$$

О построении шифров

С помощью теоремы умножения вероятностей можно определить и условную вероятность $P_{Y|X}(y|x)$:

$$P_{X|Y}(x|y) = \frac{P_X(x) \cdot P_{Y|X}(y|x)}{P_Y(y)}.$$

Напомним, что шифр Σ_B называется совершенным по Шеннону, если для любых $x \in X$, $y \in Y$ выполнено равенство $P_{X|Y}(x|y) = P_X(x)$. Приведем эквивалентные, необходимые и достаточные условия совершенного по Шеннону шифра.

О построении шифров

ПРЕДЛОЖЕНИЕ 1. Для произвольного шифра Σ_B следующие условия эквивалентны:

- (i) для любых $x \in X, y \in Y$ выполнено равенство $P_{X|Y}(x|y) = P_X(x)$;
- (ii) для любых $x \in X, y \in Y$ выполнено равенство $P_{Y|X}(y|x) = P_Y(y)$;
- (iii) для любых $x_1, x_2 \in X, y \in Y$ выполнено равенство $P_{Y|X}(y|x_1) = P_{Y|X}(y|x_2)$.

ПРЕДЛОЖЕНИЕ 2 . Пусть Σ_B — совершенный по Шеннону шифр. Тогда для шифра Σ_B будут выполнены следующие условия:

- (i) для любых $x \in X, y \in Y$ найдётся такой ключ $k \in K$, что $E_k(x) = y$;
- (ii) для множеств X, Y и K справедливо двойное неравенство $|X| \leq |Y| \leq |K|$.

О построении шифров

ТЕОРЕМА (ДОСТАТОЧНЫЕ УСЛОВИЯ СОВЕРШЕННОГО ПО ШЕННОНУ ШИФРА). Пусть для шифра Σ_B выполнены следующие условия:

- (i) $|K(x, y)| = 1$ для любых $x \in X, y \in Y$;
- (ii) распределение вероятностей $P(K)$ является равномерным.

Тогда шифр Σ_B является совершенным по Шеннону, причём распределение вероятностей $P(Y)$ будет являться равномерным и $|K| = |Y|$.

СЛЕДСТВИЕ (ТЕОРЕМА К. ШЕННОНА). Пусть Σ_B — некоторый шифр, для которого выполнено равенство $|X| = |K| = |Y|$. Шифр Σ_B является совершенным по Шеннону тогда и только тогда, когда выполнены следующие условия:

- (i) $|K(x, y)| = 1$ для любых $x \in X, y \in Y$;
- (ii) распределение вероятностей $P(K)$ является равномерным.

О построении шифров

Общие правила для абсолютно стойких шифров в симметричном шифровании:

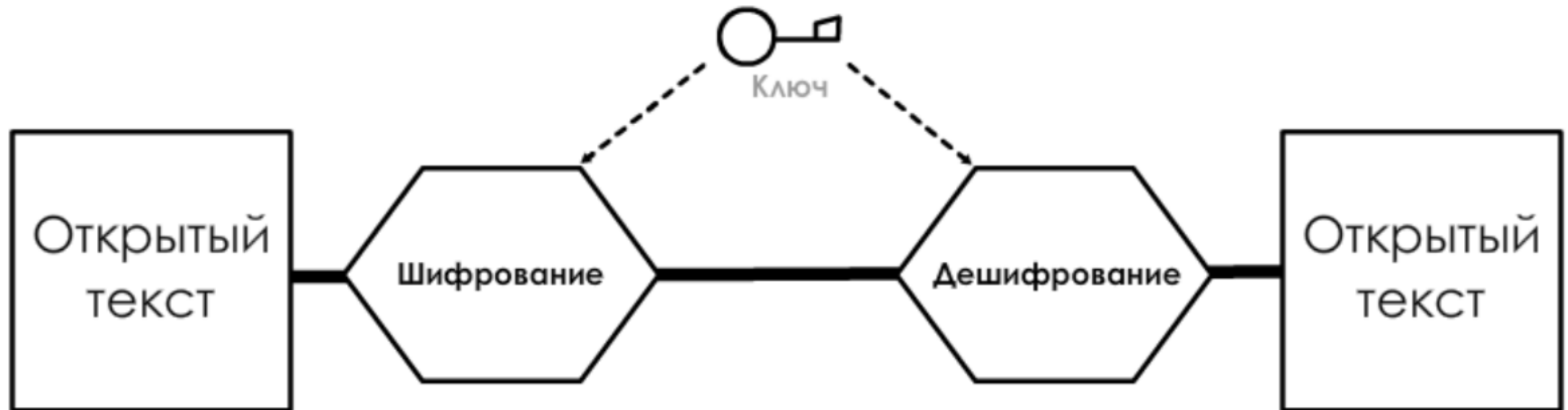
1. Одно шифрование — один уникальный ключ. Второе сообщение шифровать этим же ключом уже нельзя.
2. Ключ состоит из случайных битов, и каждый бит не зависит от другого.
3. Длина ключа должна быть равна длине сообщения или ключ должен быть чуть длиннее. Если ключ короче сообщения, которое нужно зашифровать, то абсолютной стойкости не получится.

Если в алгоритме соблюдаются все три правила, то *такой шифр взломать нельзя*.

ЗАПОМНИМ

Симметричное шифрование

Симметричное шифрование - криптографический примитив, определяемый следующими тремя компонентами: алгоритм шифрования, ключ, алгоритм расшифрования. Шифрование и расшифрование в такой схеме происходит с использованием одного и того же ключа, алгоритм шифрования обратим, а обратный ему является алгоритмом расшифрования.



Симметричное шифрование

Шифр Вернама (*Vernam cipher*) — система симметричного шифрования, изобретённая в 1917 году Гилбертом Вернамом

Шифр назван в честь телеграфиста Гильберта Вернама, который в 1917 году изобрёл, а в 1919 — запатентовал систему автоматического шифрования телетайпных сообщений.



Шифр Вернама

НП $A = 11000_2$ $\rightarrow$ $A \oplus K = 11000 \oplus 1101 = \underline{00101}$

шифрование

$K = 1101_2$

дешифрование?

$\underline{00101} \oplus 1101 = 11000$

E *A*

ВАЖНО: ключ - хаотичен!

Шифр Вернама

Для абсолютной криптографической стойкости ключ должен обладать тремя критически важными свойствами:

1) Иметь случайное дискретное равномерное распределение

$$P_k(K) = 1/2^N, \quad k - \text{ключ}, \quad N - \text{кол-во bit в ключе}$$

2) Совпадать по размеру с заданным открытым текстом

3) Применяться ОДИН РАЗ

Шифр Вернама

Например, шифрование букв латинского алфавита шифром Вернама по модулю $m = 26$
($A = 0, B = 1, \dots, Z = 25$):

Ключ: EVTIQWXQV VORMCXREPYZ

Открытый текст: ALLSWELLTHATENDSWELL (All's well that ends well)

Шифротекст: EGEAMAIBOCOIQPAJATJK

Виженер?

🤔

не совсем...

Шифр Вернама

Ключ: EVTIQWXQVVOPMCXREPYZ

Открытый текст: ALLSWELLTHATENDSWELL (All's well that ends well)

Шифротекст: EGEAMAIBOCOIQPAJATJK

Без знания ключа такое сообщение не поддаётся анализу. Даже если бы можно было перепробовать все ключи, в качестве результата мы получили бы все возможные сообщения данной длины плюс колоссальное количество бессмысленных дешифровок, выглядящих как беспорядочное нагромождение символов.

Но и среди осмысленных дешифровок не было бы никакой возможности выбрать искомую. Когда случайная последовательность (ключ) сочетается с неслучайной (открытым текстом), результат этого (шифротекст) оказывается совершенно случайным и, следовательно, лишённым тех статистических особенностей, которые могли бы быть использованы для анализа шифра.

Шифр Вернама

ОСОБЕННОСТЬ

Возможность подмены исходного текста.

Так для любых двух открытых текстов одинаковой длины T и T' существуют ключи K и K' , переводящие их в один и тот же шифротекст C .

$$K' = C \oplus T'$$

Шифр Вернама

НЕДОСТАТКИ

1
Для работы шифра Вернама необходима истинно случайная последовательность (ключ).

Под рукой всегда необходимо иметь достаточное количество ключей, которые могут понадобиться в дальнейшем для шифрования больших объёмов открытого текста.

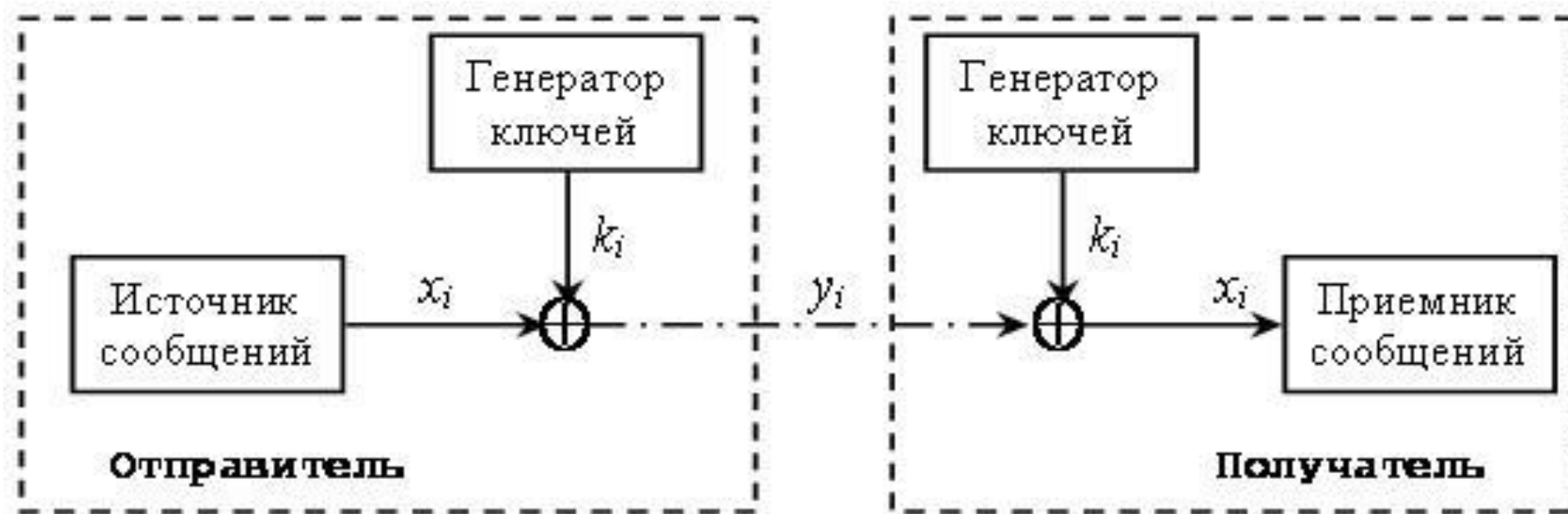
5
Шифр Вернама чувствителен к любому нарушению процедуры шифрования.

2
Недостатком использования шифра Вернама является отсутствие подтверждения подлинности и целостности сообщения. Получатель не может удостовериться в отсутствии повреждений или в подлинности отправителя.

4
Проблемой является защищённая передача последовательности и сохранение её в тайне.

Поточные шифры

Поточный шифр (stream cipher) выполняет преобразование входного сообщения по одному биту (или байту) за операцию. Поточный *алгоритм* шифрования устраняет необходимость разбивать сообщение на *целое число* блоков достаточно большой длины, следовательно, он может работать в реальном времени. Таким образом, если передается *поток* символов, каждый символ может шифроваться и передаваться сразу.



Поточные шифры

Генератор ключей выдает поток битов k_i , которые будут использоваться в качестве гаммы. Источник сообщений генерирует биты открытого текста x_i , которые складываются по модулю 2 с гаммой, в результате чего получаются биты зашифрованного сообщения y_i :

$$y_i = x_i \oplus k_i, \quad i = 1, 2, 3, \dots, n$$

Чтобы из шифротекста $y_1, y_2, \dots, y_n$ восстановить сообщение $x_1, x_2, \dots, x_n$, необходимо сгенерировать точно такую же ключевую последовательность $k_1, k_2, \dots, k_n$, что и при шифровании, и использовать для расшифрования формулу

$$x_i = y_i \oplus k_i, \quad i = 1, 2, 3, \dots, n$$

Поточные шифры

Обычно исходное сообщение и ключевая последовательность представляют собой **независимые потоки *bit***. Таким образом, так как шифрующее (и расшифрующее) преобразование для всех поточных шифров одно и то же, **они должны различаться только способом построения генераторов ключей**. Получается, что **безопасность** системы полностью **зависит от свойств генератора потока ключей**.

ВАЖНО!!!

Генераторы псевдослучайных чисел

Целью использования *генераторов псевдослучайных чисел* является получение "бесконечного" ключевого слова, располагая относительно малой длиной самого ключа.

Генератор псевдослучайных чисел создает последовательность битов, **похожую на случайную.**

Генераторы псевдослучайных чисел

Для использования в криптографических целях *генератор псевдослучайных чисел* должен обладать следующими свойствами:

1. период последовательности должен быть **очень большой**;
2. порождаемая последовательность должна быть "почти" неотличима от действительно случайной;
3. вероятности порождения различных значений должны быть в точности равны;
4. для того, чтобы только законный получатель мог расшифровать сообщение, следует при получении потока ключевых битов k_i использовать и учитывать некоторый секретный ключ, причем вычисление числа k_{i+1} по известным предыдущим элементам последовательности k_i без знания ключа **должно быть трудной задачей**

Линейный конгруэнтный генератор

Для вычисления очередного числа k_i использует формулу

$$k_i = (a * k_{i-1} + b) \bmod c,$$

где a , b , c — некоторые константы, а k_{i-1} — предыдущее псевдослучайное число. Для получения k_1 задается начальное значение k_0 .

Линейный конгруэнтный генератор

$$\textcircled{11} \quad a=5, b=3, c=11$$

$$k_0 = 1$$

$$\mathbb{Z}_m = \mathbb{Z}_{11} \Rightarrow [0 \dots 10]$$

$$k_1 = (5 \times 1 + 3) \bmod 11 = 8$$

$$k_2 = (5 \times 8 + 3) \bmod 11 = 10$$

$$k_3 = 9, k_4 = 4, k_5 = 1$$

$$k_6 = 8, k_7 = 10, k_8 = 9$$

Итог: 8, 10, 9, 4, 1, 8, 10, 9 ... (8, 10, 9, 4, 1) в цикле!

хорошо плохo

Линейный конгруэнтный генератор

Если параметры a , b и c выбраны правильно, то генератор будет порождать случайные числа с максимальным периодом, равным c . При программной реализации значение c обычно устанавливается равным 2^{b-1} или 2^b , где b — длина слова ЭВМ в битах.



Их простота и высокая скорость получения псевдослучайных значений.



В криптографических целях их **нельзя рекомендовать** к использованию, так как специалисты по криптоанализу научились восстанавливать всю последовательность ПСЧ по нескольким значениям.

Линейный конгруэнтный генератор

Например, предположим, что противник может определить значения k_0 , k_1 , k_2 , k_3 . Тогда:

$$\begin{cases} k_1 = (a \times k_0 + b) \bmod c \\ k_2 = (a \times k_1 + b) \bmod c \\ k_3 = (a \times k_2 + b) \bmod c \end{cases}$$

Решив систему из этих трех уравнений, можно найти a , b и c

Метод Фибоначчи с запаздыванием

Наибольшую популярность фибоначчиевы датчики получили в связи с тем, что скорость выполнения арифметических операций с вещественными числами сравнялась со скоростью целочисленной арифметики, а фибоначчиевы датчики естественно реализуются в *вещественной арифметике*.

$$k_i = \begin{cases} k_{i-a} - k_{i-b}, & \text{если } k_{i-a} \geq k_{i-b} \\ k_{i-a} - k_{i-b} + 1, & \text{если } k_{i-a} < k_{i-b} \end{cases}$$

где k_i — вещественные числа из диапазона $[0, 1]$, a, b — целые положительные числа, параметры генератора. Для работы фибоначчиеву датчику требуется знать $\max\{a, b\}$ предыдущих сгенерированных случайных чисел.

Метод Фибоначчи с запаздыванием

III $a=4, b=1$

$k_0 = 0,1$	$\underline{k_5} = k_1 - k_4 = 0,7 - 0,5 = 0,2$	$k_7 = 0,8$
$k_1 = 0,7$	$k_6 = k_2 - \underline{k_5} = 0,3 - 0,2 = 0,1$	$k_8 = 0,7$
$k_2 = 0,3$	$\vdots$	$k_9 = 0,5$
$k_3 = 0,9$	$\underline{k_{13}} = k_9 - k_{12} = 1$	$k_{10} = 0,6$
$k_4 = 0,5$	$k_{14} = k_{10} - \underline{k_{13}} = 0,6$	$k_{11} = 0,2$
		$k_{12} = 0,5$

рекуррентность

Метод Фибоначчи с запаздыванием

Рекомендуемые параметры a и b :

$$(a, b) = (55, 24), (17, 5) \text{ или } (97, 33).$$

Качество получаемых случайных чисел зависит от значения константы a : **чем оно больше, тем выше размерность пространства**, в котором сохраняется равномерность случайных векторов, образованных из полученных случайных чисел. В то же время с увеличением величины константы a увеличивается объём используемой алгоритмом памяти.

ГПСЧ на основе алгоритма BBS

Вначале выбираются два больших простых числа p и q . Числа p и q должны быть оба *сравнимы* с 3 по модулю 4 (то есть при делении p и q на 4 должен получаться одинаковый *остаток* 3).

Далее вычисляется число $M = p * q$, называемое **целым числом Блюма**. Затем выбирается другое случайное *целое число* x , взаимно простое (то есть не имеющее общих делителей, кроме единицы) с M . Вычисляем $x_0 = x^2 \bmod M$ (x_0 называется стартовым числом генератора).

На каждом n -м шаге работы генератора вычисляется $x_{n+1} = x_n^2 \bmod M$. Результатом n -го шага является один (обычно младший) *bit* числа x_{n+1} . Иногда в качестве результата принимают *bit* чётности, то есть количество единиц в двоичном представлении элемента. Если количество единиц в записи числа четное – *bit* четности принимается равным 0, нечетное – *bit* четности принимается равным 1.

ГПСЧ на основе алгоритма BBS

$$(17) \quad p=11, q=19$$

$$\begin{aligned} (11 \bmod 4 = 3) \\ (19 \bmod 4 = 3) \end{aligned}$$

$$M = p \times q = 209$$

$$x = 3$$

$$x_0 = x^2 \bmod M = 9$$

$$R_{nd} = \text{LSB}(x_i) \quad [\text{ИТОГ: } 10000001011]$$

$$x_1 = 9^2 \bmod 209 = 81 \quad (01010001 \Rightarrow \text{LSB} = 1)$$

$$x_2 = 81^2 \bmod 209 = 82 \quad (\text{LSB} = 0)$$

$$x_3 = 82^2 \bmod 209 = 36 \quad (\text{LSB} = 0)$$

$\vdots$

$$x_9 = 196^2 \bmod 209 = 169 \quad (\text{LSB} = 1)$$

$$x_{10} = 169^2 \bmod 209 = 137 \quad (\text{LSB} = 1)$$

ГПСЧ на основе алгоритма BBS

Для получения n -го числа последовательности не нужно вычислять все предыдущие n чисел x_i . Оказывается x_n можно сразу получить по формуле

$$x_n = x_0^{2^n \bmod ((p-1)(q-1))} \bmod M$$

Возможность "прямого" получения x_n позволяет использовать *алгоритм BBS* при потоковой шифрации, например, для файлов с произвольным доступом или фрагментов файлов с записями *базы данных*.

ГПСЧ на основе алгоритма BBS



Безопасность алгоритма BBS основана на сложности разложения большого числа M на множители. Утверждается, что если M достаточно велико, его можно даже не держать в секрете; до тех пор, пока M не разложено на множители, никто не сможет предсказать выход генератора ПСЧ. Это связано с тем, что задача разложения чисел вида $n = pq$ (p и q — простые числа) на множители является вычислительно очень трудной, если мы знаем только n , а p и q — большие числа, состоящие из нескольких десятков или сотен бит **(задача факторизации)**.

⇒ **ВАЖНО!!!**



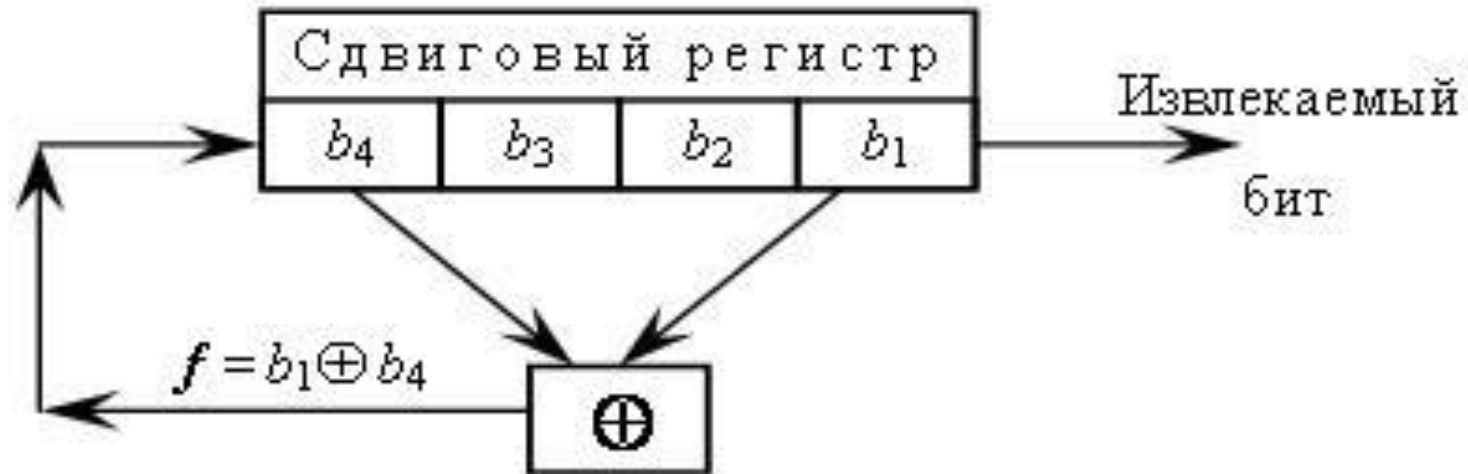
Он недостаточно быстр, что не позволяет использовать его во многих областях, например, при вычислениях в реальном времени, а также, к сожалению, и при потокном шифровании.



ГПСЧ на основе сдвиговых регистров с обратной связью

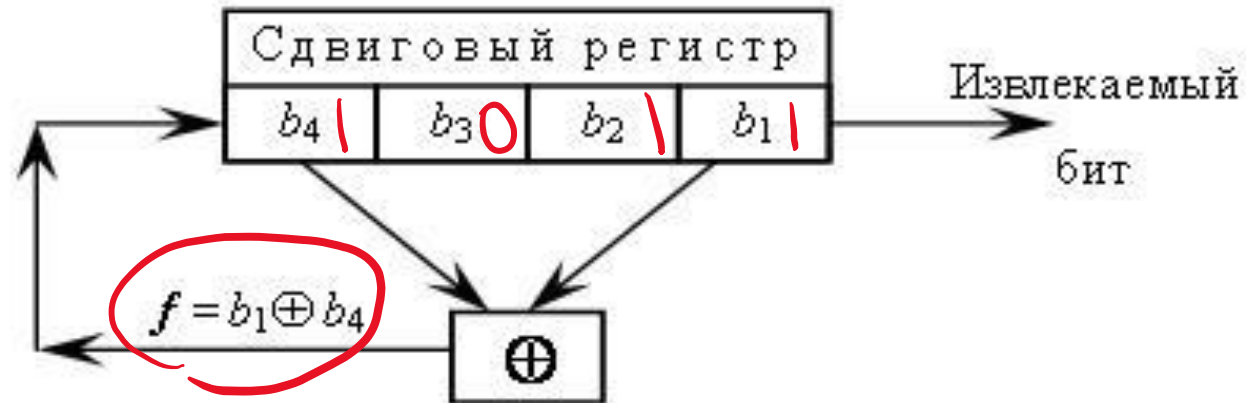


Linear feedback shift register – LFSR



Обратная связь в этом устройстве реализуется просто как сумма по модулю 2 всех (или некоторых) битов регистра. Биты, которые участвуют в обратной связи, образуют *отводную последовательность*.

Linear feedback shift register – LFSR



- ① 1011, $f = b_1 \oplus b_4 = 0 \rightarrow$ Извлекаем 1
- ② 0101, $f = \underline{\underline{1}} \rightarrow$ Извлекаем 1
- ③ 1010, $f = \underline{\underline{1}} \rightarrow$ Извлекаем 0

и т.д.

Линейный сдвиговый регистр размером n бит может находиться в одном из $2^n - 1$ состояний. Поэтому теоретически такой регистр может генерировать псевдслучайную последовательность с максимальным периодом $2^n - 1$

Алгоритм RC4

RC4 — это потоковый шифр, разработанный Рональдом Ривестом в 1987 году. Он широко использовался в различных протоколах безопасности, включая **SSL/TLS** и **WEP**, благодаря своей простоте и скорости.

Основные аспекты **RC4**:

1. Алгоритм: **RC4** — это алгоритм симметричного шифрования, который использует один и тот же ключ как для шифрования, так и для расшифрования.
2. Ключ: Поддерживает длину ключа от 1 до 256 бит. Чем длиннее ключ, тем выше уровень безопасности.
3. Потоковый шифр: Вместо того чтобы шифровать блоки данных, **RC4** работает с потоками битов, что позволяет шифровать данные по одному байту за раз.

Алгоритм RC4

Принцип работы

1. Инициализация: Алгоритм начинается с создания ключевого массива (S), который инициализируется значениями от 0 до 255. Затем он перемешивается на основе ключа.
2. Генерация псевдослучайного потока: После инициализации ключевого массива алгоритм генерирует поток псевдослучайных байтов (псевдослучайный ключ) для шифрования.
3. Шифрование: Для шифрования каждый байт открытого текста (P) комбинируется с соответствующим байтом псевдослучайного ключа (K) с использованием операции XOR:

$$C[i] = P[i] \text{ XOR } K[i]$$

где **C[i]** — зашифрованный байт, **P[i]** — байт открытого текста и **K[i]** — байт из псевдослучайного ключа.

Алгоритм RC4

1. Инициализация ключевого массива

Входные данные: Ключ (длина от 1 до 256 бит).

Создание массива S: Инициализируется массив S размером 256, содержащий значения от 0 до 255.

Перемешивание: Используя ключ, массив S перемешивается.

Для i от 0 до 255:

$S[i] = i$

$j = 0$

Для i от 0 до 255:

$j = (j + S[i] + K[i \% \text{длина_ключа}]) \% 256$

Меняем местами $S[i]$ и $S[j]$

Алгоритм RC4

2. Генерация псевдослучайного потока

Инициализация: Устанавливаются два индекса i и j .

Генерация ключевого потока: Для каждого байта выходных данных генерируется байт псевдослучайного ключа K .

$i = 0$

$j = 0$

Для каждого байта P (открытого текста):

$i = (i + 1) \% 256$

$j = (j + S[i]) \% 256$

Меняем местами $S[i]$ и $S[j]$

$K = S[(S[i] + S[j]) \% 256]$

Алгоритм RC4

3. Шифрование/Дешифрование

Процесс: Открытый текст P шифруется путем побитового XOR с ключевым потоком K.

Для каждого байта P:
$$C[i] = P[i] \text{ XOR } K$$

4. Результат

Выходные данные: Зашифрованный текст C.

RC4 использует одну и ту же процедуру для шифрования и дешифрования, поскольку операция XOR является симметричной. Это означает, что для расшифровки зашифрованного текста нужно просто применить ту же процедуру с тем же ключом.

Алгоритм RC4

Преимущества

Скорость: RC4 очень быстрый и эффективный, особенно для программного обеспечения.

Простота реализации: Алгоритм легко реализовать на разных языках программирования.

Недостатки

Уязвимости: Со временем были обнаружены уязвимости в RC4, включая статистические атаки, которые могут позволить злоумышленникам восстановить часть открытого текста. Это привело к его постепенному выведению из употребления в современных протоколах безопасности.

Предсказуемость: Псевдослучайный поток может быть предсказуемым при недостаточной длине ключа или неосторожном использовании.

Хотя **RC4** был популярным выбором для потокового шифрования, его уязвимости сделали его менее предпочтительным в современных системах безопасности. Рекомендуется использовать более безопасные алгоритмы, такие как **AES**, для защиты данных.

Алгоритм Salsa20

Salsa20 — это потоковый шифр, разработанный Даниэлем Бернштейном в 2005 году. Он был создан с акцентом на высокую производительность и безопасность, что сделало его одним из самых популярных потоковых шифров на сегодняшний день.

Вот основные аспекты **Salsa20**:

1. Алгоритм: **Salsa20** является симметричным потоковым шифром, который использует один и тот же ключ для шифрования и расшифрования.
2. Длина ключа: Поддерживает длину ключа 128 или 256 бит.
3. Устойчивость: Разработан с учетом устойчивости к различным атакам, включая дифференциальные и линейные атаки.

Алгоритм Salsa20

Принцип работы

Salsa20 использует комбинацию матричных операций, которые обеспечивают высокую скорость и безопасность.

Основные этапы работы алгоритма:

1. Инициализация:

- *Ключ и инициализационный вектор (IV)*: На вход подаются ключ и IV (обычно 64 бита).
- *Создание состояния*: Формируется начальное состояние, состоящее из 16 32-битных слов. Это состояние включает в себя константу, ключ и IV.

2. Генерация псевдослучайного потока: Используя начальное состояние, **Salsa20** выполняет 20 раундов вычислений, которые включают в себя операции сложения, XOR и вращения. Эти операции обеспечивают диффузию и смешивание данных. Результатом работы является выходной поток, который используется для шифрования данных.

3. Шифрование: Для шифрования открытого текста P используется полученный поток ключей K . Каждый байт открытого текста комбинируется с соответствующим байтом ключа с помощью операции XOR. Это позволяет получить зашифрованный текст C .

Алгоритм Salsa20

Генерация псевдослучайного потока в **Salsa20** — это ключевая часть алгоритма, которая обеспечивает создание безопасного и непредсказуемого потока ключей для шифрования. Процесс включает в себя несколько этапов, начиная с инициализации состояния и заканчивая выполнением раундов вычислений.

1. Инициализация состояния

На первом этапе создается начальное состояние, состоящее из 16 32-битных слов. Это состояние формируется на основе ключа и инициализационного вектора (IV):

❑ Ключ:

Длина ключа может составлять 128 или 256 бит (4 или 8 32-битных слов).

❑ Инициализационный вектор (IV):

Обычно 64 бита (два 32-битных слова).

❑ Константа:

Специальная константа, которая используется для инициализации состояния.

Формат состояния выглядит следующим образом:



```
state[0]  = 0x61707865  // "expa"
state[1]  = 0x3320646e  // "nd 3"
state[2]  = 0x79622d32  // "2-by"
state[3]  = 0x6b206574  // "te k"
state[4]  = ключ[0]
state[5]  = ключ[1]
state[6]  = ключ[2]
state[7]  = ключ[3]
state[8]  = ключ[4]
state[9]  = ключ[5]
state[10] = ключ[6]
state[11] = ключ[7]
state[12] = IV[0]
state[13] = IV[1]
state[14] = 0           // Номер блока
state[15] = 0           // Номер блока
```

Алгоритм Salsa20

2. Выполнение раундов

Для генерации псевдослучайного потока **Salsa20** использует 20 раундов вычислений. Каждый раунд включает в себя несколько операций, которые обеспечивают диффузию и смешивание данных.

Основные операции включают:

❑ Сложение (Addition):

Операция сложения выполняется по модулю 2^{32} .

❑ XOR:

Логическая операция XOR используется для смешивания значений.

❑ Вращение (Rotation):

Операции вращения обеспечивают дополнительное смешивание.

Процесс обновления состояния в раунде можно описать следующим образом:

Для каждого раунда:

```
state[4] += state[0]
state[12] ^= state[4]
state[12] = (state[12] << 16) | (state[12] >> 16) // Вращение
state[8] += state[12]
state[4] ^= state[8]
state[4] = (state[4] << 12) | (state[4] >> 20) // Вращение
...
```

Эти операции выполняются для всех 16 слов состояния, и в результате после 20 раундов генерируется выходной поток ключей.

Алгоритм Salsa20

3. Вывод ключевого потока

После завершения раундов **Salsa20** генерирует выходной поток, который можно использовать для шифрования. Этот поток XOR'ится с открытым текстом:

Для каждого байта P :

$$K[i] = \text{state}[i \% 16]$$
$$C[i] = P[i] \text{ XOR } K[i]$$

Алгоритм Salsa20

Преимущества

- Скорость: **Salsa20** очень быстрый, особенно в программных реализациях, и обеспечивает высокую производительность на современных процессорах.
- Безопасность: Алгоритм был подвергнут тщательному анализу и считается безопасным. Он устойчив к различным видам атак.
- Простота реализации: **Salsa20** относительно легко реализовать, что делает его популярным выбором для разработчиков.

Применение

Salsa20 используется в различных протоколах и приложениях, включая:

- TLS/SSL: Применяется в некоторых реализациях для защиты сетевых соединений.
- VPN: Используется в некоторых протоколах виртуальных частных сетей.
- Файловое шифрование: Применяется для шифрования данных на дисках и в облачных хранилищах.

Блочное шифрование

Блочный шифр — разновидность **симметричного шифра**, оперирующего группами бит фиксированной длины — блоками, характерный размер которых меняется в пределах 64–256 бит.



Блочное шифрование

Блок-шифр – это **детерминированный** шифр (KeyGen, Enc, Dec) с $\mathcal{K}, \mathcal{X} := \mathcal{M} = \mathcal{C}$, и эффективной функцией

$$\text{Enc} = f(k, \cdot) : \mathcal{X} \rightarrow \mathcal{X}.$$

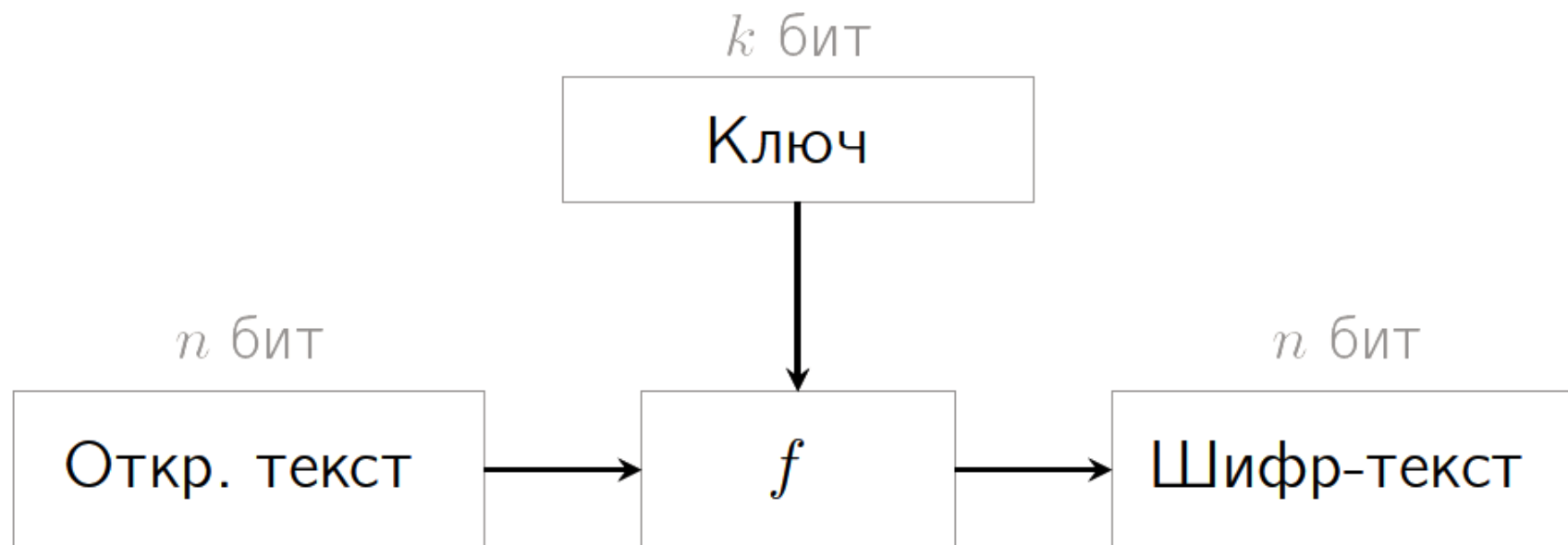
При этом выполняется

- корректность $\implies f(k, \cdot)$ – биекция для всех $k \in \mathcal{K}$
- $|\mathcal{X}| < \infty$.

То есть, $f(k, \cdot)$ – перестановка на $\mathcal{X}$.

Блочное шифрование

Шифрование блока



Примеры:

- AES: $n = 128$, $k = 128, 192, 256$
- ГОСТ 34.12-2018: $n = 128$, $k = 256$ (Кузнечик)

Блочное шифрование

Хронология развития ->

- **70'е:** IBM публикует шифр Lucifer. $k = 128, n = 128$
- **'76:** DES – стандарт $k = 56, n = 64$
- **'98:** 3DES – стандарт $k = 168, n = 64$
- **'00:** конкурс AES побеждает Rijndael $k = \{128, 192, 256\}, n = 128$

Российские стандарты:

- **'89:** ГОСТ 28147-89 $k = 256, n = 64$
- **'15 :** ГОСТ Р 34.12-2015/2018, RFC 7801 $k = 256, n = 128$

Блочное шифрование

Что такое S-box?

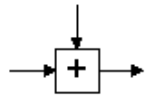
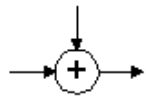
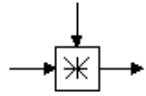
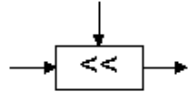
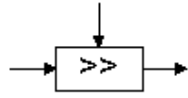
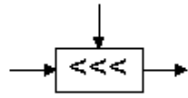
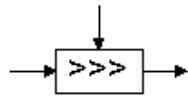
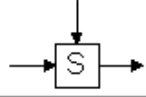
$S := \{0, 1\}^n \rightarrow \{0, 1\}^m$ — таблица подстановки

- Реализована таблицей поиска (Lookup table)
- В одном блочном шифре может быть использовано несколько S-боксов
- S-бокс не должен содержать фиксированных точек:
 $S(x) \neq x, S(x) \neq \bar{x} \forall x$
- S-бокс не должен быть линейной или аффинной булевой функцией

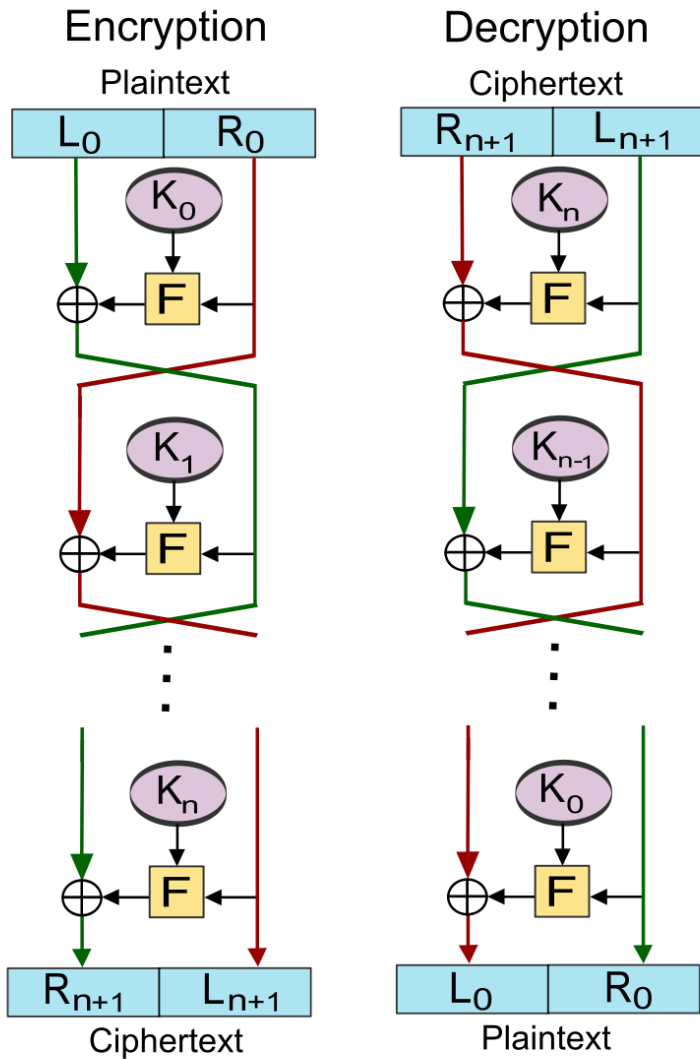
Требования к блочному алгоритму шифрования

1. Алгоритм должен обеспечивать высокий уровень стойкости, и эта стойкость не должна основываться на сохранении в тайне самого алгоритма.
2. Незначительное изменение исходного сообщения должно приводить к существенному изменению зашифрованного сообщения даже при использовании одного и того же ключа.
3. Алгоритм должен успешно противостоять атакам по выбранному тексту, то есть таким, чтобы нельзя было узнать ключ, даже зная достаточно много пар (зашифрованное сообщение, незашифрованное сообщение), полученных при шифровании с использованием данного ключа.
4. Алгоритм шифрования должен иметь возможность быть реализованным на различных платформах, которые предъявляют различные требования.
5. Алгоритм должен использовать простые операции, которые эффективны на микропроцессорах, т.е. исключаящее или, сложение, табличные подстановки, умножение по модулю. Не должно использоваться сдвигов переменной длины, побитных перестановок или условных переходов.
6. Алгоритм должен эффективно реализовываться на специализированной аппаратуре, предназначенной для выполнения операций шифрования и расшифрования, то есть реализация алгоритма в виде электронных устройств должна быть экономичной.
7. Алгоритм шифрования должен быть применим во многих приложениях. Алгоритм должен быть эффективен при шифровании файлов данных или большого потока данных, при создании определенного количества случайных битов, а также должна быть возможность его использования для формирования односторонней хеш-функции.
8. Алгоритм должен быть простым для написания кода, чтобы минимизировать вероятность программных ошибок. Также это дает возможность анализа и уменьшает закрытость алгоритма.
9. Алгоритм должен допускать любую случайную строку битов нужной длины в качестве возможного ключа (это называется *иметь плоское пространство ключей*). Не должно быть "слабых" ключей, облегчающих криптоанализ.
10. Алгоритм должен легко модифицироваться для различных уровней безопасности и удовлетворять как минимальным, так и максимальным требованиям.

Условные обозначения операций для блочных алгоритмов

Биективные математические функции		
	Сложение	$X' = X + V$
	Исключающее ИЛИ	$X' = X \text{ XOR } V$
	Умножение по модулю $2^N + 1$	$X' = (X * V) \bmod (2^N + 1)$
	Умножение по модулю 2^N	$X' = (X * V) \bmod (2^N)$
Битовые сдвиги		
	Арифметический сдвиг влево	$X' = X \text{ SHL } V$
	Арифметический сдвиг вправо	$X' = X \text{ SHR } V$
	Циклический сдвиг влево	$X' = X \text{ ROL } V$
	Циклический сдвиг вправо	$X' = X \text{ ROR } V$
Табличные подстановки		
	S-box (англ. substitute)	$X' = \text{Table}[X, V]$

Блочное шифрование. Сеть Фейстеля



Алгоритм шифрования

- Информация разбивается на блоки одинаковой (фиксированной) длины.
- Выбранный блок делится на два подблока одинакового размера — «левый» и «правый»
- «Правый подблок» изменяется функцией F с использованием раундового ключа
- Результат складывается по модулю 2 («xor») с «левым подблоком»
- Результат будет использован в следующем раунде в роли «правого подблока»
- «Правый подблок» текущего раунда будет использован в следующем раунде в роли «левого подблока»
- По какому-либо математическому правилу вычисляется следующий раундовый ключ.

Блочное шифрование. Сеть Фейстеля

Математическое описание

- X — блок данных, поступающий на вход (входной блок);
- A — некоторое инволютивное преобразование — взаимно-однозначное преобразование, которое является обратным самому себе, то есть для каждого ($\forall$) X справедливо выражение:

$$AAX = A^2X = X, \forall X;$$

- Y — блок данных, получаемый на выходе (результат).

При однократном применении преобразования A к входному блоку X получится выходной блок Y :

$$Y = AX.$$

При применении преобразования A к результату предыдущего преобразования — Y получится:

$$AY = AAX = X, \forall X.$$

Блочное шифрование. Сеть Фейстеля

Математическое описание

Пусть входной блок X состоит из двух подблоков L и R равной длины:

$$X = (L, R).$$

Определим два преобразования:

- $G(X, K)$ — шифрование данных X с ключом K :
$$G(X, K) = G((L, R), K) = (L \oplus F(K, R), R);$$
- $T(L, R)$ — перестановка подблоков L и R :
$$T(L, R) = (R, L).$$

Введём обозначения:

однократное применение преобразования G :

$$\tilde{X} = (\tilde{L}, \tilde{R}) = GX;$$

двукратное применение преобразования G :

$$\tilde{\tilde{X}} = (\tilde{\tilde{L}}, \tilde{\tilde{R}}) = G^2 X.$$

Блочное шифрование. Сеть Фейстеля

Математическое описание

Докажем инволютивность двукратного преобразования G :

Преобразование G меняет только левый подблок L , оставляя правый R неизменным:

$$\begin{aligned}\tilde{L} &= L \oplus F(K, R); \\ \tilde{R} &= R.\end{aligned}$$

Поэтому далее будем рассматривать только подблок L .

После двукратного применения преобразования G к L получим:

$$\tilde{\tilde{L}} = \tilde{L} \oplus F(K, \tilde{R}) = \tilde{L} \oplus F(K, R) = L \oplus F(K, R) \oplus F(K, R) = L.$$

Блочное шифрование. Сеть Фейстеля

Математическое описание

Таким образом:

$$\begin{aligned}G^2 L &= L; \\ G^2 R &= R,\end{aligned}$$

следовательно

$$G^2 X = X$$

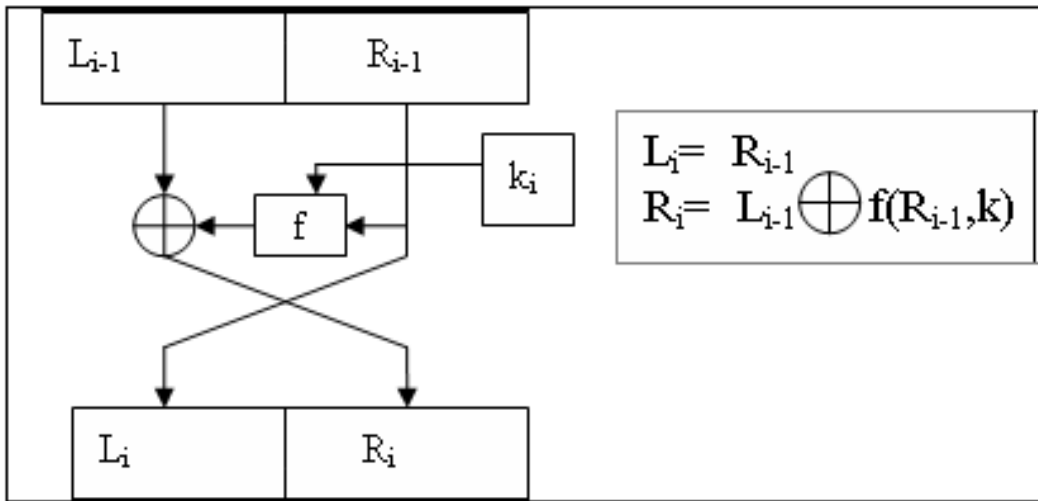
и G — инволюция.

Докажем инволютивность двукратного преобразования T :

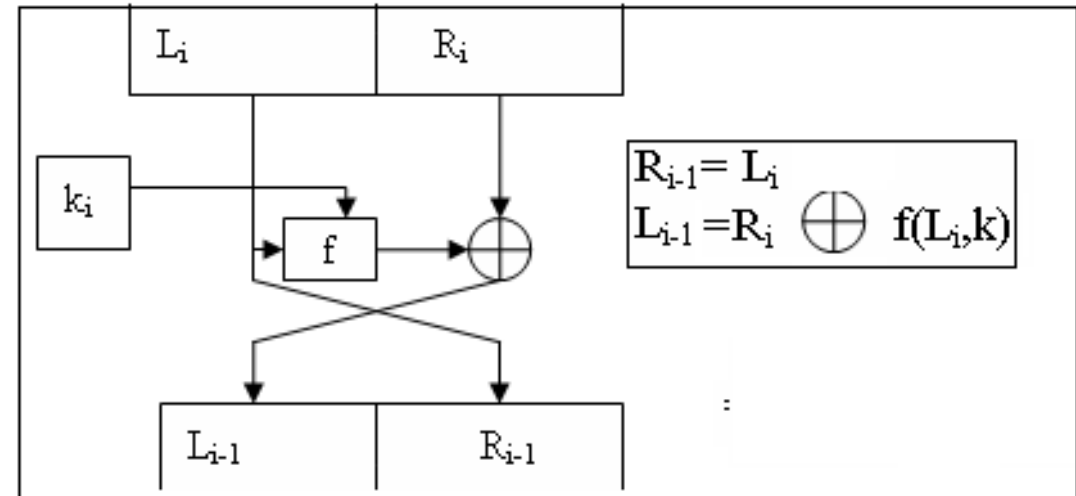
$$T^2 X = T^2(L, R) = T(R, L) = (L, R) = X.$$

DES (Data Encryption Standard)

Размер ключа	56 бит (+ 8 проверочных)
Размер блока	64 бит
Число раундов	16
Тип	Сеть Фейстеля

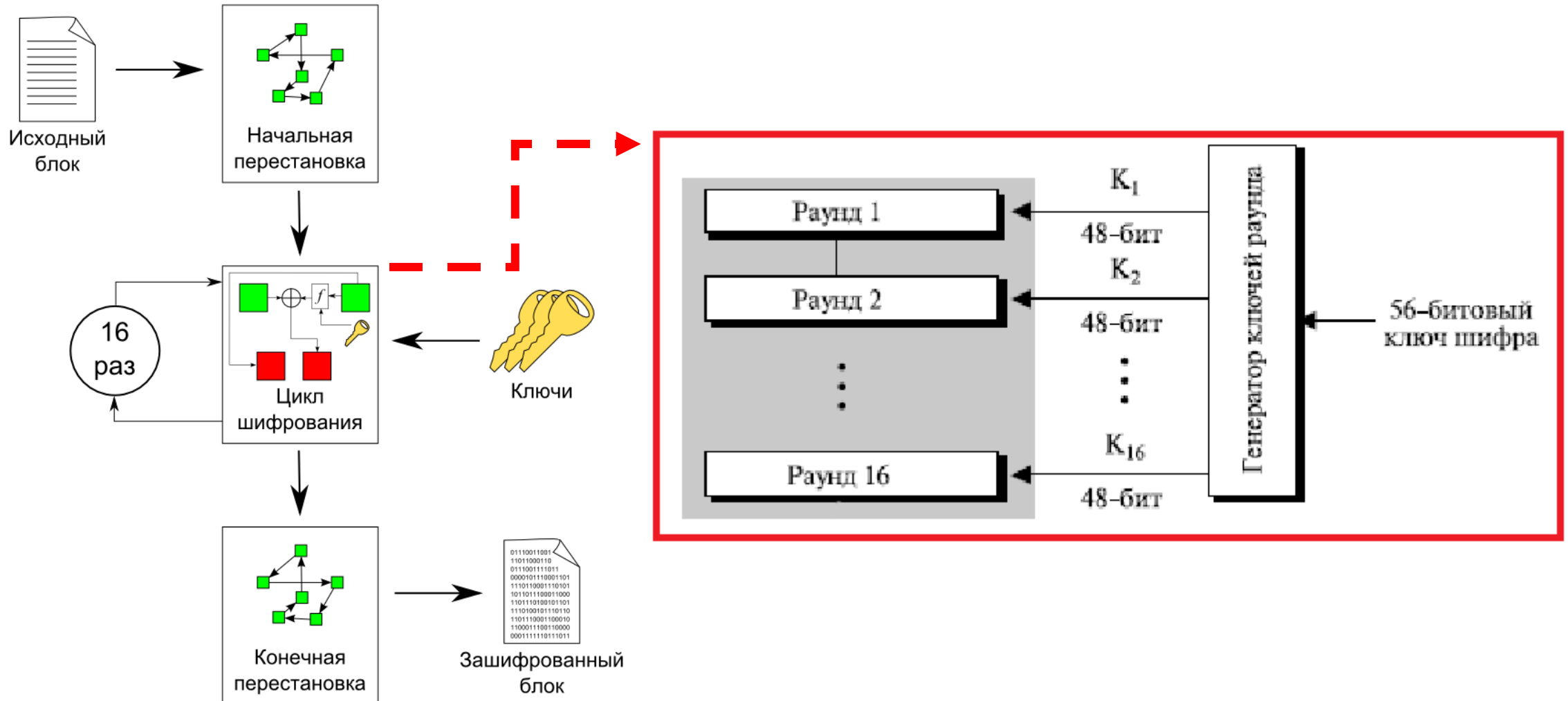


Прямое преобразование сетью Фейстеля
[ШИФРОВАНИЕ]



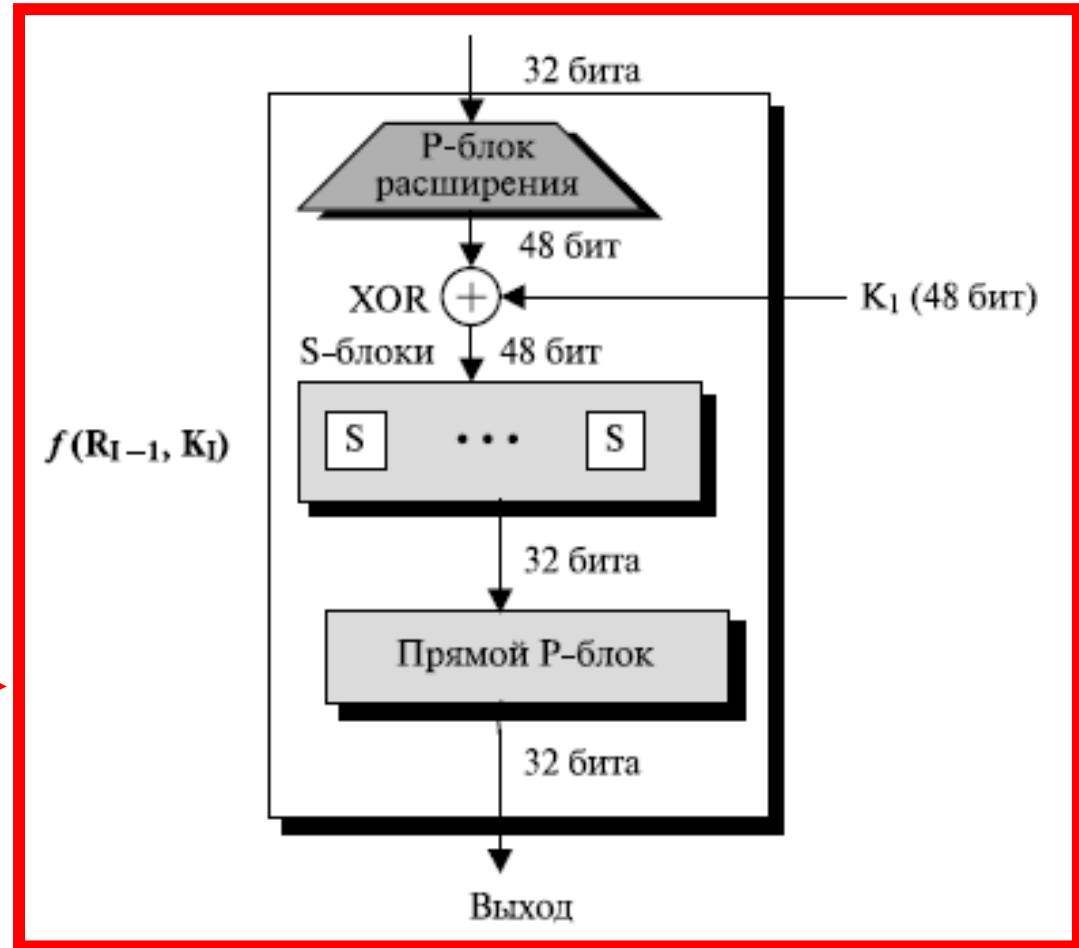
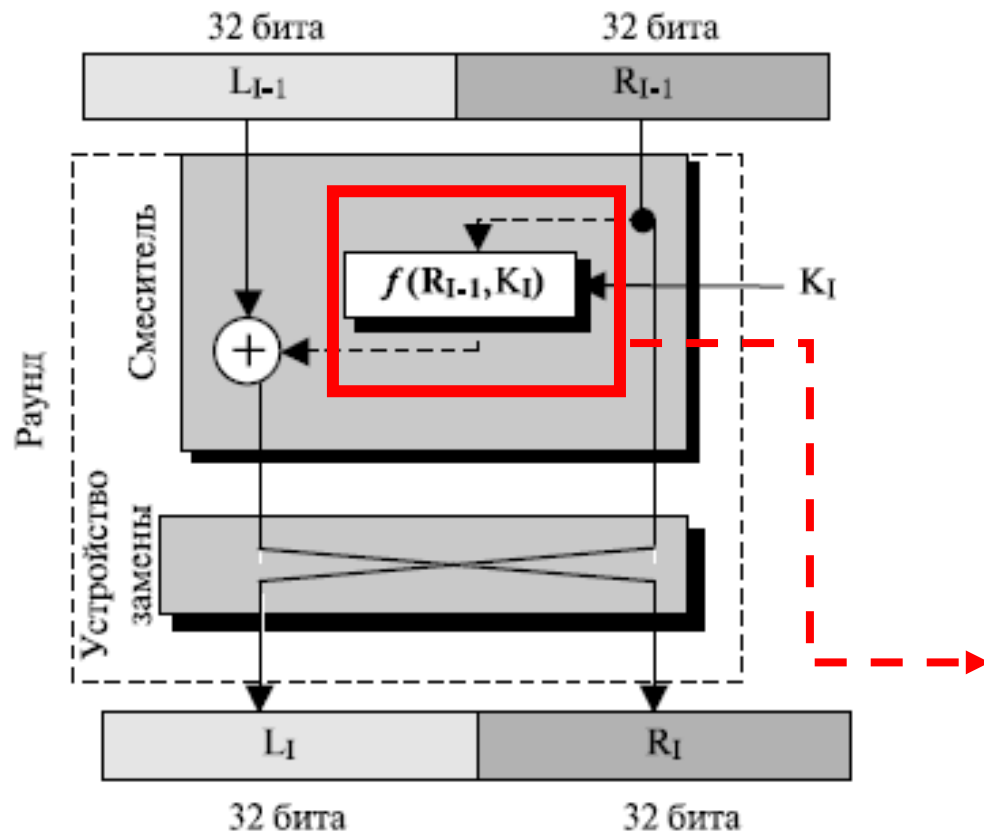
Обратное преобразование сетью Фейстеля
[РАСШИФРОВАНИЕ]

DES (Data Encryption Standard)



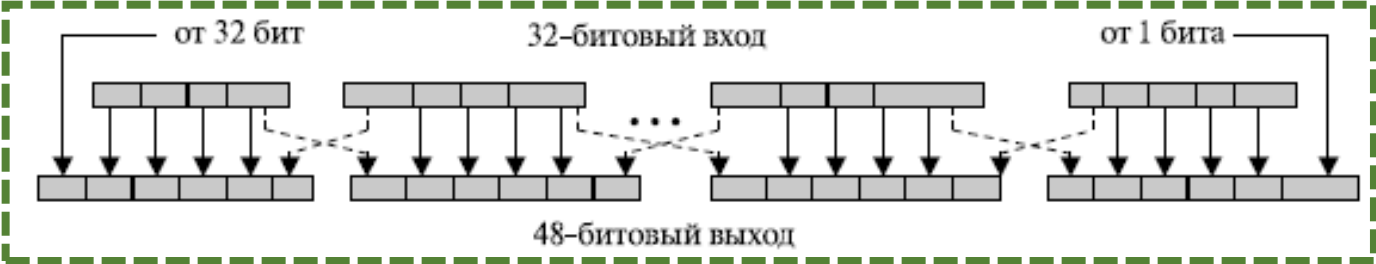
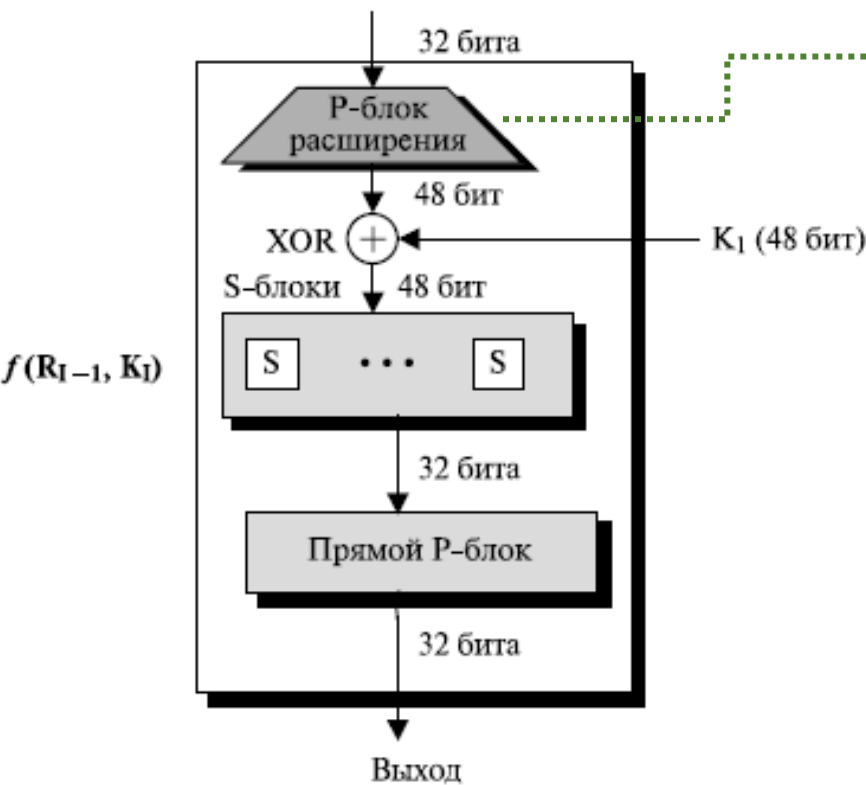
DES (Data Encryption Standard)

Раунд шифрования



DES (Data Encryption Standard)

Раунд шифрования Р-блок расширения



32	01	02	03	04	05
04	05	06	07	08	09
08	09	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	31	31	32	01

Расширенный подблок
[4 -> 6]

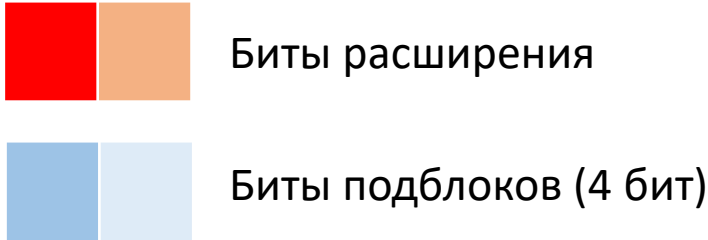
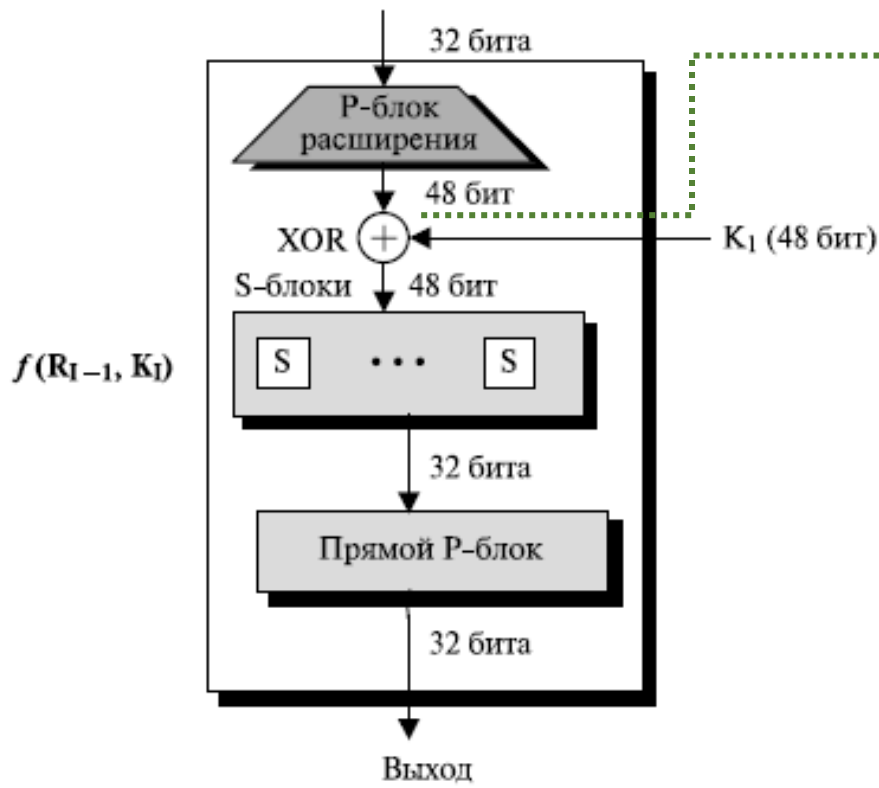


Таблица расширения

DES (Data Encryption Standard)

Раунд шифрования Отбеливатель (whitener)

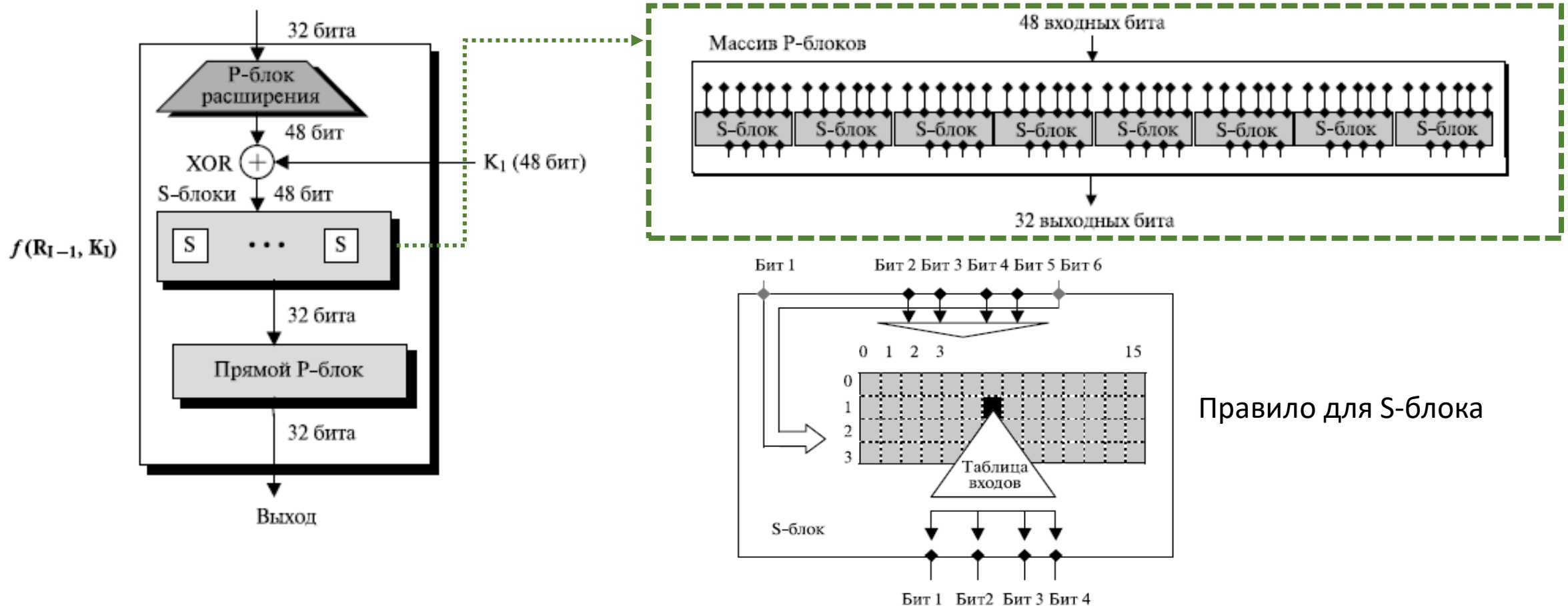


После расширения DES использует операцию **XOR (ИСКЛЮЧАЮЩЕЕ ИЛИ)** над расширенной частью правой секции и ключом раунда. Заметим, что правая секция и ключ имеют длину **48 бит**. Также заметим, что ключ раунда использует только эту операцию.

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

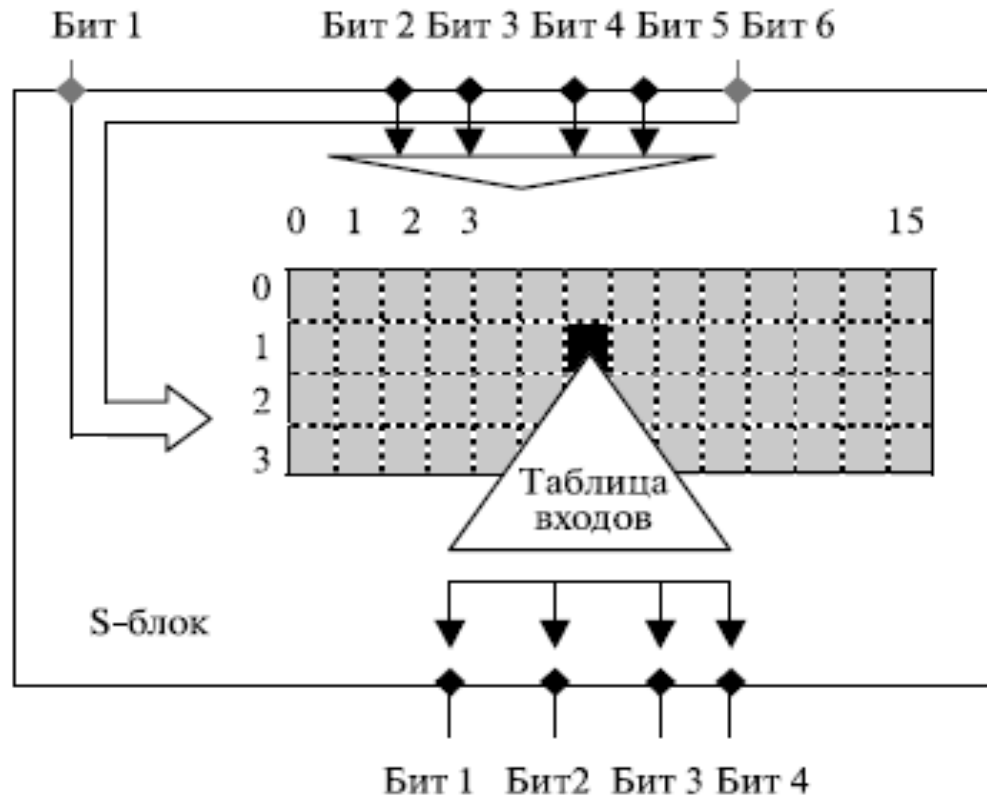
DES (Data Encryption Standard)

Раунд шифрования S-блоки



DES (Data Encryption Standard)

Раунд шифрования Правило для S-блока



С-блок №1 (из 8)

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	04	13	01	02	15	11	08	03	10	06	12	05	09	00	07
1	00	15	07	04	14	02	13	10	03	06	12	11	09	05	03	08
2	04	01	14	07	13	06	02	11	15	12	09	07	03	10	05	00
3	15	12	08	02	04	09	01	07	05	11	03	14	10	00	06	13

Входная последовательность S-блока 1 = **100011**₂

$$\mathbf{11}_2 = \mathbf{3}_{10}$$

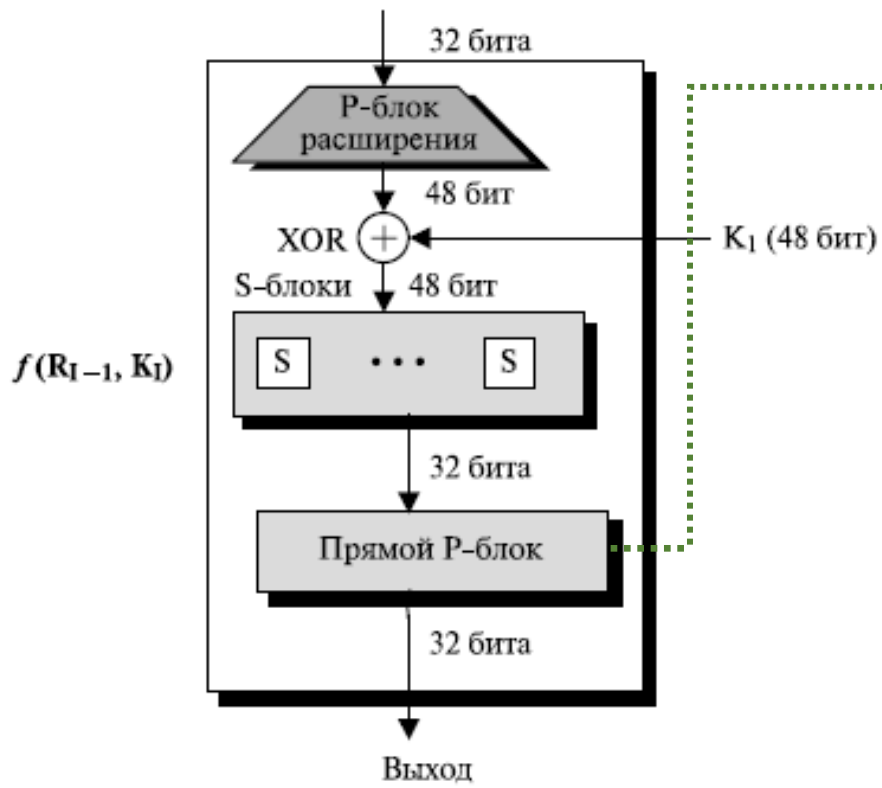
$$\mathbf{0001}_2 = \mathbf{1}_{10}$$

На пересечении получаем **12**₁₀ = **1100**₂

Таким образом вход **1100011** дает выход **1100**.

DES (Data Encryption Standard)

Раунд шифрования Прямая перестановка



Последняя операция в функции DES — прямая перестановка с 32 битами на входе и 32 битами на выходе.

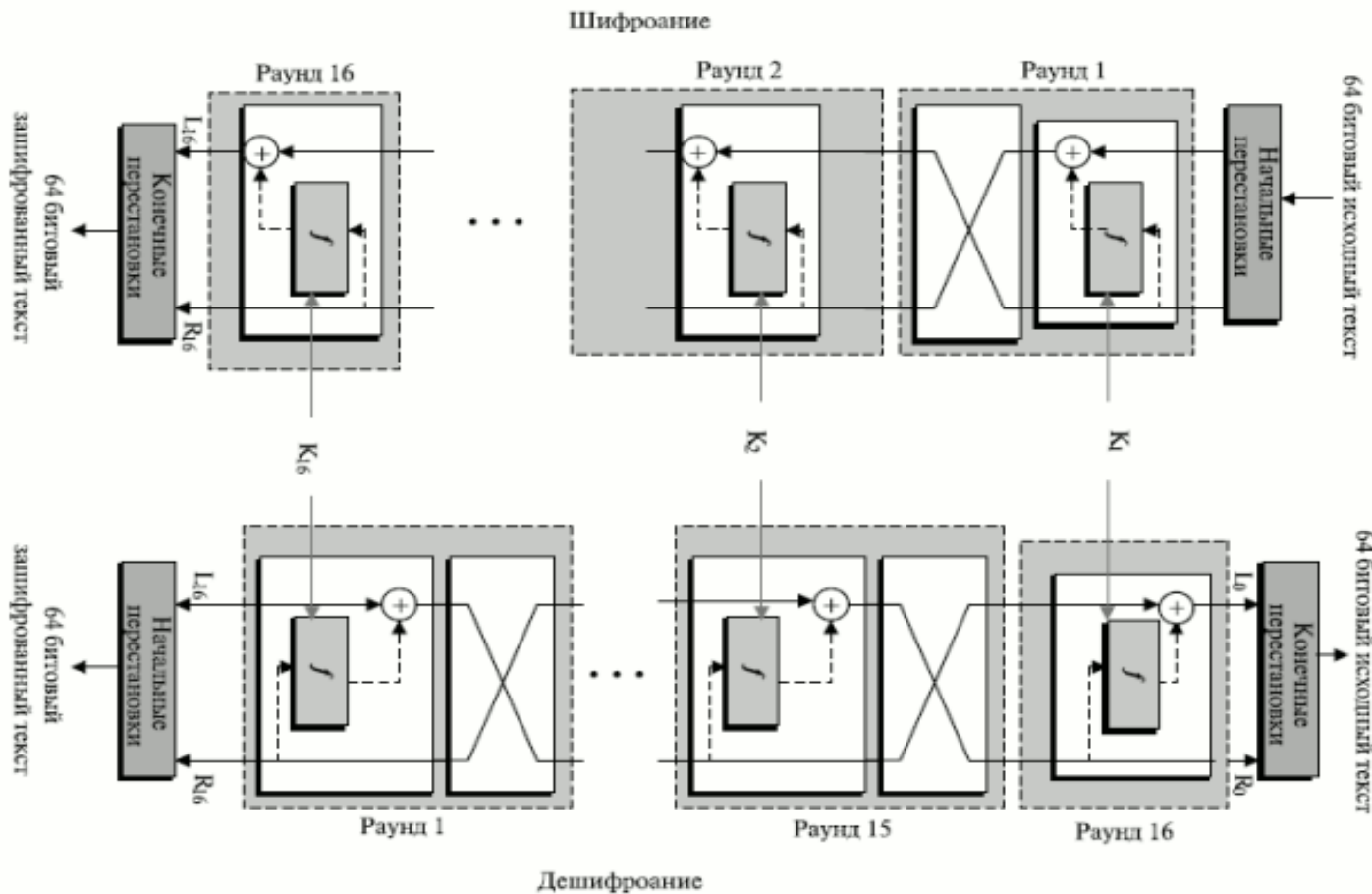
16	07	20	21	29	12	28	17
01	15	23	26	05	18	31	10
02	08	24	14	32	27	03	09
19	13	30	06	22	11	04	25

Например, седьмой бит входа становится вторым битом выхода.

Отношения "вход-выход"

DES (Data Encryption Standard)

Шифрование и Дешифрование



Левая секция исходного текста на стороне шифрования шифруется L_0 как L_{16} , и L_{16} дешифруется на стороне дешифратора как L_0 . Аналогичная ситуация с R_0 и R_{16} .

Ключи раундов (K_1 и K_{16}) применяются при шифровании и дешифровании в обратном порядке. На стороне шифрования первый раунд применяет ключ K_1 , а раунд 16 — ключ K_{16} ; при дешифровании раунд 1 использует ключ K_{16} , а раунд 16 — ключ K_1 .

DES (Data Encryption Standard)

Генератор ключей

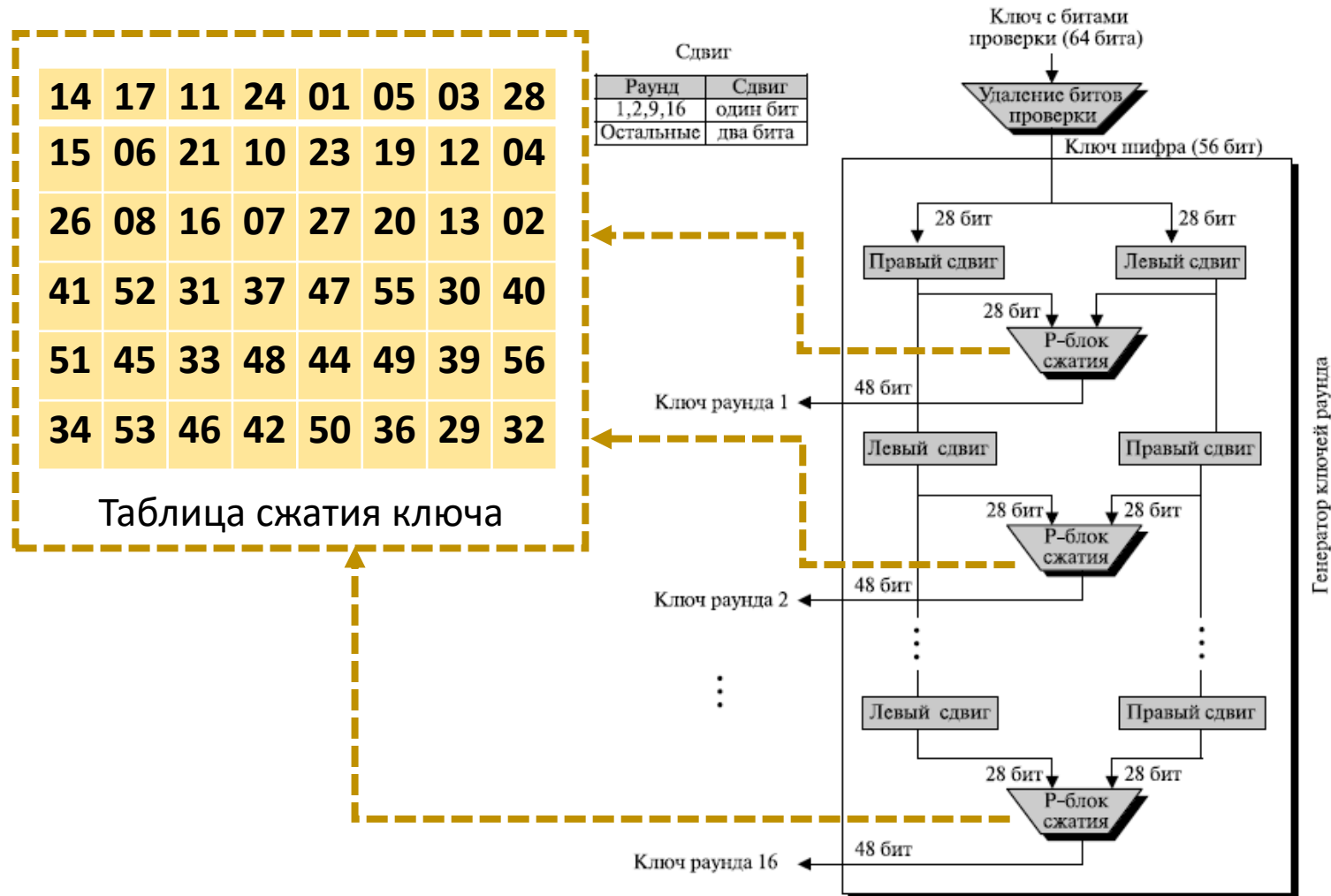
Генератор ключей создает **шестнадцать ключей** по **48 бит** из ключа шифра на **56 бит**. Однако ключ шифра обычно дается как ключ из 64-х бит, в котором 8 дополнительных битов являются битами проверки. Они отбрасываются перед фактическим процессом генерации ключей.

57	49	41	33	25	17	09	01
58	50	42	34	26	18	10	02
59	51	43	35	27	19	11	03
60	52	44	36	63	55	47	39
31	23	15	07	62	54	46	37
30	22	14	06	61	53	45	37
29	21	13	05	28	20	12	04

Таблица удаления проверочных битов

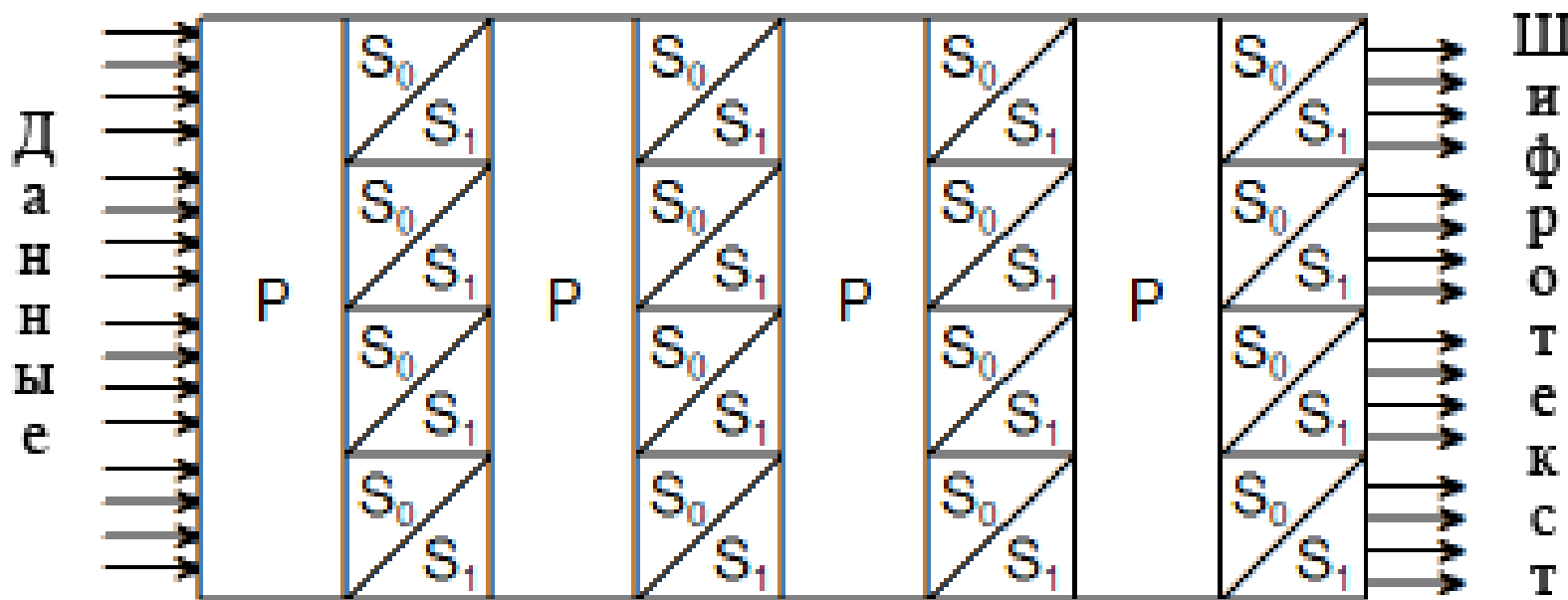
DES (Data Encryption Standard)

Генератор ключей

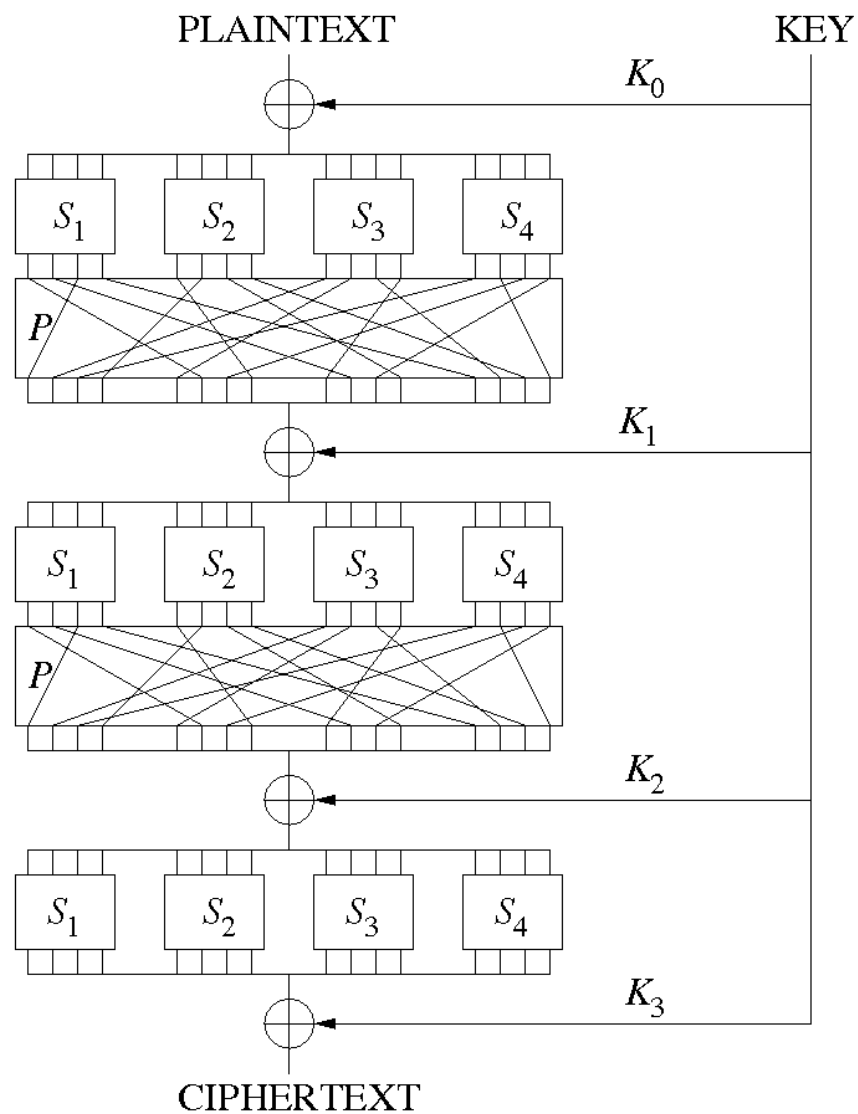


SP-сеть

SP-сеть (подстановочно-перестановочная сеть) — разновидность **блочного шифра**, в простейшем варианте представляющая собой последовательность из слоёв двух типов, используемых многократно по очереди.



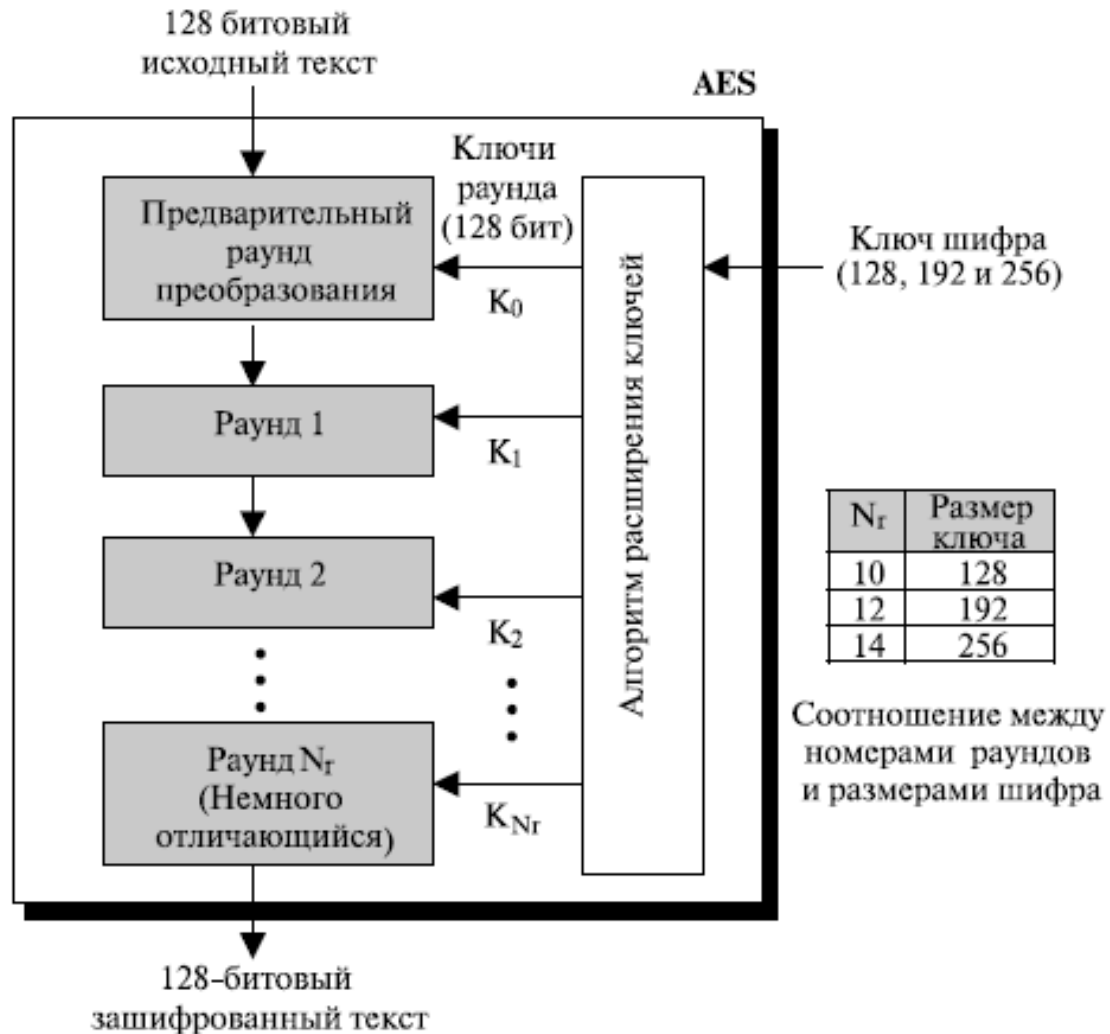
SP-сеть



S-блок (*substitution box*) замещает маленький блок входных бит на другой блок выходных бит.

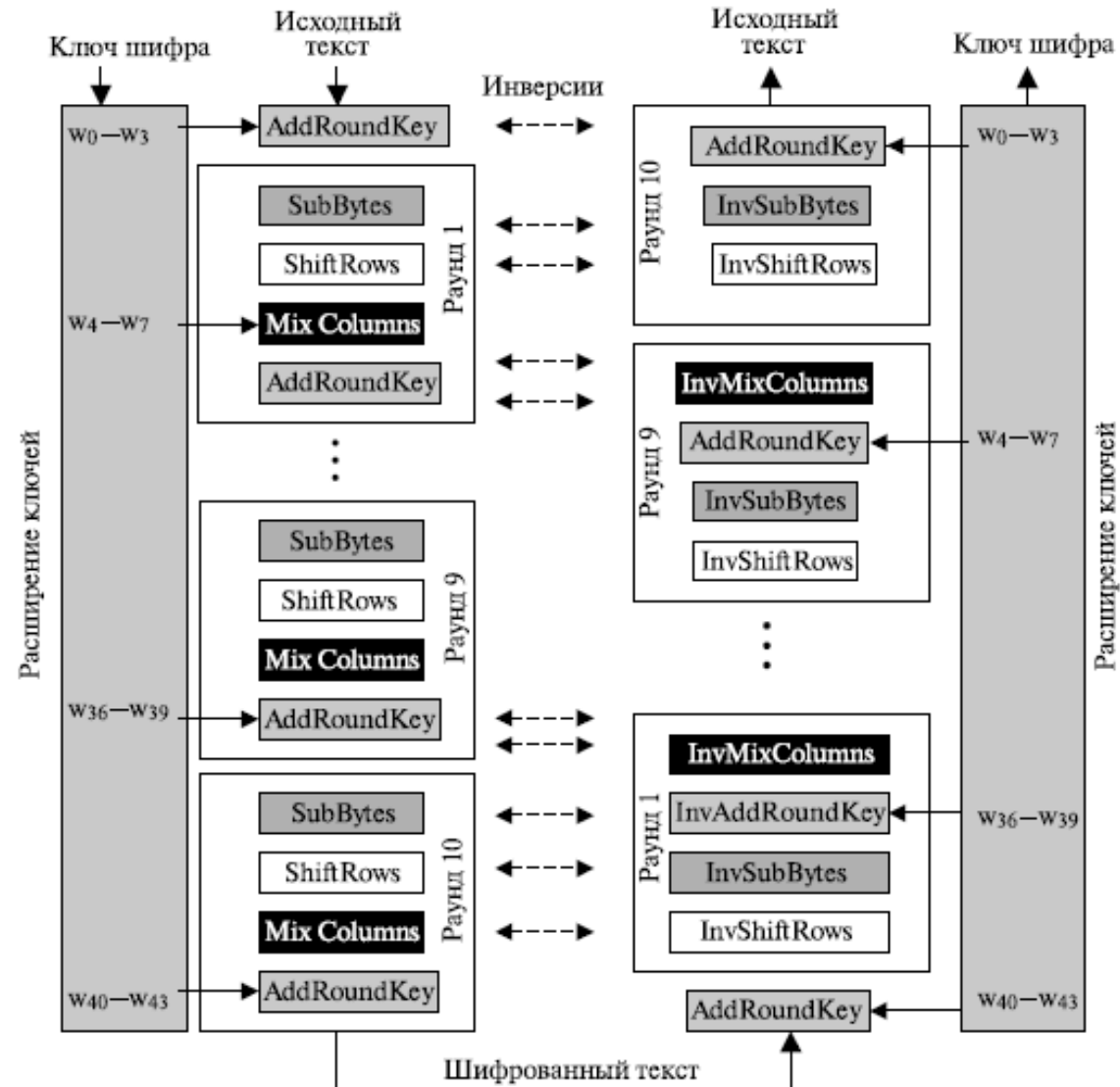
P-блок (*permutation box*) — перестановка всех бит

AES (Advanced Encryption Standard)



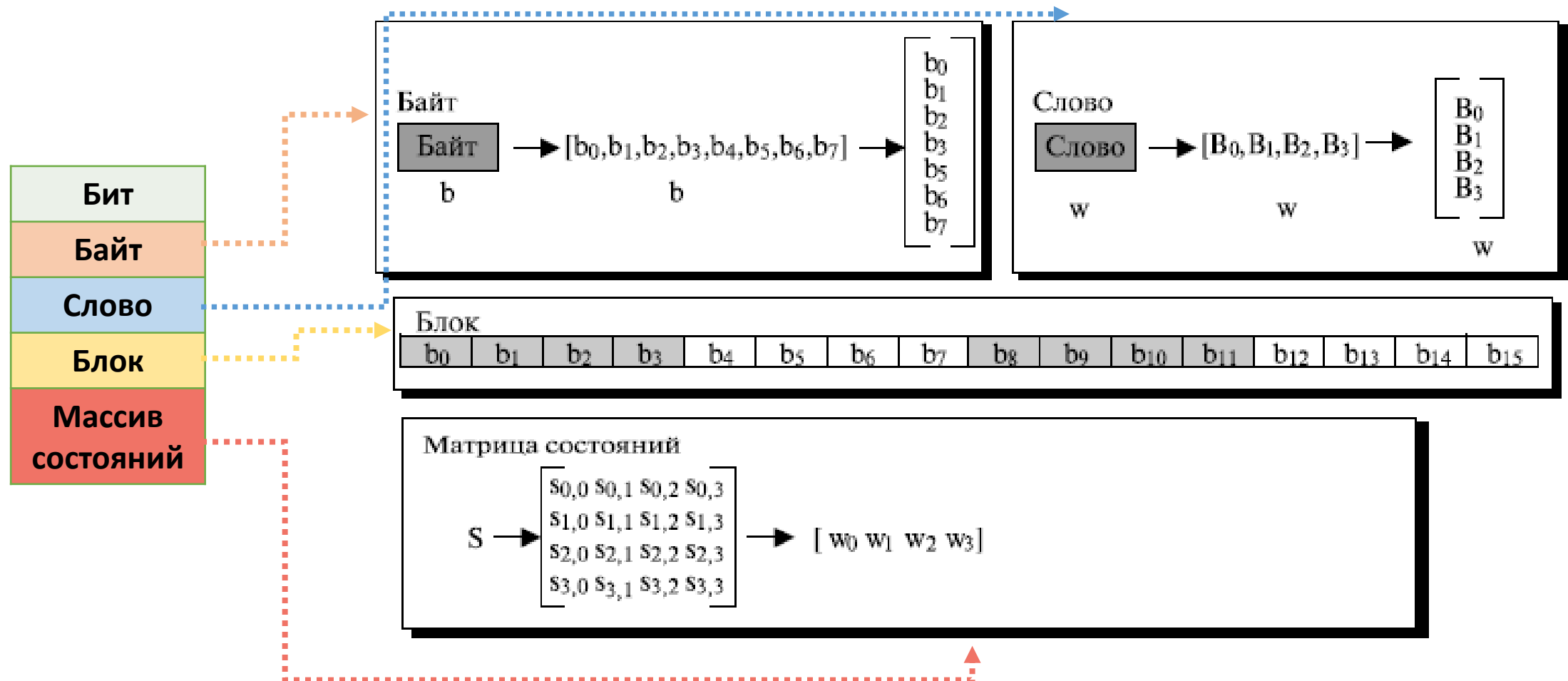
Размер ключа	128/192/256 бит
Размер блока	128 бит
Число раундов	10/12/14 (зависит от размера ключа)
Тип	SP-сеть

AES (Advanced Encryption Standard)



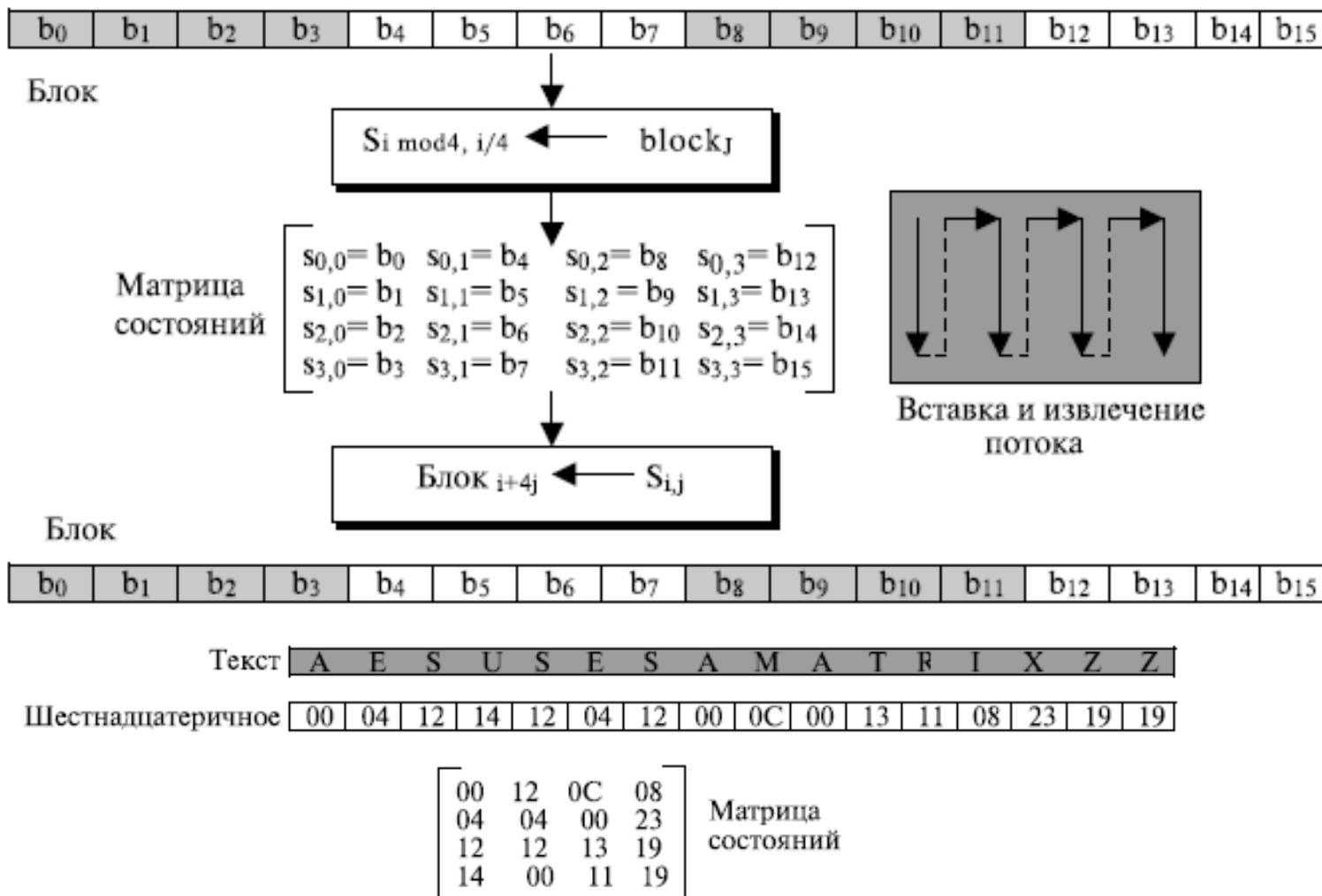
AES (Advanced Encryption Standard)

Единицы данных



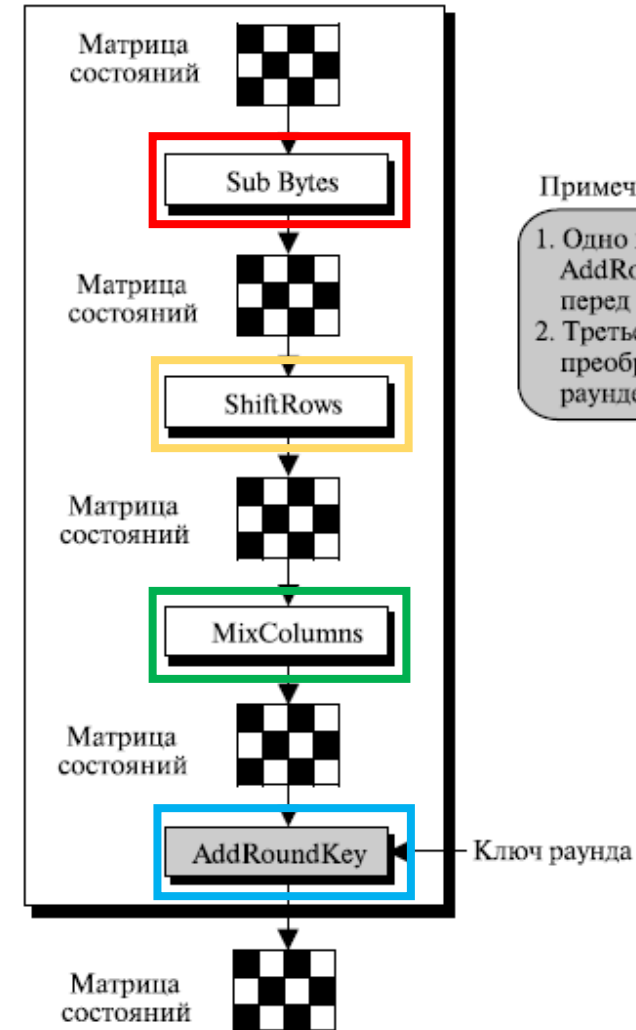
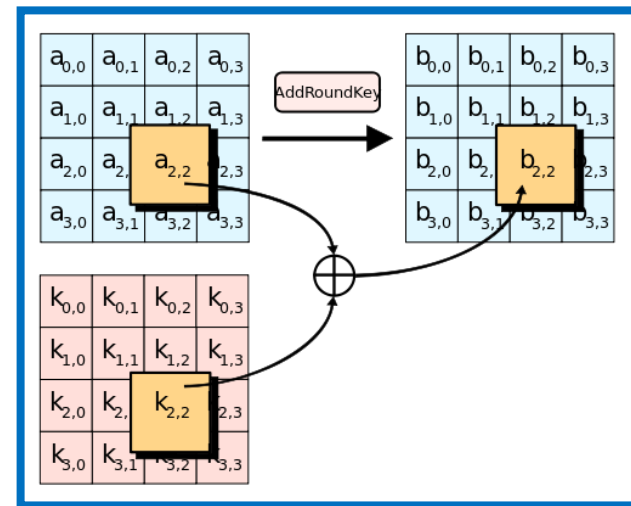
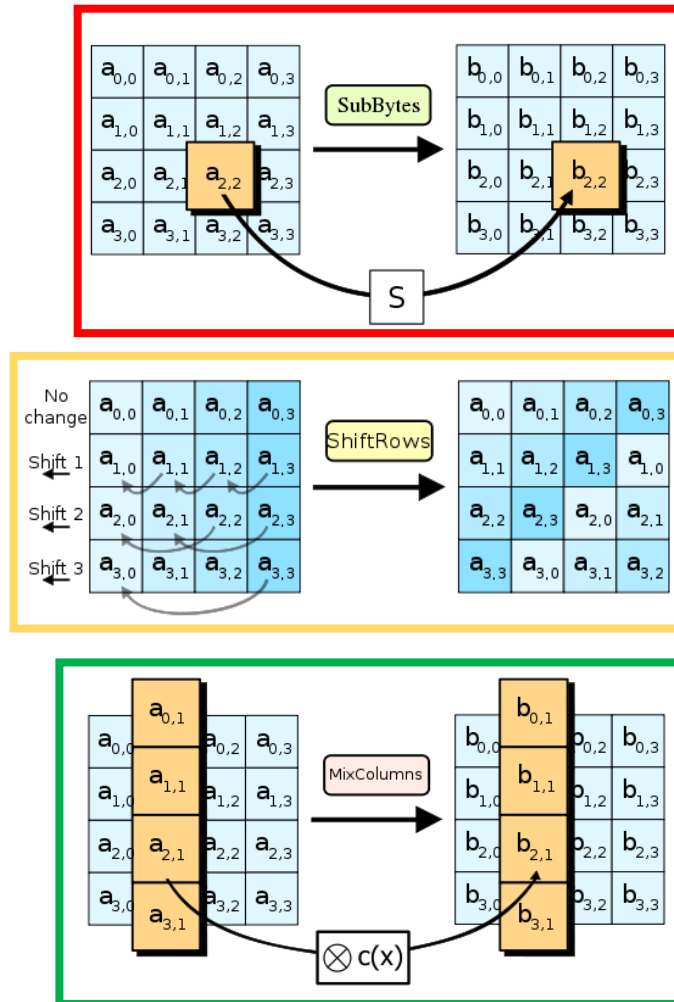
AES (Advanced Encryption Standard)

Матрица состояний



AES (Advanced Encryption Standard)

Структура раунда



Примечание

1. Одно преобразование AddRoundKey применяется перед первым раундом
2. Третье MixColumn преобразование в последнем раунде отсутствует

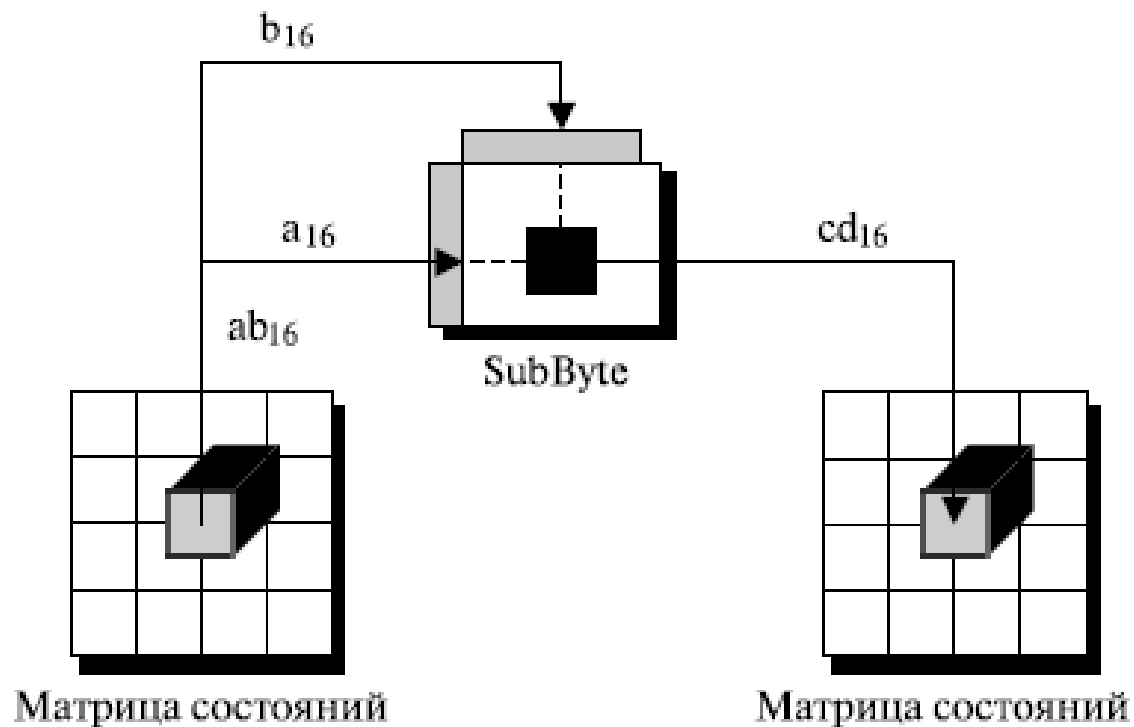
AES (Advanced Encryption Standard)

Преобразования

AddRoundKey()	трансформация при шифровании и обратном шифровании, при которой Round Key XOR'ится с State
InvMixColumns()	трансформация при расшифровании, которая является обратной по отношению к MixColumns()
InvShiftRows()	трансформация при расшифровании, которая является обратной по отношению к ShiftRows()
InvSubBytes()	трансформация при расшифровании, которая является обратной по отношению к SubBytes()
MixColumns()	трансформация при шифровании, которая берёт все столбцы State и смешивает их данные (независимо друг от друга), чтобы получить новые столбцы
RotWord()	функция, используемая в процедуре Key Expansion, которая берёт 4-байтовое слово и производит над ним циклическую перестановку
ShiftRows()	трансформации при шифровании, которые обрабатывают State, циклически смещая последние три строки State на разные величины
SubBytes()	трансформации при шифровании, которые обрабатывают State, используя нелинейную таблицу замещения байтов (S-box), применяя её независимо к каждому байту State
SubWord()	функция, используемая в процедуре Key Expansion, которая берёт на входе четырёхбайтовое слово и, применяя S-box к каждому из четырёх байтов, выдаёт выходное слово

AES (Advanced Encryption Standard)

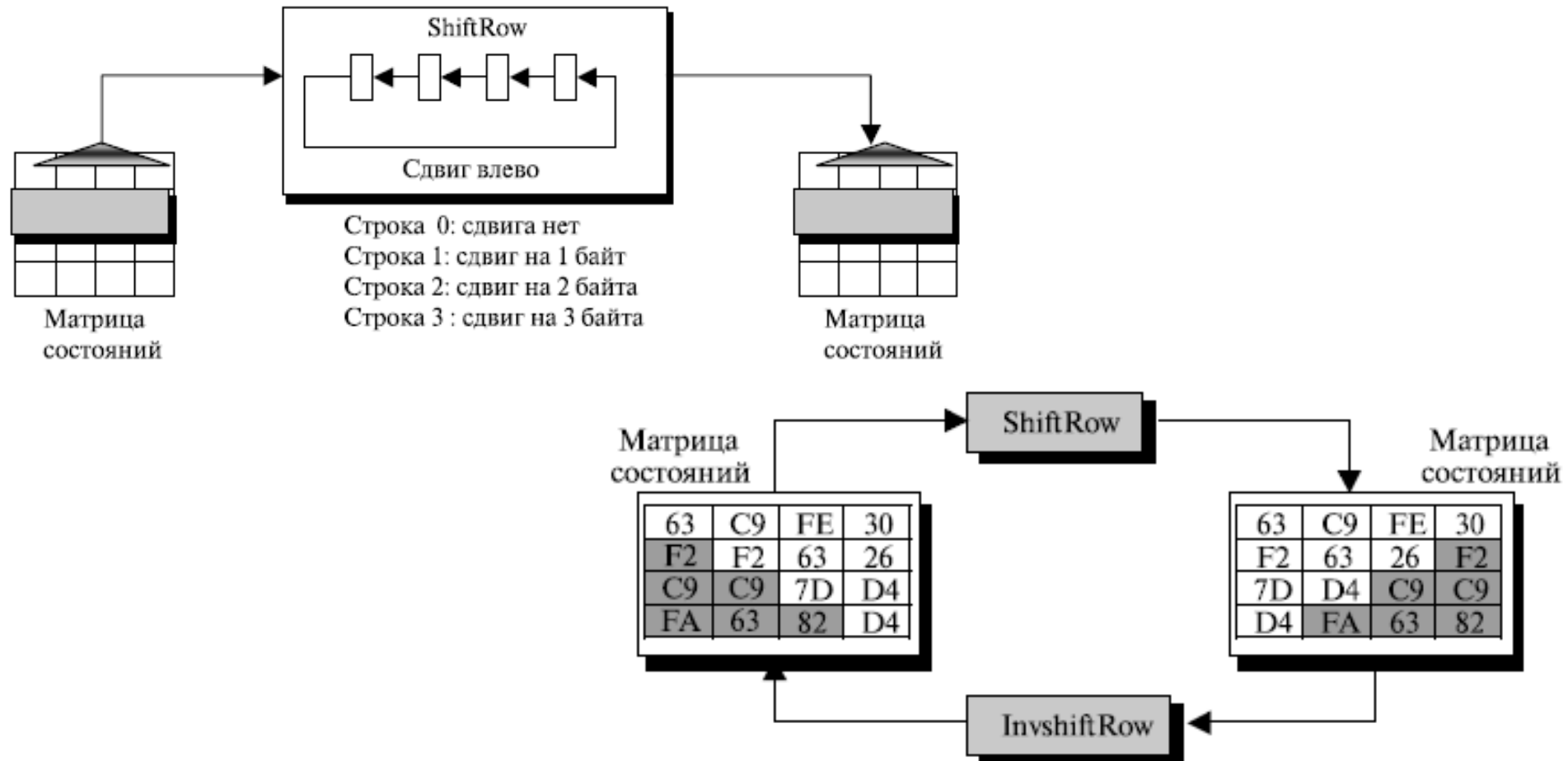
Преобразования SubBytes



	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	FO	AD	D4	A2	AF	9C	A4	72	CO
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	AO	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	DO	EF	AA	FB	43	4D	33	85	45	F9	02	F7	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DE
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	CB	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	IF	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	OE	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	OD	BF	E6	42	68	41	99	2D	OF	BO	54	BB	16

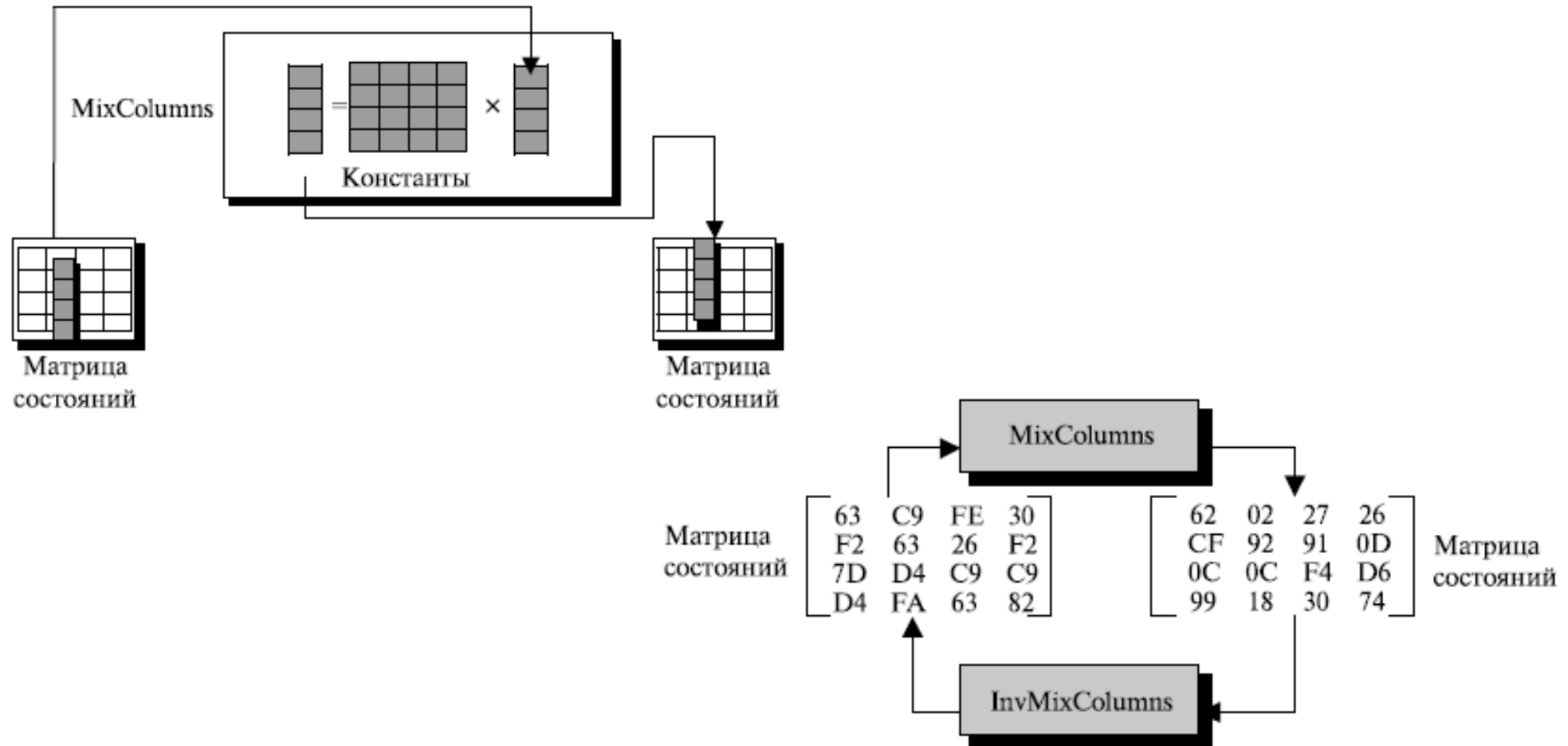
AES (Advanced Encryption Standard)

Преобразования ShiftRows



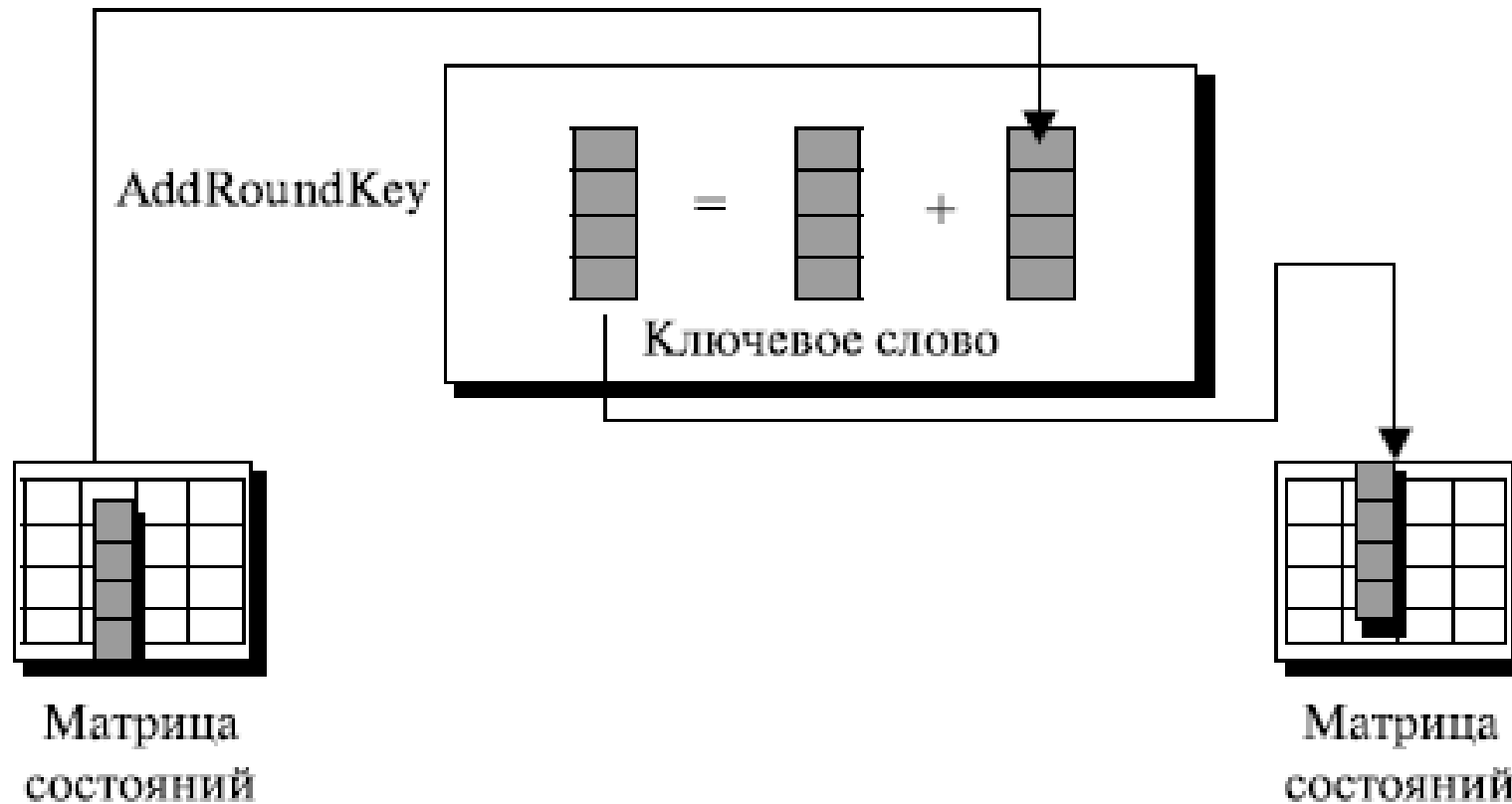
AES (Advanced Encryption Standard)

Преобразования MixColumns



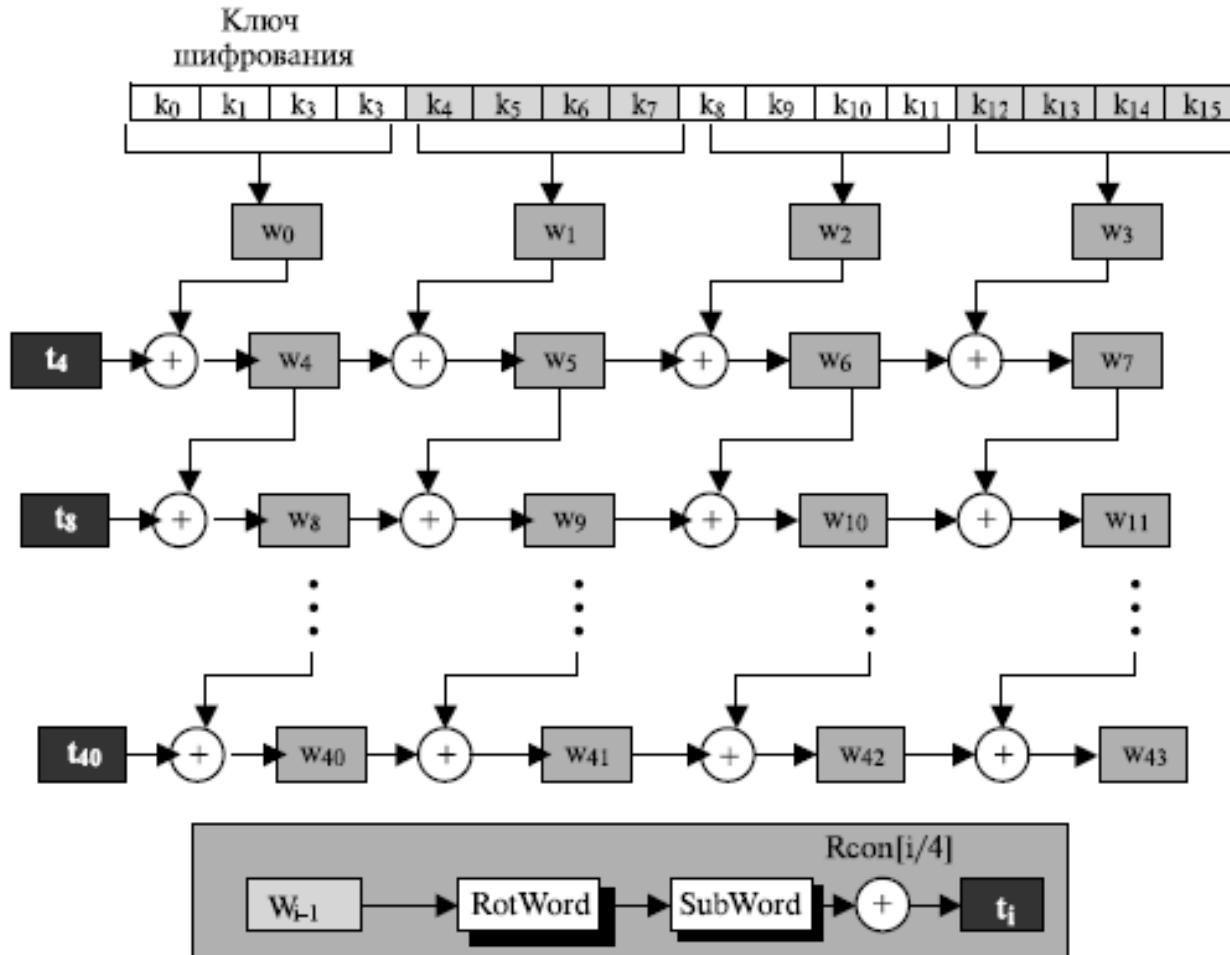
AES (Advanced Encryption Standard)

Преобразования AddRoundKey



AES (Advanced Encryption Standard)

Расширение ключа



Вырабатываются t_i (временные) слова $i = 4N_r$

RotWord (rotate word) — процедура, подобная преобразованию **ShiftRows**, но применяется только к одной строке. Процедура принимает слово как массив из четырех байт и сдвигает каждый байт влево с конвертированием.

SubWord (substitute word) — процедура, подобная преобразованию **SubBytes**, но применяется только к одной строке. Процедура принимает каждый байт в слове и заменяет его другим.

RoundConstants R_{con} — это 4-байтовое значение, в котором самые правые три байта являются всегда нулевыми.

AES (Advanced Encryption Standard)

Расширение ключа в AES

Задача: расширить ключ $k \in \{0, 1\}^{128}$ до 10 раундовых ключей $k_i \in \{0, 1\}^{128}$

- $k_0 = k = (w_{0,0}, w_{0,1}, w_{0,2}, w_{0,3})$, $w_{0,1} \in \{0, 1\}^{32}$
- $k_i = (w_{i,0}, w_{i,1}, w_{i,2}, w_{i,3})$, где

$$w_{i,0} = w_{i-1,0} \oplus g_i(w_{i-1,3})$$

$$w_{i,1} = w_{i-1,1} \oplus w_{i,0}$$

$$w_{i,2} = w_{i-1,2} \oplus w_{i,1}$$

$$w_{i,3} = w_{i-1,3} \oplus w_{i,2}$$

$g_i : \{0, 1\}^{32} \rightarrow \{0, 1\}^{32}$ – функция, состоящая из сдвигов, подстановки SubBytes и XOR с раундовыми константами c_i .

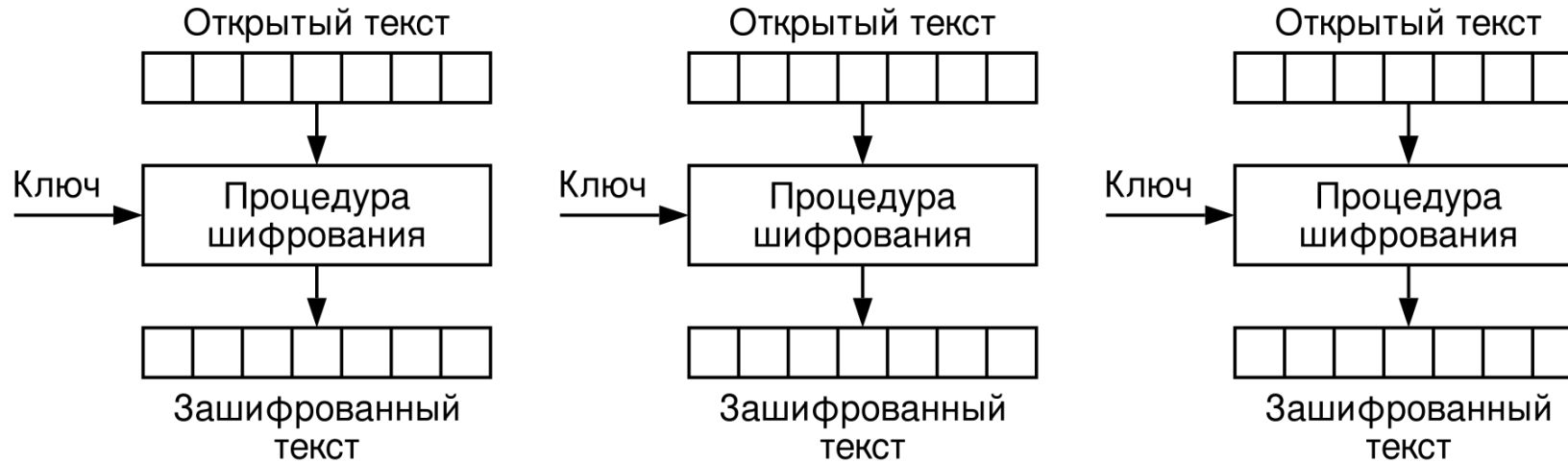
Блочное шифрование

Режимы шифрования

1. Режим электронной кодовой книги или режим простой замены (Electronic Block Code, EBC)
2. Режим сцепления блоков шифротекста (Cipher Block Chain, CBC)
3. Режим счётчика (Counter mode, CTR)

Блочное шифрование

ЕВС



$$C_i = E_k(P_i, k), P_i = D_k(C_i, k)$$

где:

- i — номер блока;
- k — ключ;
- P_i — блок сообщения (открытый текст);
- C_i — зашифрованный блок (шифротекст);
- E_k — функция, выполняющая блочное шифрование.

В результате получаются зашифрованные блоки $C_1, C_2, \dots, C_q$.

Блочное шифрование

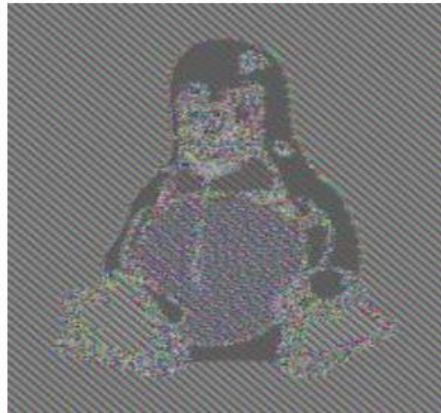
ECB

Недостатки ECB:

- сохранение статистических особенностей открытого текста (поскольку одинаковым блокам шифротекста соответствуют одинаковые блоки открытого текста).



Открытый текст в виде
изображения



Криптограмма, полученная шифрованием в режиме
ECB. На изображении видны черты исходного
изображения



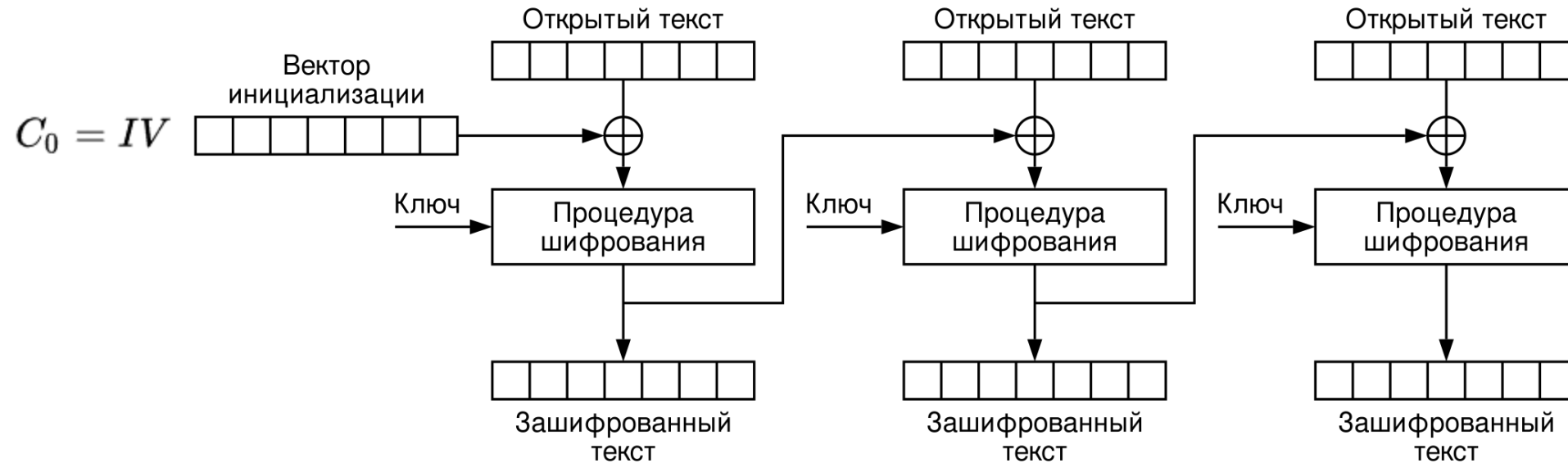
Криптограмма, полученная шифрованием в режиме, отличном от
ECB. Изображение представляет собой псевдослучайную
последовательность пикселей

Достоинства ECB:

- постоянная скорость обработки блоков (скорость определяется эффективностью реализации шифра);
- возможно распараллеливание вычислений (так как блоки не связаны между собой).

Блочное шифрование

CBC



$$C_i = E_k(P_i \oplus C_{i-1}, k), \quad P_i = C_{i-1} \oplus D_k(C_i, k)$$

где:

- i — номер блока;
- k — ключ;
- IV — вектор инициализации (синхропосылка);
- P_i — блок сообщения (открытый текст);
- C_{i-1} — зашифрованный блок (шифротекст), полученный на предыдущем шаге шифрования;
- E_k — функция, выполняющая блочное шифрование.

Блочное шифрование

СВС

Недостатки СВС:

- возможность определения начала изменения данных по изменению шифротекста (если сравнить шифротексты двух сообщений с одним и тем же ключом, то номер первого блока, в котором шифротексты различаются, будет соответствовать номеру первого блока, в котором различаются исходные сообщения);
- возможность изменения блока шифротекста C_{i-1} путём изменения блока P_i сообщения;
- невозможность распараллеливания шифрования (поскольку для шифрования каждого i -го блока требуется блок, зашифрованный на предыдущем шаге (блоки связаны между собой)).

Достоинства СВС:

- постоянная скорость обработки блоков (скорость определяется эффективностью реализации шифра; время выполнения операции «xor» пренебрежимо мало);
- отсутствие статистических особенностей, характерных для режима ЕСВ (поскольку каждый блок открытого текста «смешивается» с блоком шифротекста, полученным на предыдущем шаге шифрования);
- возможность распараллеливания расшифровки.

Блочное шифрование

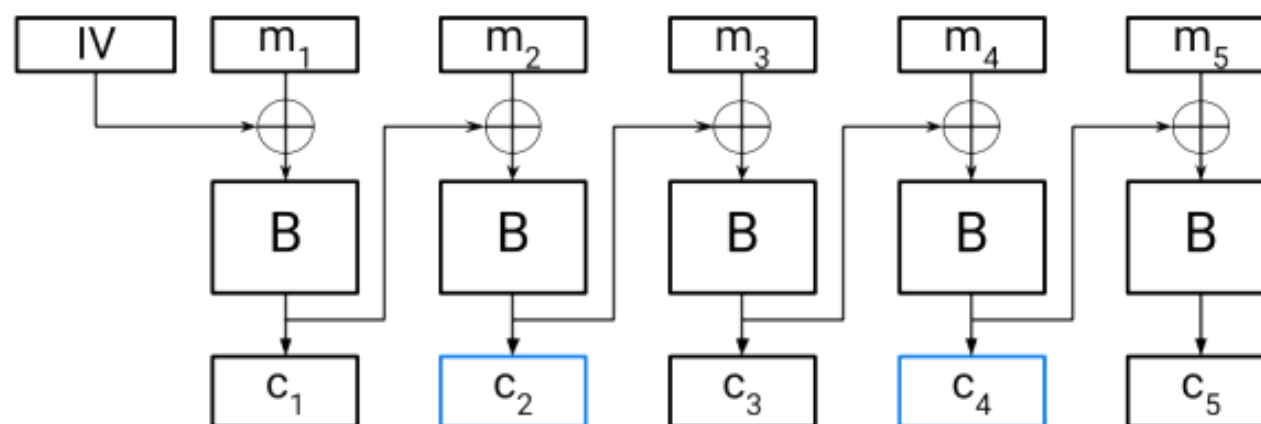
Безопасность режима сцепления блоков

- Начальное значение IV должно быть **случайным** (если атакующий может предсказать IV, шифрование CBC небезопасно). См. атаку на TLS 1.1.
- IV необходимо обновлять

Блочное шифрование

Безопасность режима сцепления блоков

Положим, мы используем одно и тоже IV для длинного сообщения $m = (m_1, \dots, m_t)$ при $t > 2^{n/2}$.



Парадокс Дней Рождений: имея $2^{n/2}$ блоков шифр-текста c_i , с большой вероятностью мы увидим два одинаковых c_i .

$$c_1 \oplus m_2 == c_3 \oplus m_4$$

Далее применяются статистические атаки на m .

Блочное шифрование

Парадокс Дней Рождений

Определить вероятность того, что в комнате из 30 человек двое родились в один день.

$$\left(1 - \frac{1}{365}\right) \left(1 - \frac{2}{365}\right) \left(1 - \frac{3}{365}\right) \cdot \dots \cdot \left(1 - \frac{29}{365}\right) \approx 0.294$$

Значит, с вероятностью $1 - 0.294 > 0.7$ найдутся двое таких людей.

Блочное шифрование

Обобщение Парадокса Дней Рождений

Для m человек и N возможных дней рождений, вероятность того, что все m человек имеют разные дни рождения:

$$P := \prod_{i=1}^{m-1} \left(1 - \frac{i}{N}\right) \approx e^{-m^2/2N}$$

Для $m = \sqrt{2N \ln 2}$, $P \approx 1/2$. Вероятность P быстро увеличивается при росте m .

Для блок-шифра с длиной блока n , имеем 2^n всевозможных блоков шифр-текстов.

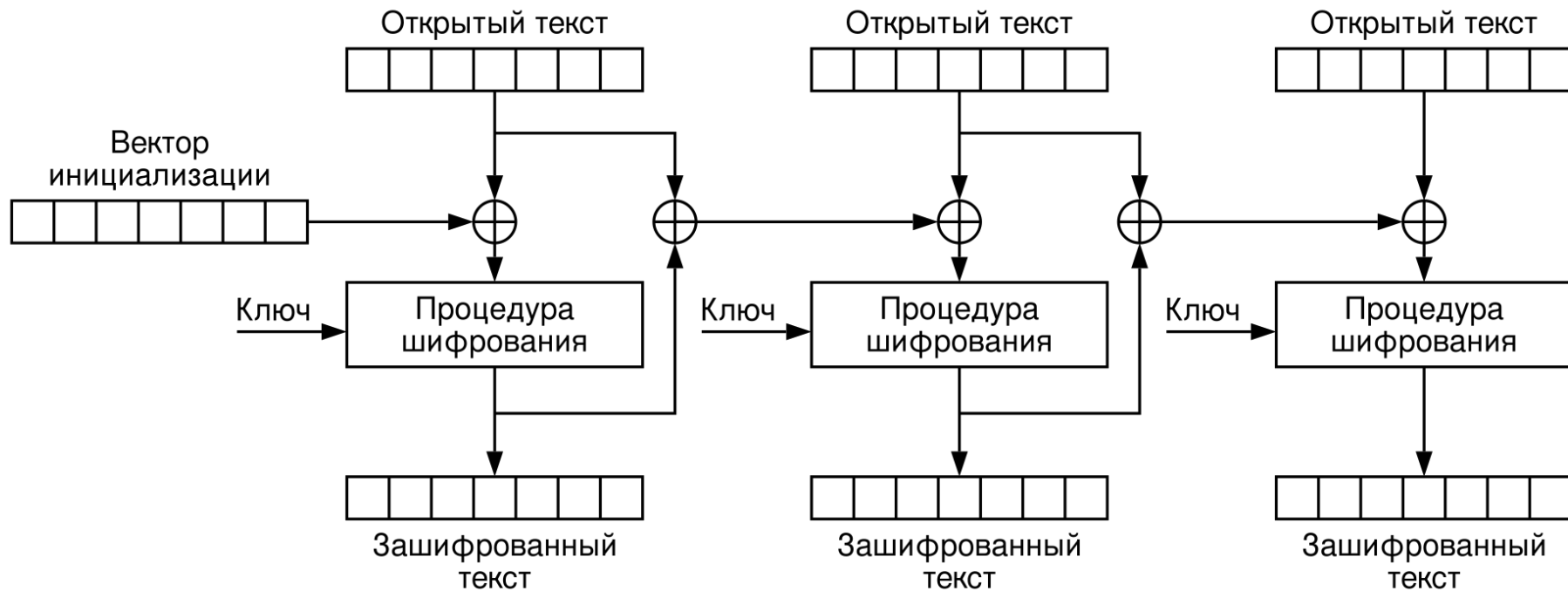
При $m = \mathcal{O}(2^{n/2})$ шифр-блоков c_i 's, некоторые два из них равны с константной вероятностью.

Для режима CBC: $c_i == c_j$ при $m = (m_1, \dots, m_t)$, $t \approx 2^{n/2}$:

$$c_{i-1} \oplus m_i == c_{j-1} \oplus m_j$$

Блочное шифрование

PCBC



Режим PCBC — вариант режима CBC, обладающий специфическим свойством — ошибка шифротекста приводит к неправильному дешифрированию всех последующих блоков. Это соответственно означает, что проверка стандартного блока в конце сообщения обеспечивает целостность всего сообщения.

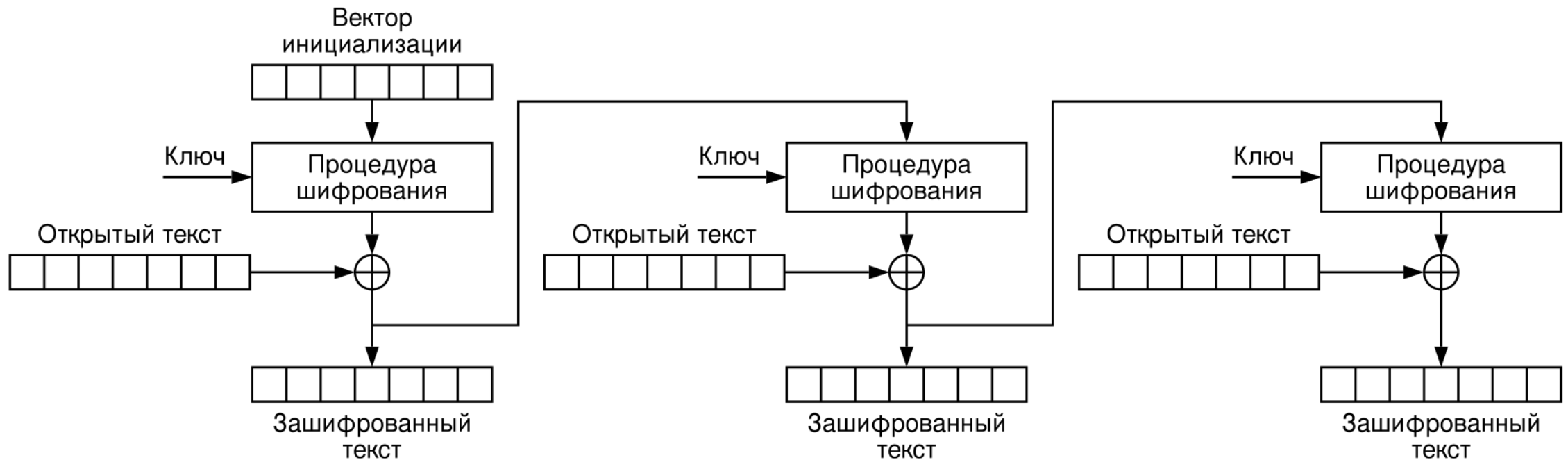
Блочное шифрование

CFB

$$C_0 = IV$$

$$C_i = E_k(C_{i-1}, k) \oplus P_i$$

$$P_i = E_k(C_{i-1}, k) \oplus C_i$$



Блочное шифрование

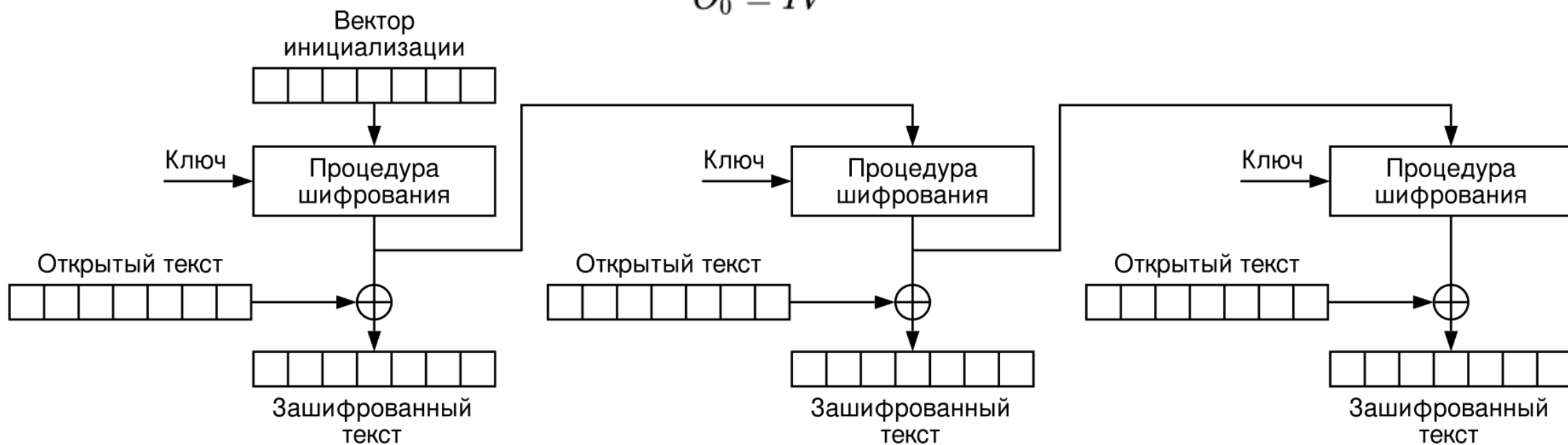
OFB

$$C_i = P_i \oplus O_i$$

$$P_i = C_i \oplus O_i$$

$$O_i = E_k(O_{i-1})$$

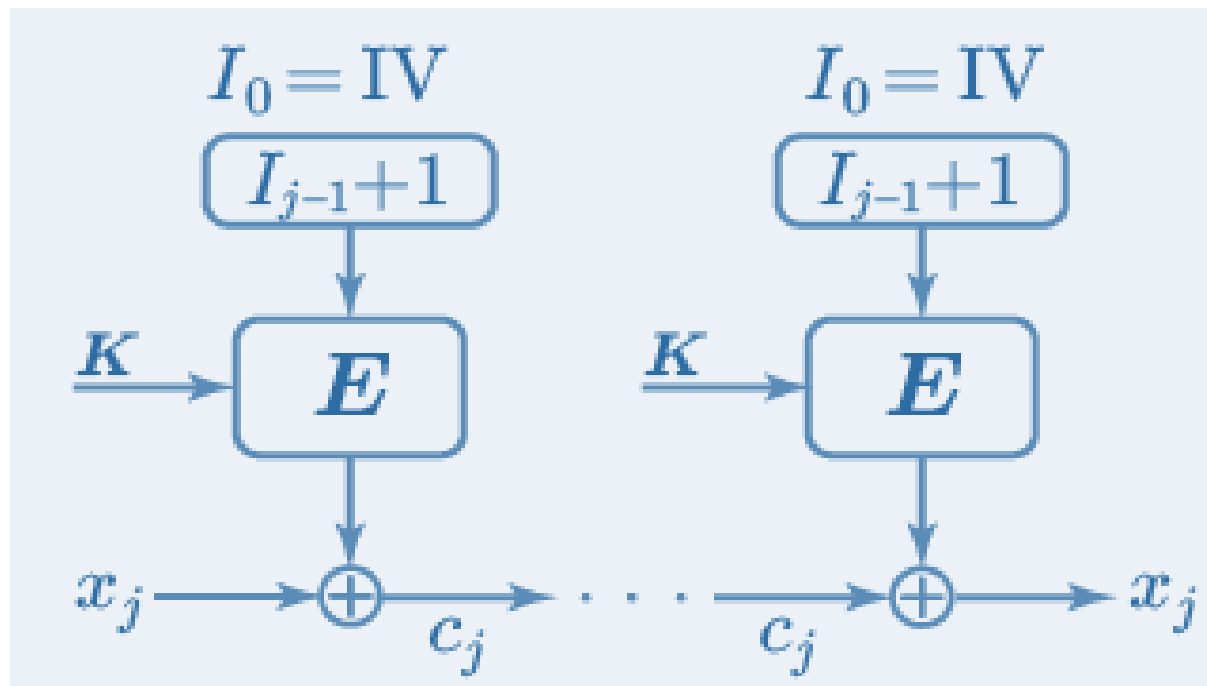
$$O_0 = IV$$



Блочное шифрование

CTR

Один из самых популярных режимов шифрования
Здесь IV - начальное значение счетчика.
Счетчик увеличивается для каждого нового блока.



CTR создает потоковое шифрование из блок-шифра

Шифрование в режиме CTR

$$C_i = P_i \oplus E_k(Ctr_i); i = 1, 2, \dots, m$$

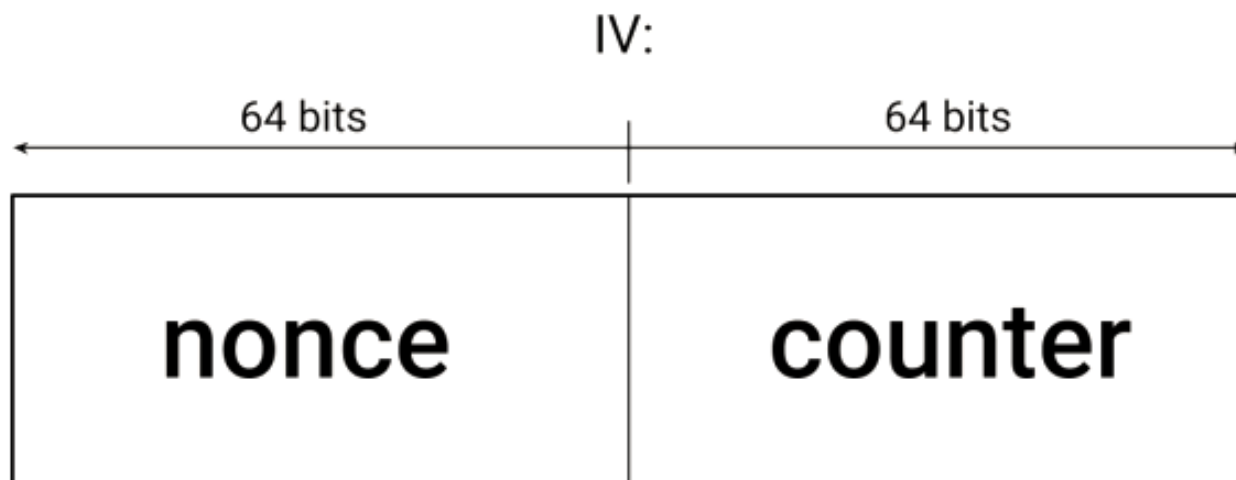
Расшифровка в режиме CTR

$$P_i = C_i \oplus E_k(Ctr_i); i = 1, 2, \dots, m$$

Ctr_i — значение счётчика для i -го блока.

Блочное шифрование

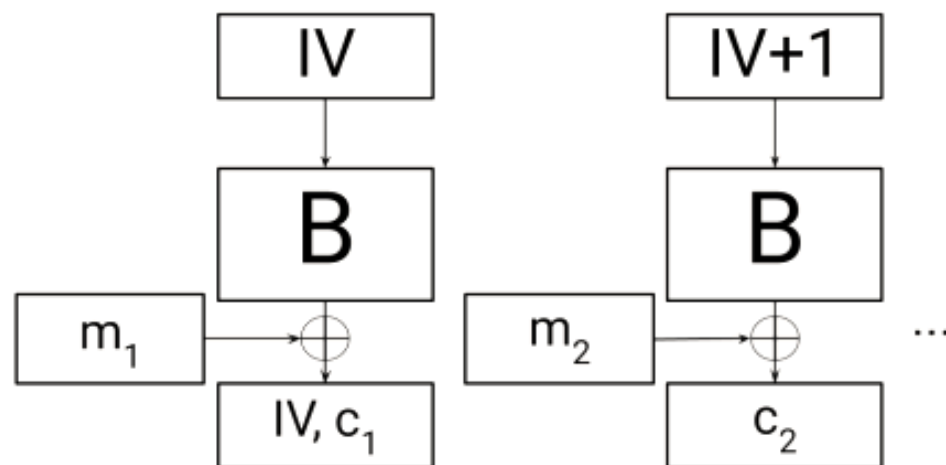
Как выглядит IV



- Nonce (нонс) должен быть псевдослучайным (64-битный выход PRG) и не должен повторяться для одного и того же ключа k
- Счетчик увеличивается для каждого нового блока
- Значение счетчика не передается в протоколах, обеспечивающих последовательную доставку пакетов (https)
- Нонс обновляется после 2^{64} зашифрованных блоков.

Блочное шифрование

Преимущества режима Counter Mode



- Нонс известен шифрующей и дешифрующей сторонам
- Простая процедура дешифрования
- Лёгкая параллелизация (в отличие от CBC)
- Не нужно использовать набивку
- 64-битный нонс (CTR) vs. 128-битный IV

Блочное шифрование

Random Delta (RD)

Режим Random Delta используется для устранения предсказуемости изменения счётчика в режиме CTR.

К примеру:

- AES, размер блока 16 байт.
- Берётся случайный IV.
- Его младшие 8 байт считаются случайной дельтой — Random Delta (RD):

Initialization Vector, 16 bytes



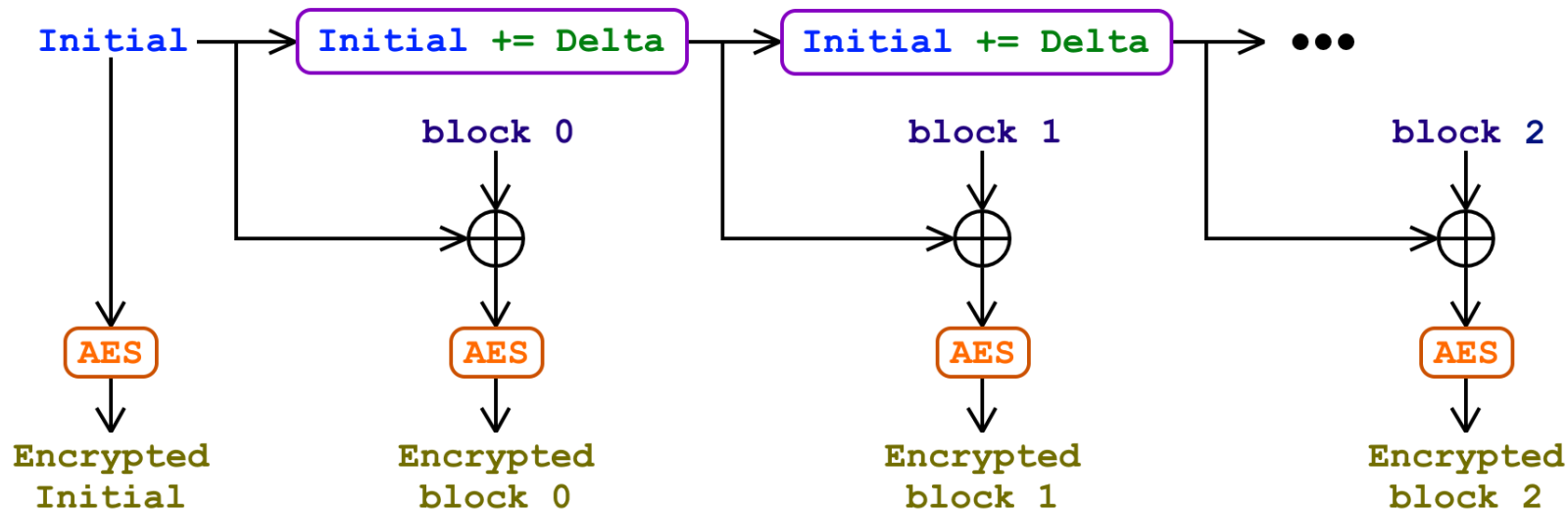
Блочное шифрование

Random Delta (RD)

Initial (IV) шифруется и передаётся в начале сообщения. Блок 0 перед шифрованием XOR-ится с Initial. Для каждого последующего блока величина Initial увеличивается на Delta:

$$Initial[128bits] = Initial[128bits] + Delta[64bits]$$

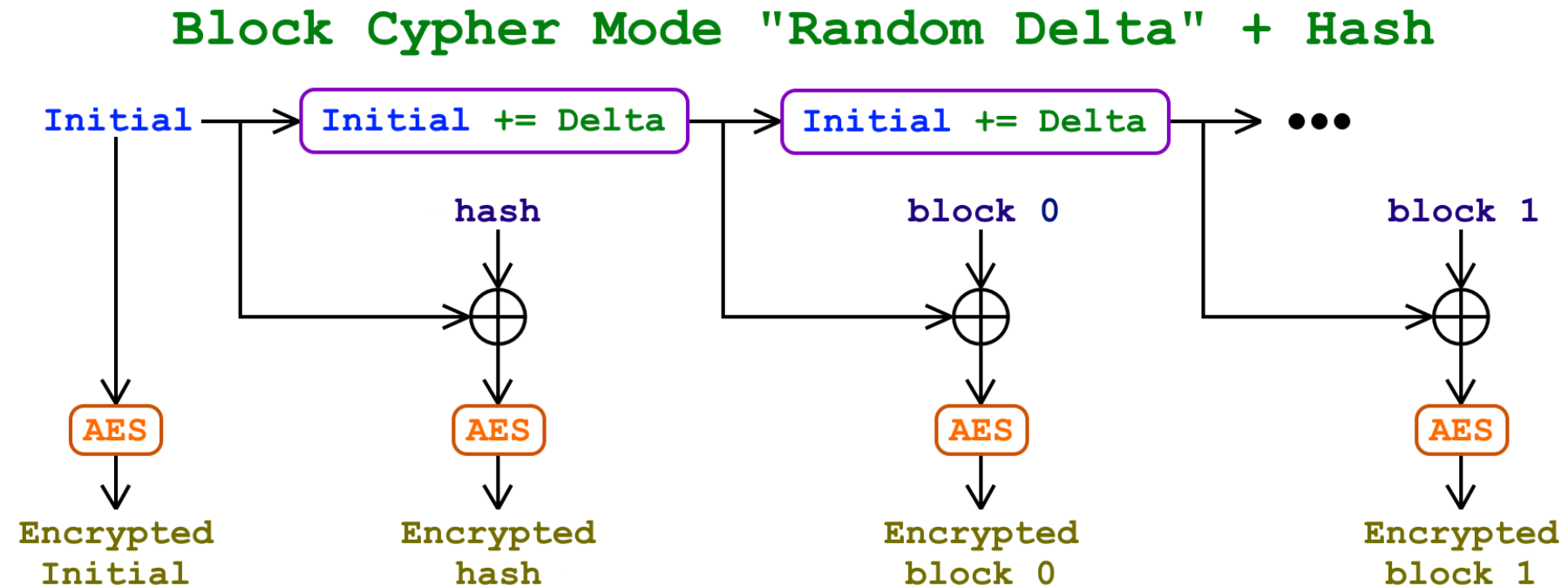
Block Cypher Mode "Random Delta"



Блочное шифрование

Random Delta (RD)

Для проверки целостности в блочную последовательность может быть добавлен хэш:

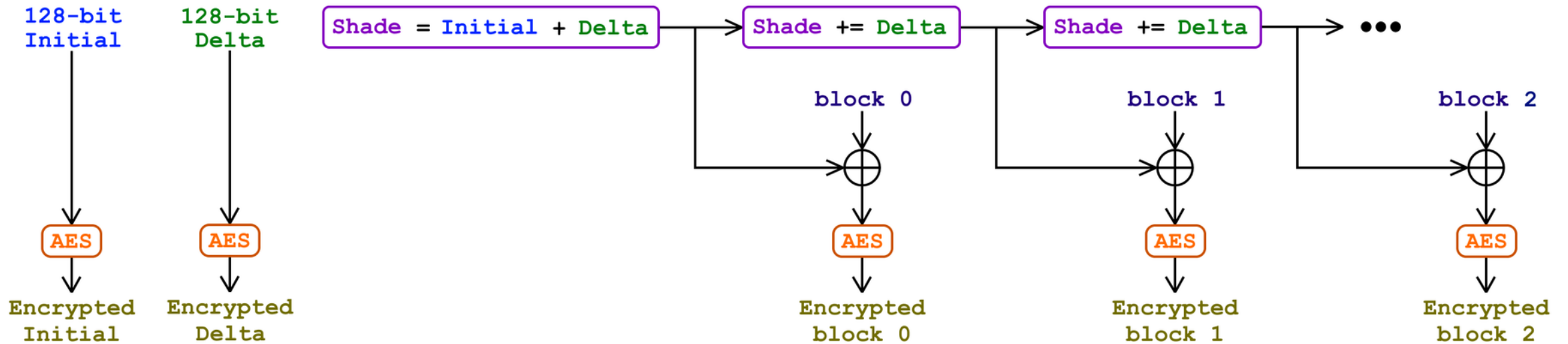


Блочное шифрование

Random Delta (RD)

Режим "Random Delta 128" отличается использованием отдельных 128-битных Initial и Delta.

Block Cypher Mode "Random Delta 128"



Криптографические хэш-функции

Хэш-функции - математические алгоритмы, которые принимают на вход данные произвольного размера и возвращают строку фиксированной длины, называемую хэш-значением (или дайджестом).

Основные свойства криптографических хэш-функций

1. Определённость

- Для одного и того же входного значения хэш-функция всегда возвращает одно и то же хэш-значение.

2. Высокая скорость вычисления

- Хэш-функция должна быть достаточно быстрой для обработки больших объёмов данных.

3. Сложность обратного вычисления

- По хэшу невозможно (или крайне сложно) восстановить исходные данные.

4. Устойчивость к коллизиям

- Практически невозможно найти два разных входных значения, которые дают одинаковый хэш.

5. Устойчивость к атаке второго прообраза

- Зная хэш и одно из входных значений, практически невозможно найти другое входное значение, которое даст тот же самый хэш.

6. Эффект лавины

- Малейшее изменение во входных данных приводит к значительному изменению в хэш-значении.

7. Фиксированный размер выходных данных

- Независимо от размера входных данных, длина хэша остаётся постоянной (например, 256 бит для SHA-256).

Криптографические хэш-функции

Примеры популярных криптографических хэш-функций

1. MD5 (Message Digest 5)

- Длина выходного значения: 128 бит.
- Разработана в 1991 году.
- Считается устаревшей из-за выявленных уязвимостей (коллизии).
- Используется только в не критичных приложениях.

2. SHA-1 (Secure Hash Algorithm 1)

- Длина выходного значения: 160 бит.
- Разработана NSA в 1993 году.
- Уязвима к атакам на коллизии, поэтому не рекомендуется для использования в современных системах.

Криптографические хэш-функции

Примеры популярных криптографических хэш-функций

3. SHA-2 (Secure Hash Algorithm 2)

- Включает несколько вариантов: SHA-224, SHA-256, SHA-384, SHA-512.
- Длина выходного значения: от 224 до 512 бит.
- Высокая безопасность и производительность делают SHA-2 стандартом для большинства современных систем.

4. SHA-3 (Secure Hash Algorithm 3)

- Основана на алгоритме Кескак.
- Длина выходного значения: от 224 до 512 бит.
- Принята как стандарт в 2015 году.
- Предлагает альтернативный подход к построению хэш-функций.

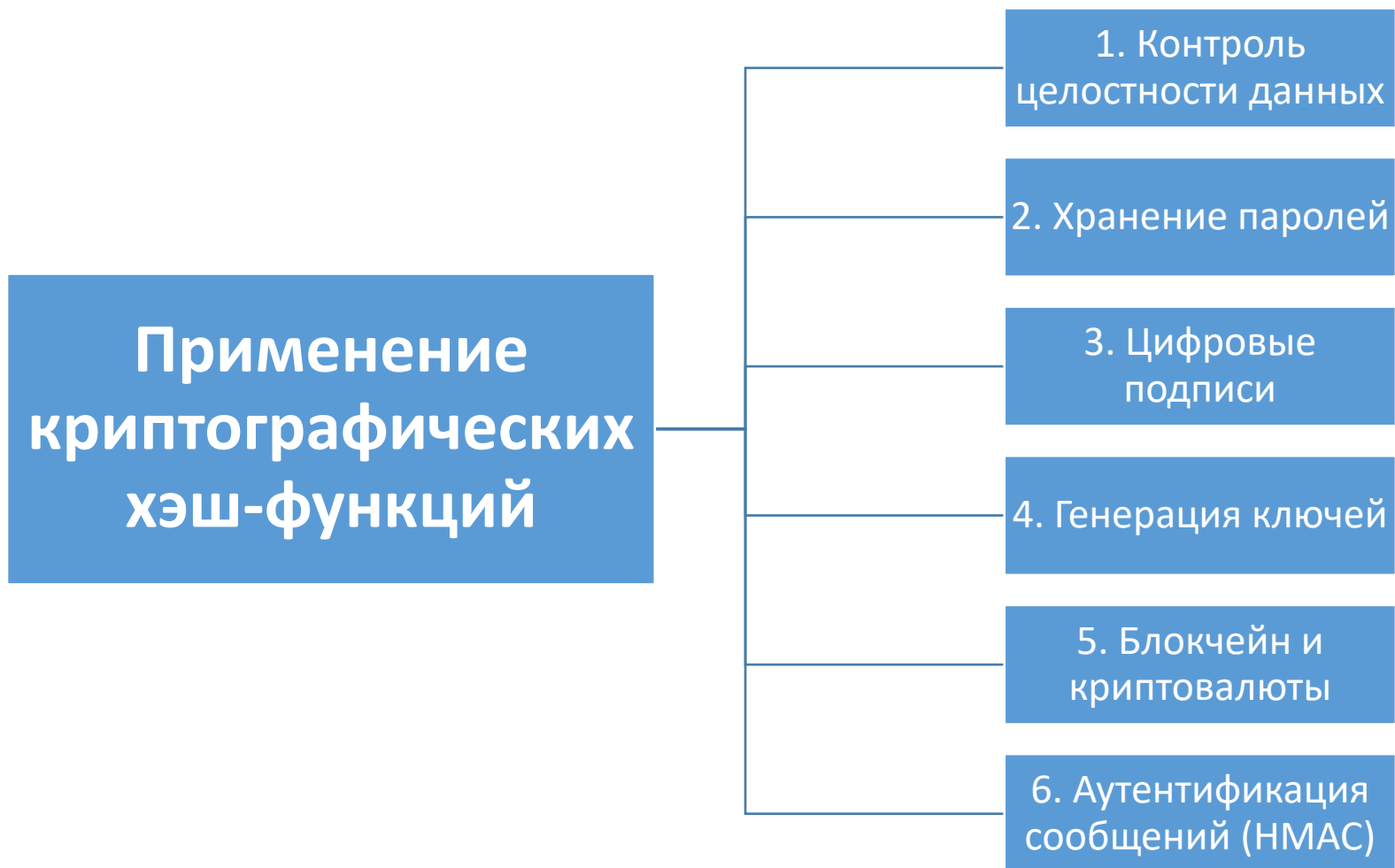
Криптографические хэш-функции

Примеры популярных криптографических хэш-функций

5. BLAKE2

- Современная хэш-функция с высокой скоростью и безопасностью.
- Быстрее SHA-2 и SHA-3.
- Подходит для криптографических приложений, таких как цифровые подписи.

Криптографические хэш-функции



Криптографические хэш-функции

Угрозы и атаки на криптографические хэш-функции

Атака на коллизии

- Злоумышленник пытается найти два разных входных значения с одинаковым хэшем.

2. Атака на прообраз

- Попытка восстановить исходные данные по известному хэшу.

3. Радужные таблицы (Rainbow tables)

- Используются для поиска исходного текста по его хэшу при помощи заранее вычисленных таблиц значений.

4. Брутфорс-атаки

- Перебор всех возможных входных данных для нахождения совпадения с заданным хэшем.

Криптографические хэш-функции

Современные рекомендации по использованию

1. Используйте только современные алгоритмы (например, SHA-2, SHA-3 или BLAKE2).

2. Для хранения паролей применяйте алгоритмы с функцией замедления вычислений, такие как bcrypt, scrypt или Argon2.

3. Регулярно обновляйте криптографическое ПО и следите за новыми уязвимостями.

4. Для проверки целостности используйте HMAC, чтобы защититься от атак типа "человек посередине".

SHA-2 (Secure Hash Algorithm 2)

SHA-2 — это семейство криптографических хэш-функций, разработанных Агентством национальной безопасности США (NSA) и опубликованных в 2001 году Национальным институтом стандартов и технологий (NIST) как часть стандарта FIPS PUB 180-2. SHA-2 является усовершенствованием своего предшественника SHA-1, который был признан уязвимым к криптографическим атакам. SHA-2 считается безопасным и широко используется в современных криптографических приложениях, таких как цифровые подписи, сертификаты и блокчейны.

Основные характеристики SHA-2

1. Многообразие вариантов

2. Длина входных данных

3. Фиксированный размер выходного значения

4. Эффект лавины

5. Устойчивость к атакам

SHA-2 (Secure Hash Algorithm 2)

Структура SHA-2

SHA-2 основан на конструкции **Меркла-Дамгарда** с использованием сжатия данных. Алгоритм работает по следующим этапам:

1. Предварительная обработка данных

- Входные данные выравниваются (padding) до длины, кратной блоку (512 бит для SHA-256 и 1024 бита для SHA-512).
- К данным добавляется бит "1", за которым следуют нули, чтобы дополнить сообщение до нужной длины.
- В конце добавляется длина исходного сообщения в битах.

2. Инициализация начальных переменных

- Алгоритм использует фиксированные начальные значения (IV — Initialization Vector), которые определены стандартом.

3. Обработка данных по блокам

- Сообщение разбивается на блоки фиксированного размера (512 бит или 1024 бита).
- Каждый блок обрабатывается через функцию сжатия, которая использует алгоритмы перестановок, побитовых операций и сложения по модулю 2^{32} или 2^{64} .

4. Функция сжатия

- В основе функции сжатия лежит использование логических операций XOR, AND, OR и циклических сдвигов.

5. Выходное значение

- После обработки всех блоков результирующее значение (хэш) объединяется из внутренних переменных состояния.

SHA-2 (Secure Hash Algorithm 2)

Различия между алгоритмами семейства SHA-2

Алгоритм	Размер блока	Размер хэша	Количество раундов	Используемые операции
SHA-224	512 бит	224 бит	64	Логические операции XOR, AND, OR, сдвиги
SHA-256	512 бит	256 бит	64	Логические операции XOR, AND, OR, сдвиги
SHA-384	1024 бит	384 бит	80	Логические операции XOR, AND, OR, сдвиги
SHA-512	1024 бит	512 бит	80	Логические операции XOR, AND, OR, сдвиги
SHA-512/224	1024 бит	224 бит	80	Логические операции XOR, AND, OR, сдвиги
SHA-512/256	1024 бит	256 бит	80	Логические операции XOR, AND, OR, сдвиги

SHA-2 (Secure Hash Algorithm 2)

Применение SHA-2

1. Цифровые подписи и сертификаты

SHA-2 используется в цифровых подписях (например, в алгоритмах RSA или ECDSA) для проверки подлинности документов.

2. SSL/TLS протоколы

Протоколы HTTPS используют SHA-2 для обеспечения целостности соединения.

3. Хранение паролей

Вместо хранения паролей в открытом виде системы сохраняют их хэши (с добавлением соли).

4. Блокчейн и криптовалюты

В Bitcoin используется SHA-256 для создания блоков и проверки транзакций.

5. Контроль целостности файлов

Хэш-функции применяются для проверки того, что файл не был изменён при передаче или хранении.

6. HMAC (Hash-based Message Authentication Code)

В комбинации с секретным ключом SHA-2 используется для проверки подлинности сообщений.

SHA-2 (Secure Hash Algorithm 2)

Преимущества и недостатки

Преимущества:

- 1. Высокая производительность.
- 2. Устойчивость к известным атакам (коллизии, прообразы).
- 3. Широкая поддержка в программном обеспечении и оборудовании.

Недостатки:

- 1. Высокие требования к вычислительным ресурсам по сравнению с устаревшими алгоритмами (например, MD5).
- 2. Уязвимость к потенциальным квантовым атакам в будущем.

Безопасность SHA-2

На момент октября 2023 года SHA-2 остаётся безопасным для большинства применений. Однако из-за роста вычислительных мощностей и появления квантовых компьютеров в будущем могут появиться уязвимости. Поэтому уже сейчас разрабатываются альтернативы (например, SHA-3 или постквантовые алгоритмы).

BLAKE2

BLAKE2 — это высокоэффективный криптографический хэш-алгоритм, разработанный в 2012 году Жаном-Филиппом Омассоном (Jean-Philippe Aumasson), Самуэлем Невесом (Samuel Neves), Кристианом Виллемсеном (Christian Winnerlein) и Зуко Уилкоксом-О'Хирном (Zooko Wilcox-O'Hearn). Он является улучшенной версией алгоритма BLAKE, который был финалистом конкурса NIST SHA-3 в 2012 году. BLAKE2 сочетает высокую производительность с сильной криптографической стойкостью и используется в широком спектре приложений.

1. Высокая производительность

- BLAKE2 быстрее, чем многие другие криптографические хэш-функции, такие как SHA-2 и SHA-3.
- Он оптимизирован для современных процессоров и поддерживает аппаратное ускорение.

2. Криптографическая стойкость

- BLAKE2 обеспечивает защиту от известных атак, таких как атаки на коллизии и атаки на прообразы.
- Использует конструкцию на основе хаотических преобразований (ChaCha-like permutation).

3. Универсальность

- Поддерживает различные размеры выходного хэша (от 1 до 512 бит).
- Может использоваться как стандартная хэш-функция, а также для HMAC (Hash-based Message Authentication Code) и KDF (Key Derivation Function).

4. Простота реализации

- Алгоритм легко реализуется как в программном, так и в аппаратном обеспечении.
- Не требует таблиц или сложных предварительных вычислений.

5. Отсутствие необходимости в таблицах S-box

- В отличие от некоторых других алгоритмов, таких как MD5 или SHA-1, BLAKE2 не использует таблицы замены (S-box), что делает его менее подверженным атакам на основе анализа таблиц.

6. Встроенная поддержка параметров

- Поддерживает настройку параметров, таких как длина ключа, длина выходного значения, соль и персонализация.

BLAKE2

Разновидности BLAKE2

Существует две основные версии BLAKE2, каждая из которых предназначена для различных сценариев использования:

1. BLAKE2b

- Оптимизирован для 64-битных архитектур.
- Производит хэш длиной до 64 байт (512 бит).
- Используется в приложениях, требующих высокой производительности на 64-битных системах (например, серверы, блокчейн).

2. BLAKE2s

- Оптимизирован для 8-, 16- и 32-битных архитектур.
- Производит хэш длиной до 32 байт (256 бит).
- Подходит для встроенных систем и мобильных устройств.

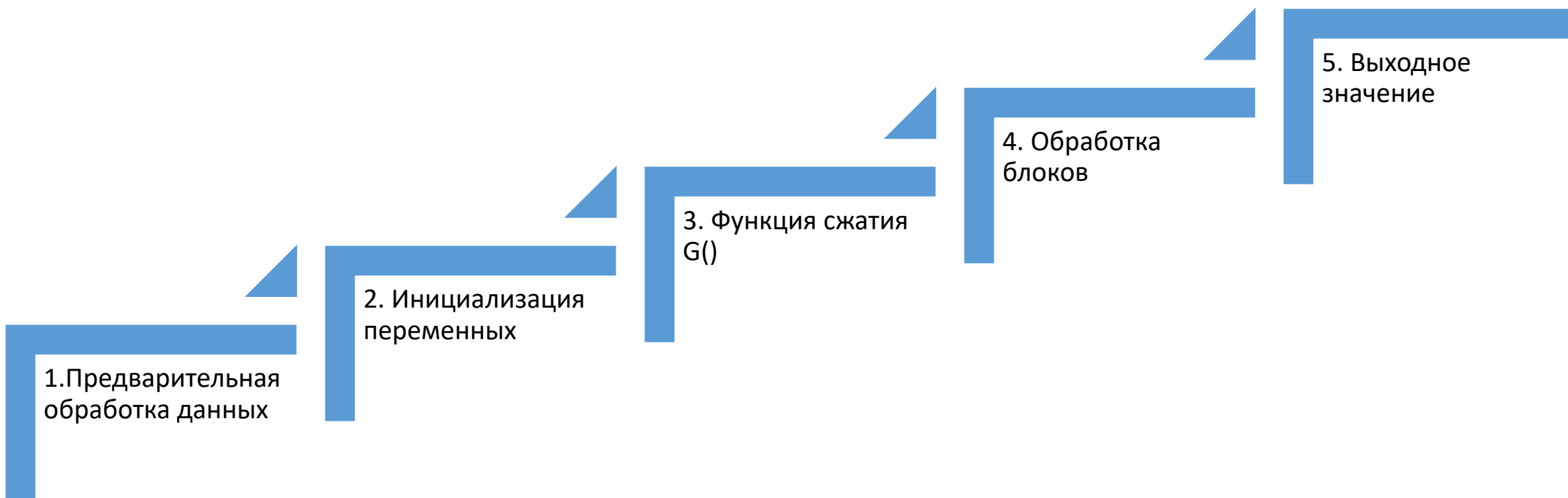
Кроме того, существуют специализированные версии:

- BLAKE2bp: Параллельная версия BLAKE2b.
- BLAKE2sp: Параллельная версия BLAKE2s.
- BLAKE3: Более новая версия, основанная на BLAKE2, с улучшенной производительностью и параллелизмом.

BLAKE2

Структура и работа алгоритма BLAKE2

Алгоритм BLAKE2 основан на конструкции Меркла-Дамгарда с использованием функции сжатия $G()$, которая является модификацией функции ChaCha. Основные этапы работы:



BLAKE2

Параметры настройки BLAKE2

Алгоритм позволяет использовать дополнительные параметры для настройки:

1. Длина выходного значения: От 1 до 64 байт (для BLAKE2b) или от 1 до 32 байт (для BLAKE2s).
2. Ключ: Может быть указан секретный ключ для использования алгоритма в режиме HMAC.
3. Соль: Позволяет добавить случайность к хэшу.
4. Персонализация: Позволяет настроить алгоритм под конкретное приложение или пользователя.

BLAKE2

Преимущества BLAKE2

- 1. Быстрее SHA-256 и SHA-3
 - • За счёт оптимизации под современные процессоры.
- 2. Гибкость
 - • Возможность настройки параметров под конкретные задачи.
- 3. Простота реализации
 - • Отсутствие сложных структур, таких как S-box.
- 4. Криптографическая стойкость
 - • Устойчивость к коллизиям и другим криптографическим атакам.
- 5. Широкое применение
 - • Используется в приложениях шифрования, проверки целостности файлов, цифровых подписей и блокчейнах.

Недостатки BLAKE2

- 1. Относительно новая технология
 - • Несмотря на её преимущества, она менее проверена временем по сравнению с SHA-256.
- 2. Ограниченная поддержка аппаратного ускорения
 - • Хотя алгоритм оптимизирован для современных процессоров, он не так широко поддерживается аппаратно, как SHA-256.

BLAKE2

Применение BLAKE2

1. Проверка целостности файлов

- Используется для создания контрольных сумм файлов.

2. Системы аутентификации

- Может использоваться для хранения хэшей паролей или генерации HMAC.

3. Блокчейн

- Применяется в некоторых криптовалютах для создания блоков или проверки транзакций.

4. Криптографические библиотеки

- Включён в популярные библиотеки, такие как OpenSSL, libsodium и многие другие.

5. Встроенные системы

- Благодаря своей эффективности используется в микроконтроллерах и IoT-устройствах.

Соль в хэш-функции

Соль — это случайная строка данных, которая добавляется к входным данным перед их хэшированием. Основная цель соли — повысить безопасность хэша, особенно в контексте хранения паролей, чтобы защитить их от атак, таких как атаки по словарю или атаки с использованием радужных таблиц. Соль делает каждый хэш уникальным даже при одинаковых входных данных.



Соль в хэш-функции

Почему нужна соль?

Хэш-функции преобразуют входные данные фиксированной длины в выходное значение фиксированной длины (хэш). Если два пользователя используют одинаковый пароль, то без соли их хэши будут одинаковыми. Это создаёт уязвимость:

1. Использование радужных таблиц

- Радужные таблицы — это заранее вычисленные таблицы хэшей для множества возможных паролей. Если злоумышленник получает доступ к базе данных хэшей, он может легко найти совпадения в таблице и восстановить пароли.

2. Атаки по словарю

- Злоумышленник может использовать предсказуемые пароли (например, "123456", "password") и быстро проверить их против базы хэшей.

Добавление соли решает эти проблемы, так как делает каждый хэш уникальным, даже если входные данные (например, пароли) одинаковы.

Соль в хэш-функции

Как работает соль?

1. Генерация соли

- Для каждого набора входных данных (например, пароля) генерируется случайная строка фиксированной длины.
- Соль должна быть уникальной для каждого входного значения и достаточно длинной, чтобы затруднить предсказание.

2. Добавление соли к данным

- Соль объединяется с исходными данными перед хэшированием. Например:

$$\text{salted_input} = \text{salt} + \text{password}$$

3. Хэширование с солью

- Результирующее значение (соль + данные) передаётся в хэш-функцию:

$$\text{hash} = H(\text{salt} + \text{password})$$

4. Сохранение соли

- Соль сохраняется вместе с хэшем (например, в базе данных), чтобы её можно было использовать для проверки подлинности позже.

Соль в хэш-функции

Пример использования соли

```
import hashlib
import os

# Генерация случайной соли
salt = os.urandom(16) # 16 байт случайной соли

# Пароль пользователя
password = "mypassword"

# Добавление соли к паролю и хэширование
salted_password = salt + password.encode()
hashed_password = hashlib.sha256(salted_password).hexdigest()

print(f"Соль: {salt.hex()}")
print(f"Хэшированный пароль: {hashed_password}")
```

Соль в хэш-функции

Ключевые свойства соли

1. Уникальность:

- Каждая соль должна быть уникальной для каждого пользователя или каждого хэша. Это предотвращает возможность повторного использования радужных таблиц.

2. Случайность:

- Соль должна быть случайной и непредсказуемой. Использование предсказуемых значений (например, имени пользователя или даты) снижает безопасность.

3. Длина:

- Длина соли должна быть достаточно большой, чтобы исключить возможность перебора всех возможных значений. Обычно используется длина от 16 до 32 байт.

4. Хранение:

- Соль должна храниться вместе с хэшем, чтобы можно было проверить подлинность данных позже.

Соль в хэш-функции

Ошибки при использовании соли

1. Повторное использование одной и той же соли:

- Если одна и та же соль используется для всех пользователей или данных, это снижает безопасность системы.

2. Короткая или предсказуемая соль:

- Например, использование даты или имени пользователя в качестве соли делает систему уязвимой для атак.

3. Отсутствие соли:

- Если соль не используется вообще, система становится уязвимой для радужных таблиц и атак по словарю.

Соль в хэш-функции

Сравнение с другими методами защиты

Метод	Защита от радужных таблиц	Защита от атак по словарю	Уникальность
Без соли	Нет	Нет	Нет
Соль	Да	Да	Да
Перец (<i>pepper</i>)	Частично	Частично	Нет
Соль + Перец	Да	Да	Да

Перец — это дополнительный секретный ключ (shared secret), который добавляется ко всем данным перед их хэшированием. В отличие от соли, перец обычно хранится отдельно от базы данных.

Радужные таблицы

Радужные таблицы — это предвычисленные таблицы, которые используются для нахождения исходных данных (например, паролей) по их хэш-значениям. Они представляют собой оптимизированный способ выполнения атак на хэш-функции, особенно в контексте взлома паролей.

Радужные таблицы — это компромисс между временем и памятью. Вместо того чтобы хранить все возможные пары "входные данные — хэш" (как в случае с прямым словарём), радужные таблицы используют специальные методы для уменьшения объёма памяти, необходимой для хранения данных, сохраняя при этом возможность эффективного восстановления исходных данных.

- Основная идея:

Радужные таблицы используют цепочки хэшей и так называемые **функции редукции**, чтобы сжать информацию о большом количестве возможных входных данных. Это позволяет злоумышленникам быстро находить соответствия между хэшами и их исходными значениями.

Радужные таблицы

Как работают радужные таблицы?

1. Создание радужной таблицы

- Выбирается диапазон возможных входных данных (например, пароли длиной от 6 до 8 символов).
- Для каждого возможного входного значения вычисляется его хэш с помощью выбранной хэш-функции (например, MD5 или SHA-256).
- Затем применяется **функция редукции**, которая преобразует хэш обратно в формат, похожий на исходные данные (например, строку, похожую на пароль). Это не обратное преобразование хэша, а лишь способ генерации нового значения, которое можно снова захэшировать.
- Этот процесс повторяется несколько раз, создавая цепочку значений:
$$password1 \rightarrow hash1 \rightarrow reduced_value1 \rightarrow hash2 \rightarrow reduced_value2 \rightarrow \dots \rightarrow hashN$$
- В таблице сохраняются только начальное значение цепочки (**password1**) и конечное значение (**hashN**).

Радужные таблицы

Как работают радужные таблицы?

2. Использование радужной таблицы

- Злоумышленник получает доступ к базе данных с хэшами (например, в результате утечки).
- Он пытается найти совпадение между хэшами из базы данных и конечными значениями цепочек в радужной таблице.
- Если совпадение найдено:
 - Злоумышленник восстанавливает цепочку от конечного значения к начальному.
 - Процесс повторяется до тех пор, пока не будет найдено исходное значение (пароль), соответствующее хэшу.

Радужные таблицы

Пример работы радужной таблицы

1. Создание цепочки

Предположим, что мы используем простую хэш-функцию $H(x)$ и функцию редукции $R(x)$:

- Входное значение: "abc"
- Хэш: $H(\text{"abc"}) = 12345$
- Редукция: $R(12345) = \text{"def"}$

Цепочка может выглядеть так:

"abc" $\rightarrow$ 12345 $\rightarrow$ "def" $\rightarrow$ 67890 $\rightarrow$ "ghi" $\rightarrow$ 11223 $\rightarrow$...

В таблице сохраняются только начальное значение "abc" и конечное значение "ghi".

Радужные таблицы

Пример работы радужной таблицы

2. Атака с использованием таблицы

Предположим, что злоумышленник имеет хэш 11223 и хочет найти соответствующий пароль. Он находит совпадение с конечным значением цепочки "ghi" в радужной таблице. Затем он восстанавливает цепочку назад:

"ghi" $\leftarrow$ 11223 $\leftarrow$ "def" $\leftarrow$ 67890 $\leftarrow$ "abc"

Исходное значение "abc" — это искомый пароль.

Радужные таблицы

Преимущества радужных таблиц

1. Оптимизация памяти:

- Радужные таблицы требуют меньше памяти по сравнению с прямыми словарями, так как они используют цепочки вместо хранения всех пар "входные данные — хэш".

2. Быстрота атаки:

- После создания таблицы поиск подходящего значения для заданного хэша выполняется быстро.

3. Универсальность:

- Таблицы можно использовать многократно для взлома различных баз данных, если они используют одну и ту же хэш-функцию.

Радужные таблицы

Ограничения радужных таблиц

1. Размер таблиц:

- Несмотря на оптимизацию памяти, радужные таблицы всё равно могут быть очень большими, особенно для сложных паролей или длинных входных данных.

2. Не подходят для защищённых систем:

- Если система использует соль (salt), радужные таблицы становятся бесполезными, так как каждый хэш становится уникальным.

3. Сложность создания:

- Создание радужной таблицы требует значительных вычислительных ресурсов и времени.

4. Зависимость от хэш-функции:

- Таблицы подходят только для одной конкретной хэш-функции. Если система использует другую хэш-функцию (например, SHA-256 вместо MD5), то существующие таблицы становятся бесполезными.

Применение хэш-функций в электронной подписи

Электронная подпись (ЭП) — это криптографический механизм, обеспечивающий подлинность, целостность и неотказуемость электронных данных. Хэш-функции играют ключевую роль в процессе создания и проверки электронной подписи, поскольку они значительно уменьшают объем данных для обработки и гарантируют неизменность подписываемого сообщения.

Роль хэш-функций в электронной подписи

1. Сжатие данных:

- Документы или сообщения могут быть большого размера, что делает их прямое шифрование (например, с использованием закрытого ключа) неэффективным.
- Хэш-функция преобразует данные в фиксированный по размеру хэш (хэш-сумму), который затем подписывается.

2. Гарантия целостности:

- Хэш-функции обладают свойством *чувствительности к изменениям*: даже небольшое изменение исходного сообщения приводит к полностью отличному хэшу.
- Это позволяет проверить, что данные не были изменены после их подписания.

3. Ускорение процесса:

- Вычисление хэша гораздо быстрее, чем обработка всего документа с помощью криптографических алгоритмов.
- Это делает процесс создания и проверки подписи более эффективным.

4. Универсальность:

- Хэш-функции стандартизированы и могут использоваться с различными алгоритмами шифрования (например, RSA, DSA, ECDSA).

Применение хэш-функций в электронной подписи

Процесс создания электронной подписи с использованием хэш-функции

1. Генерация хэша сообщения

- Исходное сообщение (документ, файл или текст) передается через хэш-функцию (например, SHA-256).
- Хэш-функция вычисляет уникальный фиксированный по размеру хэш (хэш-сумму) для данного сообщения:

$$hash = H(message)$$

2. Шифрование хэша

- Полученный хэш шифруется с использованием закрытого ключа отправителя (асимметричная криптография). Это создаёт электронную подпись:

$$signature = Encrypt(hash, private_key)$$

- Шифрование может выполняться с использованием алгоритмов RSA, DSA или других.

3. Отправка данных

- Отправитель передает получателю:
- Оригинальное сообщение,
- Электронную подпись.

Применение хэш-функций в электронной подписи

Процесс проверки электронной подписи

1. Генерация хэша из сообщения

- Получатель пропускает полученное сообщение через ту же хэш-функцию (например, SHA-256), чтобы получить хэш:

$$\text{hash_from_message} = H(\text{message})$$

2. Расшифровка подписи

- Получатель расшифровывает электронную подпись с использованием открытого ключа отправителя:

$$\text{decrypted_hash} = \text{Decrypt}(\text{signature}, \text{public_key})$$

3. Сравнение хэшей

- Сравнивается хэш из сообщения (hash_from_message) с расшифрованным хэшем (decrypted_hash):

if hash_from_message == decrypted_hash:

Подпись действительна

else:

Подпись недействительна

Если хэши совпадают, это означает:

- Сообщение не было изменено (гарантия целостности).
- Подпись была создана отправителем, так как только он владеет закрытым ключом (гарантия подлинности и неотказуемости).

Применение хэш-функций в электронной подписи

Свойства хэш-функций, важные для электронной подписи

Для обеспечения безопасности электронной подписи используются криптографические хэш-функции с определёнными свойствами:

1. Односторонность:

- Невозможно восстановить исходное сообщение из его хэша.

2. Стойкость к коллизиям:

- Невозможно найти два различных сообщения, которые дают одинаковый хэш.

3. Чувствительность к изменениям:

- Даже минимальное изменение входных данных приводит к кардинально отличному хэшу.

4. Быстрота вычислений:

- Хэш-функция должна быть быстрой для обработки больших объёмов данных.

Примеры популярных криптографических хэш-функций: SHA-256, SHA-3, SHA-512.

Применение хэш-функций в электронной подписи

Преимущества использования хэш-функций в ЭЦП



Стандарты хэш-функций в РФ

В Российской Федерации применяются собственные криптографические стандарты, разработанные в соответствии с требованиями Федеральной службы по техническому и экспортному контролю (ФСТЭК России) и Федеральной службы безопасности (ФСБ России). Хэш-функции, используемые в РФ, регламентируются стандартами семейства ГОСТ. Рассмотрим их подробнее

1. ГОСТ Р 34.11-94 (Устаревший стандарт)

| Описание:

Первый отечественный стандарт хэширования, утверждённый в 1994 году. Он был основан на алгоритме хэширования, аналогичном американскому стандарту MD5 и SHA-1. Этот стандарт использовался для создания электронной цифровой подписи (ЭЦП) и других криптографических целей.

| Основные характеристики:

- Размер хэша: 256 бит (32 байта).
- Блоки входных данных: Размер входного блока — 256 бит.
- Принцип работы: Алгоритм основан на итеративном преобразовании входных данных с использованием нелинейных замен и перестановок.

| Проблемы:

- С развитием криптоанализа были найдены уязвимости, которые делают ГОСТ Р 34.11-94 недостаточно безопасным для современных задач.
- В связи с этим в 2012 году был принят новый стандарт — ГОСТ Р 34.11-2012.

Стандарты хэш-функций в РФ

2. ГОСТ Р 34.11-2012 ("Стрибог")

| Описание:

Этот стандарт заменил устаревший ГОСТ Р 34.11-94 и является современным российским стандартом криптографических хэш-функций. Алгоритм "Стрибог" был разработан с учетом современных требований к стойкости хэш-функций и является частью пакета российских криптографических стандартов.

ГОСТ Р 34.11-2012 соответствует международным требованиям и может использоваться для защиты информации, включая создание электронной подписи, контроль целостности данных и другие задачи.

| Основные характеристики:

1. Длина хэша:

- 256 бит (32 байта).
- 512 бит (64 байта).

2. Размер блока данных:

- Алгоритм обрабатывает данные блоками по 512 бит (64 байта).

3. Принцип работы:

- Алгоритм построен на основе итеративного хэширования.
- Используются нелинейные преобразования (S-блоки), перестановки и сложение по модулю 2.
- Включает механизм "добавления длины сообщения", что предотвращает атаки на добавление данных (length extension attack).

Стандарты хэш-функций в РФ

4. Свойства:

- Стойкость к коллизиям (сложность нахождения двух сообщений с одинаковым хэшем).
- Стойкость к нахождению прообраза (невозможность восстановления исходного сообщения по хэшу).
- Устойчивость к атаке "дней рождения" благодаря длине хэша.

5. Производительность:

- Алгоритм оптимизирован для работы как на программных, так и на аппаратных платформах.

6. Математическая основа:

- Использует арифметику конечных полей и нелинейные преобразования.
- S-блоки в алгоритме заимствованы из алгоритма шифрования ГОСТ Р 34.12-2015 ("Кузнечик").

| Применение:

- Электронная подпись.
- Контроль целостности данных.
- Генерация ключей.
- Хранение паролей.

| Преимущества:

- Соответствие современным требованиям безопасности.
- Высокая производительность на современных процессорах.
- Возможность выбора длины хэша (256 или 512 бит).

Стандарты хэш-функций в РФ

| 3. ГОСТ Р 34.11-2018

В 2018 году был принят новый стандарт, который является адаптацией ГОСТ Р 34.11-2012 для международного использования. Он входит в семейство стандартов ISO/IEC и обозначается как ISO/IEC 10118-3:2018.

Сравнение российских стандартов с международными

Характеристика	ГОСТ Р 34.11-94	ГОСТ Р 34.11-2012 ("Стрибог")	SHA-256	SHA-3
Длина хэша	256 бит	256 или 512 бит	256 бит	224, 256, 384, 512 бит
Размер блока	256 бит	512 бит	512 бит	Переменный
Год разработки	1994	2012	2001	2015
Стойкость к коллизиям	Уязвим	Высокая	Высокая	Очень высокая
Производительность	Средняя	Высокая	Высокая	Средняя

Стандарты хэш-функций в РФ

Особенности применения российских хэш-функций

1. Законодательное регулирование:

- • В соответствии с Федеральным законом № 149-ФЗ "Об информации, информационных технологиях и о защите информации", для защиты информации в РФ должны использоваться сертифицированные криптографические средства.
- • Хэш-функции ГОСТ являются обязательными при работе с государственной тайной, конфиденциальной информацией и в системах ЭЦП.

2. Интеграция с другими стандартами:

- • Хэш-функции ГОСТ используются совместно с другими российскими криптографическими стандартами, такими как:
- • • ГОСТ Р 34.10-2012 (электронная подпись на основе эллиптических кривых);
- • • ГОСТ Р 34.12-2015 ("Кузнечик" — блочный шифр).

3. Совместимость:

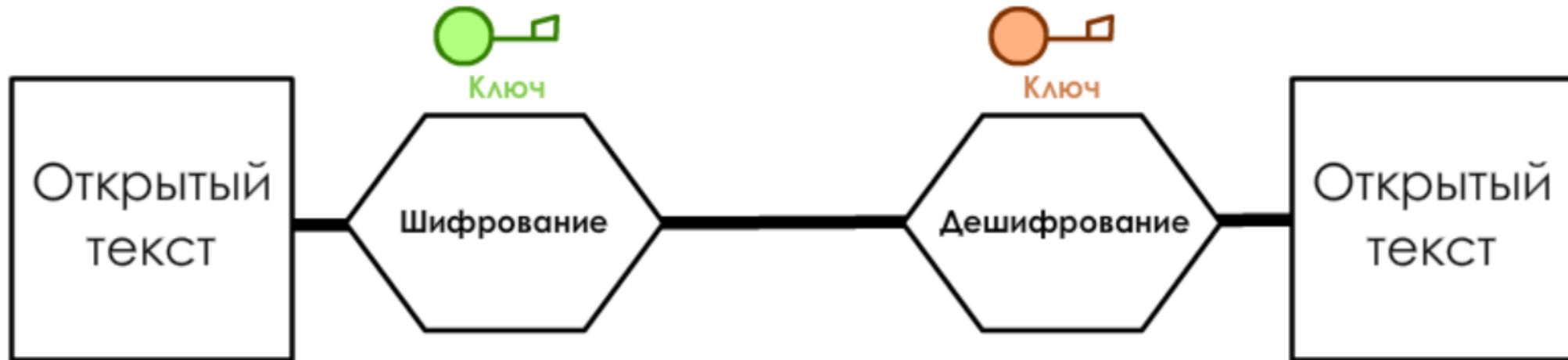
- • Для интеграции российских систем с международными используется адаптация стандартов ГОСТ под международные требования (например, ISO/IEC).

4. Аппаратная реализация:

- • Алгоритмы "Стрибог" оптимизированы для реализации на аппаратном уровне, что делает их эффективными для использования в смарт-картах, токенах и других устройствах.

Асимметричное шифрование

Суть **шифрования с открытым ключом** заключается в том, что для шифрования данных используется один ключ, а для расшифрования другой (поэтому такие системы часто называют **асимметричными**).



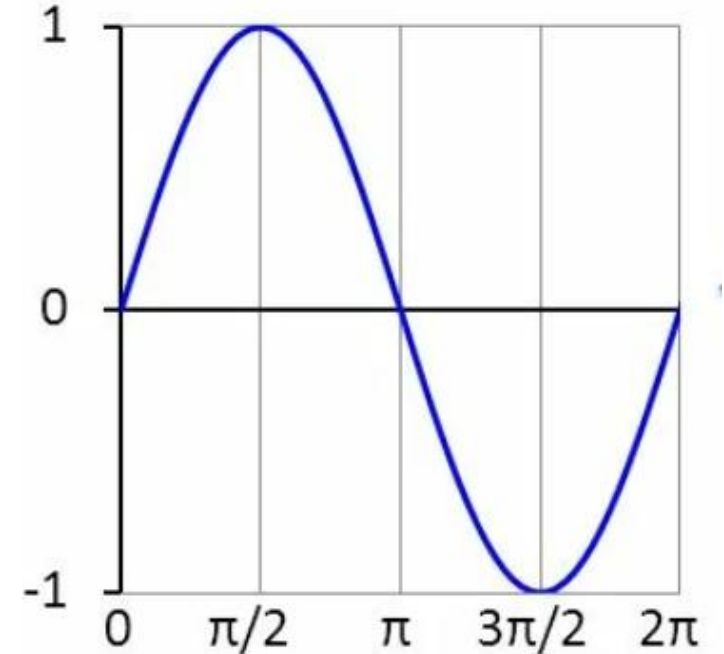
Асимметричное шифрование

Алгоритмы шифрования с открытым ключом используют **необратимые или односторонние функции**.

Эти функции обладают следующим свойством:

при заданном значении аргумента x относительно просто вычислить значение функции (x) , однако, если известно значение функции $y = f(x)$, то нет простого пути для вычисления значения аргумента x .

Например, функция **SIN**. Зная x , легко найти значение **SIN(x)** (например, $x = '3.14'$, тогда **SIN('3.14')** = 0). Однако, если **SIN(x)** = 0, однозначно определить x нельзя, т.к. в этом случае x может быть любым числом, определяемым по формуле $i * '3.14'$, где i – целое число.



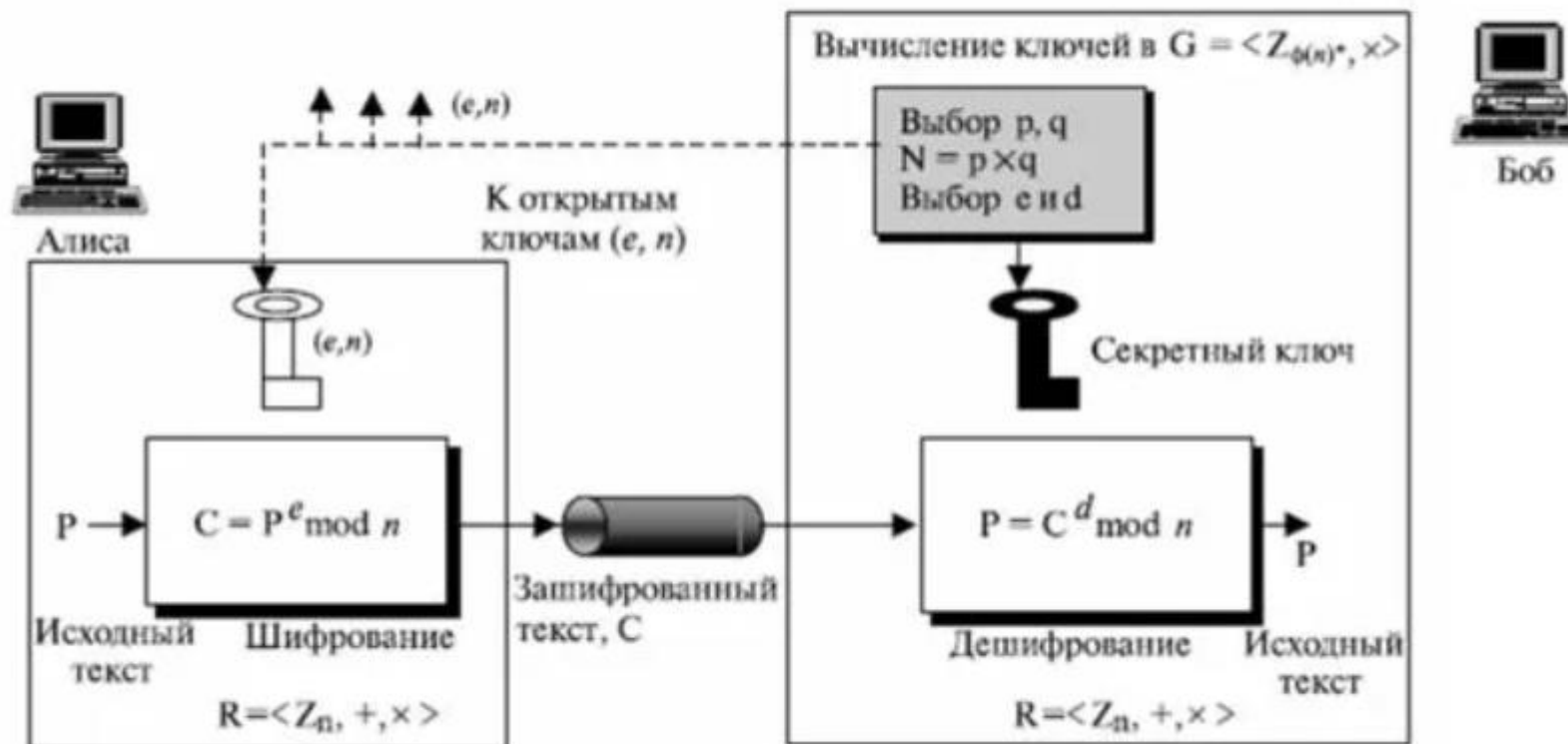
Асимметричное шифрование

Два важных и очевидных требования:

1. Преобразование исходного текста должно быть условно необратимым и исключать его восстановление на основе открытого ключа.
2. Определение закрытого ключа, на основе открытого также должно быть невозможным на современном технологическом уровне.

RSA

Стойкость **RSA** основывается на большой вычислительной сложности известных алгоритмов разложения произведения простых чисел на сомножители.



RSA

№ п/п	Описание операции	Пример
1	Выбираются два простых числа ¹ p и q .	p=7, q=13
2	Вычисляется произведение n = p * q.	n=91
3	Вычисляется функция Эйлера ² φ(n) .	$\phi(n)=(7-1)(13-1)=$ $91-7-13+1 = 72$
4	Выбирается открытый ключ e , как произвольное число ($0 < e < n$), взаимно простое ³ с результатом функции Эйлера ($e \perp \phi(n)$).	e=5
5	Вычисляется секретный ключ d , как обратное число ⁴ к e по модулю φ(n) , из соотношения $(d * e) \bmod \phi(n) = 1$.	$(d * 5) \bmod 72 = 1, d = 29$
6	Публикуются открытый ключ (e , n) в специальном хранилище, где исключается возможность его подмены (общедоступном сертифицированном справочнике).	

Примечания:

- Простое число – натуральное число, большее единицы и не имеющее других делителей, кроме самого себя и единицы.
- Функция Эйлера $\phi(n) = (p-1)*(q-1)$.
- Вычисление **d** производится алгоритмом Евклида таким образом, что $(e*d - 1)$ делится на $\phi(n)$.
- Взаимно простые числа – числа, не имеющие общих делителей, кроме 1
(например, p=3, q=5, n=15, $\phi(n)=8$ – взаимно простые с 15 – 1, 2, 4, 7, 8, 11, 13, 14).

RSA

Процедуры шифрования и дешифрования выполняются по следующим формулам:

$$C = T^e \bmod n,$$

$$T = C^d \bmod n.$$

где T , C – блоки исходного и зашифрованного сообщения.

Открытое сообщение, Т	Символ	А	Б	Р	А	М	О	В
	Код	1	2	18	1	14	16	3
Шифрограмма, $C = T^5 \bmod 91$		1	32	44	1	14	74	61
Открытое сообщение, $T = C^{29} \bmod 91$		1	2	18	1	14	16	3

Пример шифрования по алгоритму RSA

Diffie—Hellman (DH)

Предположим, существует два абонента: Алиса и Боб. Обоим абонентам известны некоторые два числа g и p , которые не являются секретными и могут быть известны также другим заинтересованным лицам. Для того, чтобы создать неизвестный более никому секретный ключ, оба абонента генерируют большие случайные числа: Алиса — число a , Боб — число b . Затем Алиса вычисляет остаток от деления:

$$A = g^a \bmod p \quad (1)$$

и пересылает его Бобу, а Боб вычисляет остаток от деления:

$$B = g^b \bmod p \quad (2)$$

и передаёт Алисе. Предполагается, что злоумышленник может получить оба этих значения, но не модифицировать их (то есть, у него нет возможности вмешаться в процесс передачи).

Diffie—Hellman (DH)

На втором этапе Алиса на основе имеющегося у неё a и полученного по сети B вычисляет значение:

$$B^a \bmod p = g^{ab} \bmod p$$

Боб на основе имеющегося у него b и полученного по сети A вычисляет значение:

$$A^b \bmod p = g^{ab} \bmod p$$

Как нетрудно видеть, у Алисы и Боба получилось одно и то же число:

$$K = g^{ab} \bmod p$$

Его они и могут использовать в качестве секретного ключа, поскольку здесь злоумышленник встретится с практически неразрешимой (за разумное время) проблемой вычисления (1) или (2) по перехваченным $g^a \bmod p$ и $g^b \bmod p$, если числа p, a, b выбраны достаточно большими.

Diffie—Hellman (DH)

При работе алгоритма каждая сторона:

1. генерирует случайное натуральное число a — *закрытый ключ*;

2. совместно с удалённой стороной устанавливает *открытые параметры* p и g (обычно значения p и g генерируются на одной стороне и передаются другой), где p является случайным простым числом, $(p-1)/2$ также должно быть случайным простым числом (для повышения безопасности), g является первообразным корнем по модулю p (также является простым числом)

3. вычисляет *открытый ключ* A , используя преобразование над *закрытым ключом*

$$A = g^a \bmod p$$

5. обменивается *открытыми ключами* с удалённой стороной

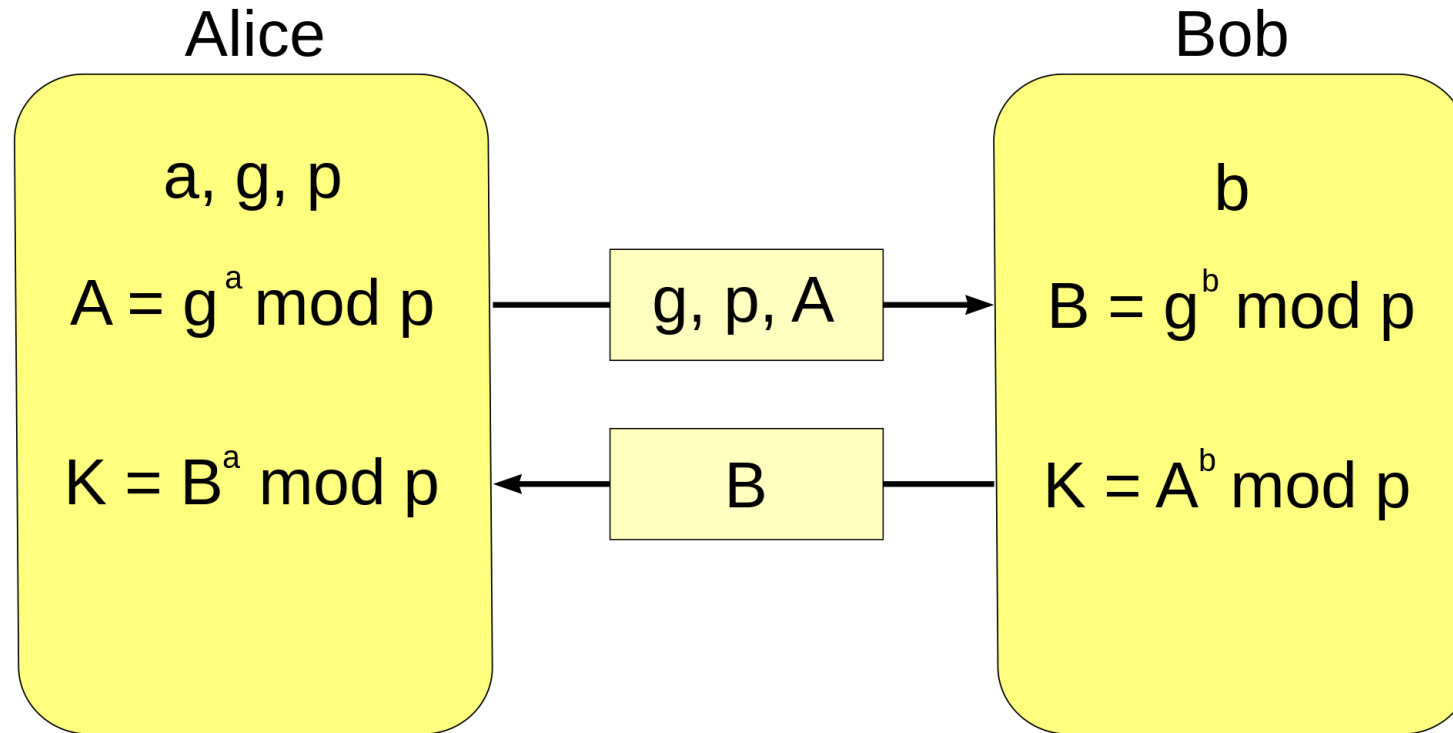
6. вычисляет *общий секретный ключ* K , используя открытый ключ удаленной стороны B и свой закрытый ключ a

$$K = B^a \bmod p$$

K получается равным с обеих сторон, потому что:

$$B^a \bmod p = (g^b \bmod p)^a \bmod p = \mathbf{g^{ab} \bmod p} = (g^a \bmod p)^b \bmod p = A^b \bmod p$$

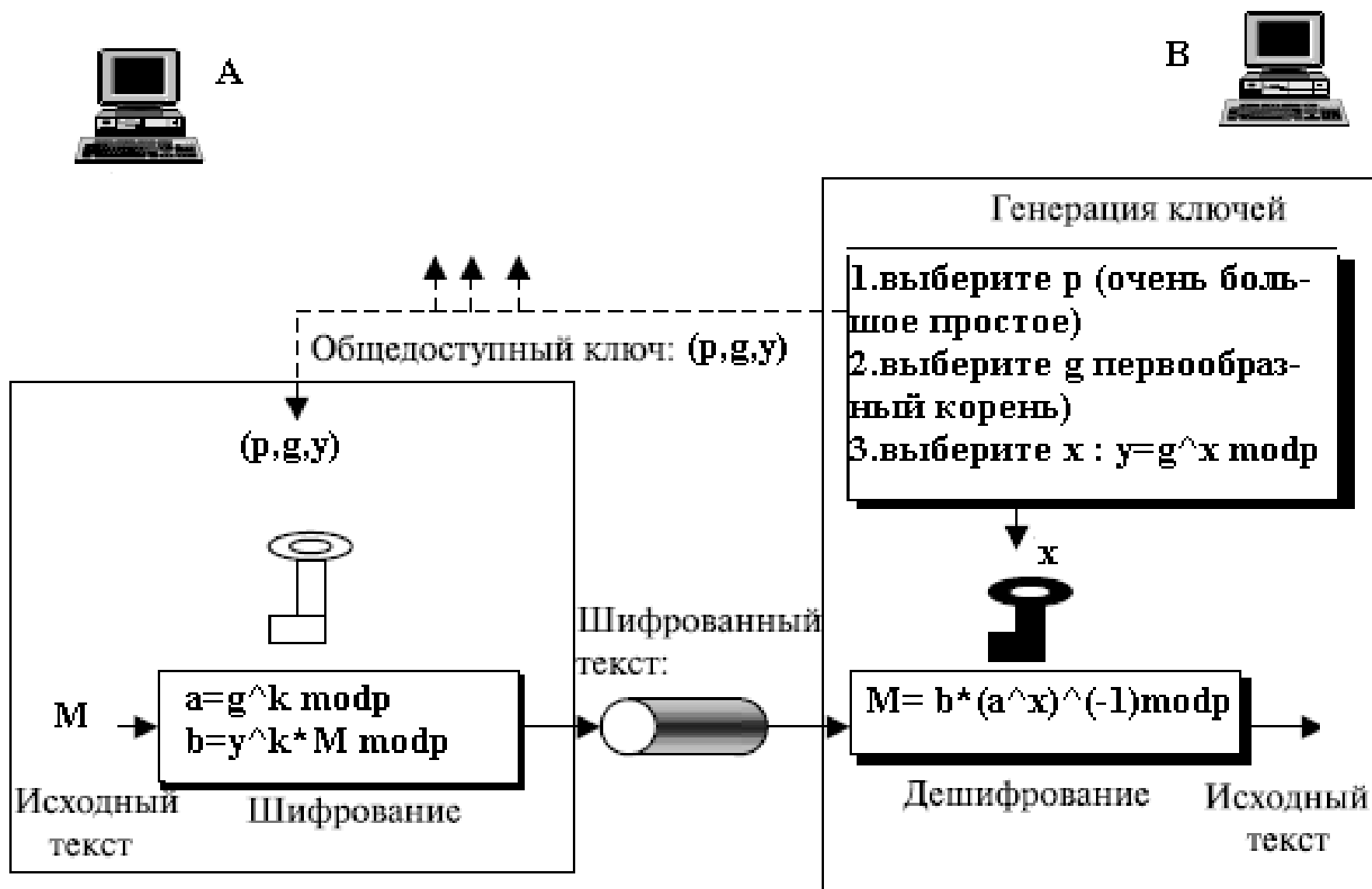
Diffie—Hellman (DH)



$$K = A^b \bmod p = (g^a \bmod p)^b \bmod p = g^{ab} \bmod p = (g^b \bmod p)^a \bmod p = B^a \bmod p$$

В практических реализациях для a и b используются числа порядка 10^{100} и p порядка 10^{300} . Число g не обязано быть большим и обычно имеет значение в пределах первого десятка.

El Gamal



El Gamal

Сложность данного алгоритма базируется на сложности решения задачи дискретного логарифмирования. Суть задачи заключается в следующем. Имеется уравнение

$$g^x \bmod p = y$$

Требуется по известным **g**, **y** и **p** найти целое неотрицательное число **x** (**дискретный логарифм**)

№ п/п	Описание операции	Пример
1	Выбирается простое число p .	$p=37$
2	Выбирается число g , являющееся первообразным корнем по модулю p и меньшее p .	$g=2$
3	Выбираются произвольное число x , меньшее p .	$x=5$
4	Вычисляется $y = g^x \bmod p$	$y = 2^5 \bmod 37 = 32 \bmod 37 = 32$
5	Открытый ключ - y , g и p . Причем g и p можно сделать общими для группы пользователей. Закрытый ключ - x .	

Процедура создания ключей

El Gamal

Для шифрования каждого отдельного блока исходного сообщения должно выбираться случайное число k ($1 < k < p-1$). После чего шифрограмма генерируется по следующим формулам

$$a = g^k \bmod p,$$

$$b = (y^k T) \bmod p,$$

где T – исходное сообщение;
 (a,b) – зашифрованное сообщение.

Дешифрование сообщения выполняется по следующей формуле

$$T = (b (a^x)^{-1}) \bmod p$$

или

$$T = (b a^{p-1-x}) \bmod p,$$

El Gamal

Пример шифрования и дешифрования по алгоритму Эль-Гамала при $k = 7$ приведен в таблице, хотя для шифрования каждого блока (в нашем случае буквы) исходного сообщения надо использовать свое случайное число k .

Первая часть шифрованного сообщения – $a = 5^7 \bmod 23 = 17$.

$a^x = 17^3 = 4913$, $(a^x)^{-1} = 5$ ($4913 * 5 \bmod 23 = 1$) или $a^{p-1-x} = 17^{23-1-3} = 239072435685151324847153$.

Пример шифрования по алгоритму Эль-Гамала (при $k = \text{const}$)

Открытое сообщение, Т	Символ	А	Б	Р	А	М	О	В
	Код	1	2	18	1	14	16	3
Вторая часть шифрограммы, $b = (32^7 * T) \bmod 37$		19	1	9	19	7	8	20
Открытое сообщение, $T = (b * 2) \bmod 37$		1	2	18	1	14	16	3

Протокол обмена ключами («на пальцах»)

1. Абонент **А** обращается в **Центр** и запрашивает сеансовый ключ для связи с абонентом **Б**.
2. В **Центре** создается случайный сеансовый ключ. Зашифровываются две копии этого сеансового ключа – одна с использованием секретного ключа абонента **А**, другая – с использованием секретного ключа абонента **Б**. Затем обе зашифрованные копии пересылаются из **Центра** абоненту **А**.
3. Абонент **А** расшифровывает свою копию сеансового ключа и пересылает вторую зашифрованную копию абоненту **Б**.
4. Абонент **Б** расшифровывает свою копию сеансового ключа.
5. Абоненты **А** и **Б** используют полученный сеансовый ключ для секретного обмена информацией.

